Mondragon Unibertsitatea

Escuela Politécnica Superior

TFM

TRABAJO DE FIN DE MÁSTER

Hacia la operativización de modelos IA aplicados en el ámbito financiero

Unai Elorriaga Aramburu

2025-2026

Máster en Análisis de Datos, Ciberseguridad y Computación en la Nube

TRABAJO DE FIN DE MÁSTER TRABAJO DE FIN DE MÁSTER TRABAJO DE FIN DE MÁSTER TRABAJO DE FIN DE MÁSTER TRABAJO DE FIN DE MÁSTER TRABAJO DE FIN DE MÁSTER

Hacia la operativización de modelos IA aplicados en el ámbito financiero

Trabajo Fin de Máster presentado en: Escuela Politécnica Superior de MU

Para la obtención del título de: Máster en Análisis de Datos, Ciberseguridad y Computación en la Nube

Autor/a: Unai Elorriaga Aramburu

Director/a: Unai Barreiro Buezo

Tutor/a: Oscar Serradilla Casado

Empresa/Organización en la que se ha realizado el proyecto: Caja Laboral Popular Coop. de Crédito

☒ El autor/la autora del Trabajo Fin de Máster autoriza a la Escuela Politécnica Superior de Mondragon Unibertsitatea, con carácter gratuito y con fines exclusivamente de investigación y docencia, los derechos de reproducción y comunicación pública de este documento siempre que: se cite el autor/la autora original, y el uso que se haga de la obra no sea comercial.

☒ **Reconocimiento – NoComercial – CompartirIgual (by-nc-sa)**: No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

DECLARACION DE ORIGINALIDAD, AUTORIA Y USO ETICO DE IA

Yo Unai Elorriaga Aramburu

Declaro que este Trabajo de Fin de Máster es original, fruto de mi trabajo personal, y que no ha sido previamente presentado para obtener otro título o calificación profesional. Las ideas, formulaciones, imágenes e ilustraciones tomadas de fuentes ajenas han sido debidamente citadas y referenciadas. En los casos en los que se han utilizado herramientas de IA, su uso ha sido complementario, responsable y bajo validación crítica humana.

Agradecimientos

En primer lugar, agradezco a mis tutores, Oscar Serradilla y Unai Barreiro, por su guía, dedicación y confianza durante todo el proceso. Sus orientaciones y comentarios han sido fundamentales para el desarrollo del trabajo.

También agradezco a Laboral Kutxa y Mondragon Unibertsitatea por facilitar los recursos necesarios y por la oportunidad de desarrollar este proyecto en un entorno profesional real del sector financiero.

Por último, agradezco a mis compañeros del equipo por crear un entorno de aprendizaje enriquecedor durante todo el TFM.

Resumen

Resumen ejecutivo (no técnico). Este trabajo resuelve un problema operativo: pasar modelos de inteligencia artificial de pruebas aisladas a funcionamiento diario con control en banca. La solución ha sido estandarizar el ciclo de vida en Azure, con reglas comunes para entrenar, desplegar, registrar y auditar resultados. El cambio práctico es reducir dependencia de ejecuciones manuales y aumentar la trazabilidad técnica de cada salida. La evidencia principal es la implantación de pipelines y componentes reutilizables en dos casos reales (Chronos y Asset Allocation), junto con una plantilla común para reutilización futura. La principal limitación es que la validación de transferibilidad se realiza en un segundo caso, pero no en un portafolio amplio de modelos en paralelo.

Este Trabajo de Fin de Máster aborda la operativización de modelos de inteligencia artificial en el ámbito financiero mediante prácticas de Machine Learning Operations (MLOps) en entornos cloud. El proyecto, desarrollado en Laboral Kutxa, parte de un contexto con procesos heterogéneos de entrenamiento, despliegue e inferencia, con alta dependencia de tareas manuales y baja trazabilidad extremo a extremo.

La solución propuesta estandariza el ciclo de vida del modelo en Azure Machine Learning (AML). Como caso principal se operativiza Chronos Engine, un modelo que analiza índices de Renta Fija Privada (RFP) y evalúa si están caros o baratos comparándolos con las predicciones del modelo, con entrenamiento, inferencia, backtesting y reporting. En paralelo, se consolida un estándar de productivización basado en plantillas reutilizables y se operativiza Asset Allocation, un modelo de optimización de carteras que propone un balanceo con más o menos riesgo en función del perfil de riesgo seleccionado, como caso de transferencia para validar la reutilización del enfoque en otro problema de negocio.

Como resultados del trabajo, se estructuran 44 pipelines y 25 componentes AML para Chronos Engine; un estándar de plantillas reutilizables con 3 pipelines y 11 componentes; y una validación de transferencia del modelo Asset Allocation con 2 pipelines y 11 componentes para 9 perfiles de cartera. Estas cifras no se interpretan como inventario técnico, sino como evidencia de capacidad operativa: menor dependencia de tareas manuales, mayor consistencia entre entornos y mejor auditabilidad del ciclo completo.

Se espera un rendimiento y una agilidad mayor para la productivización de futuros proyectos debido a la reutilización de plantillas, contratos y mecanismos de gobernanza establecidos. El trabajo es

reproducible porque la solución se apoya en contratos declarativos, pipelines versionados y evidencias de ejecución trazables.

Palabras Clave: MLOps, AML, MLflow (MLflow), Modelos financieros.

Impacto en los Objetivos de Desarrollo Sostenible (Objetivo de Desarrollo Sostenible (ODS)): ODS 9, ODS 8, ODS 16.

Laburpena

Laburpen exekutiboa (ez-teknikoa). Lan honek arazo operatibo bat ebazten du: adimen artifizialeko ereduak proba-inguruneetatik bankuko eguneroko jardunera kontrolarekin eramatea. Irtenbidea Azureko bizi-zikloa estandarizatzea izan da, entrenamendua, hedapena, erregistroa eta auditagarritasuna arau komunekin lotuz. Ondorio praktikoa da eskuzko lanaren mendekotasuna murriztea eta irteeren trazabilitatea handitzea. Froga nagusiak Chronos eta Asset Allocation kasu errealetan egindako ezarpena eta etorkizunerako txantiloi berrerabilgarriak dira. Muga nagusia da transferigarritasuna bigarren kasu bakarrean egiaztatu dela.

Master Amaierako Lan honek finantza arloan aplikatutako adimen artifizialeko ereduen operazionalizazioa lantzen du, cloud inguruneetan MLOps praktikak erabiliz. Proiektua Laboral Kutxa-n garatu da, eta abiapuntuan entrenamendu, hedapen eta inferentziako prozesu heterogeneoak zeuden, eskuzko lanarekiko mendekotasun handiarekin eta muturretik muturrerako trazabilitate mugatuarekin.

Proposatutako irtenbideak ereduaren bizi zikloa estandarizatzen du AML plataforman. Kasu nagusi gisa, Chronos Engine operatibizatu da, Renta Fija Pribatuaren indizeak aztertzen dituen eta ereduaren aurreikuspenekin alderatuta garesti edo merke dagoen ebaluatzen duen eredua, entrenamendu, inferentzia, backtesting eta reporting pipelineekin. Aldi berean, txantiloietan oinarritutako produktibizazio estandar berrerabilgarri bat finkatu da, eta Asset Allocation operatibizatu da, arrisku-profil hautatuaren arabera arrisku handiago edo txikiagoko karteren oreka proposatzen duen kartera optimizazioko eredua, transferentzia kasu gisa ikuspegi bera beste negozio arazo batean balioztatzeko.

Lanaren emaitza gisa, Chronos Enginerako 44 pipeline eta 25 AML osagai egituratu dira; txantiloi berrerabilgarrietan oinarritutako estandar batek 3 pipeline eta 11 osagai ezarri ditu; eta Asset Allocation ereduaren transferentzia 2 pipeline eta 11 osagairekin baliozkotu da 9 arrisku-profilerentzako. Txantiloi horiek Laboral Kutxarentzat Azureko ereduen hedapenerako estandar gisa prestatu dira, kontrol teknikoa eta gobernantza indartzen dituen marko operatibo erreproduzigarri batean.

Etorkizuneko proiektuen produktibizazioan errendimendu eta arintasuna handiagoa espero da, txantiloiak, kontratuak eta gobernantza mekanismoak berrerabiltzeagatik. Lana erreproduzigarria da, exekuzio-ebidentzia eta kontratu deklaratiboak bertsio bidez kontrolatzen direlako.

Hitz gakoak: MLOps, AML, MLflow, Finantza ereduak.

Garapen Jasangarriko Helburuetan (Garapen Jasangarriko Helburua (GJH)) eragina: 9. GJH, 8. GJH, 16. GJH.

Abstract

Executive summary (non-technical). This work addresses an operational gap: moving AI models from isolated experimentation to stable daily use in a banking context. The solution standardizes the lifecycle in Azure with common rules for training, deployment, registration, and auditability. The practical effect is reduced manual dependency and improved traceability of outputs. The main evidence is the deployment in two real use cases (Chronos and Asset Allocation), supported by a reusable template for future projects. The main limitation is that transferability is validated in a second case, but not yet at broad portfolio scale.

This Master's Thesis addresses the operationalization of AI models in the financial domain through MLOps practices in cloud environments. The project, developed at Laboral Kutxa, starts from a context with heterogeneous training, deployment and inference processes, high manual effort, and limited end-to-end traceability.

The proposed solution standardizes the model lifecycle in AML. The primary case is Chronos Engine, a model that analyzes private fixed-income indices and evaluates whether they are expensive or cheap by comparing observed levels with the model predictions, operationalized through training, inference, backtesting and reporting pipelines. In parallel, a reusable template-based productivization standard is consolidated, and Asset Allocation is operationalized as a portfolio optimization model that proposes rebalancing with more or less risk depending on the selected risk profile, used as a transfer case to validate reuse in a different business problem.

As results of this work, 44 pipelines and 25 AML components are structured for Chronos Engine; a reusable template-based standard with 3 pipelines and 11 components is established; and transfer validation of the Asset Allocation model is completed with 2 pipelines and 11 components across 9 portfolio profiles. These templates are prepared to define a standard for model deployment in Azure at Laboral Kutxa, within a reproducible operating framework that strengthens technical control and governance.

Higher efficiency and agility are expected for the productivization of future projects due to the reuse of templates, contracts and governance mechanisms. The work is reproducible because it relies on versioned templates, explicit contracts and execution evidence.

Keywords: MLOps, AML, MLflow, Financial models.

Sustainable Development Goals (ODS) impact: ODS 9, ODS 8, ODS 16.

Índice general

1 Introducción 1

1.1 Laboral Kutxa 1

1.2 Tesorería y Mercados 2

1.3 Contexto 3

1.4 Problemática y motivación 3

1.5 Antecedentes 4

1.5.1 Trabajo previo y aportaciones de la memoria 2024–2025 5

1.5.2 Estado inicial de la plataforma al inicio del TFM 5

1.6 Objetivos 6

1.6.1 Objetivos específicos 6

1.6.2 Competencias tecnicas del TFM y criterio de cumplimiento 7

1.6.3 Competencias transversales y criterio de cumplimiento 8

1.7 Alcance 8

1.8 Planificación 9

2 Estado del arte 11

2.1 Fundamentos de MLOps 11

2.1.1 Roles, responsabilidades y artefactos 13

2.2 Ciclo de vida MLOps 13

2.2.1 Artefactos críticos y versionado 15

2.2.2 Gobernanza y auditoría en MLOps 15

2.2.3 Observabilidad operativa y niveles de servicio 16

2.2.4 Niveles de madurez: De Google a Microsoft 16

2.3 Herramientas y tecnologías relevantes 17

2.3.1 Arquitecturas de referencia en plataformas gestionadas 17

2.4 Metodologias agiles 18

2.5 Plataforma MLOps en Azure 19

2.5.1 Arquitectura de red y seguridad perimetral 21

2.5.2 Azure DevOps y Library 21

2.5.3 Resource Group y recursos base en Azure 21

2.5.4 Azure Datastore y Key Vault 22

2.5.5 Compute Instance y clústeres de cálculo 22
2.5.6 AML: Environments, Components y Pipelines 22
2.6 Buenas prácticas 23
2.6.1 Separación de lógica de negocio y plataforma 23
2.6.2 Modularización y parametrización del código 23
2.6.3 Contratos explícitos y trazabilidad entre componentes 24
2.6.4 Logging y evidencias de ejecución 24
2.6.5 Automatización y control de cambios 24
2.7 Síntesis del estado del arte y gap abordado 24
2.8 Bases Financieras Mínimas 25
2.8.1 CDS y la dinámica de los spreads 25
2.8.2 Perfiles de gestión y control del riesgo 25

3 Marco regulatorio 27
3.1 Normativa y regulaciones aplicables al proyecto 27
3.2 Riesgos del proyecto y plan de contingencia 28
3.3 Evolución normativa y horizonte de cumplimiento 29

4 Desarrollo y operativización de modelos 30
4.1 Metodologias agiles 30
4.2 Chronos Engine: operativización del modelo de pricing RFP 31
4.2.1 Motivación y contexto operativo 31
4.2.2 Cómo se usa en el día a día 31
4.2.3 Modularización y desacoplamiento de la lógica analítica 32
4.2.4 Contratos de datos y validaciones operativas 32
4.2.5 Configuración de infraestructura en Azure Machine Learning 32
4.2.6 Estrategia de despliegue y el dilema de los 12 sectores 33
4.2.7 Orquestación masiva y el reto de los 12 sectores 34
4.2.8 Ventanas operativas y control de ejecución 34
4.2.9 Arquitectura técnica del pipeline y componentes operativos 34
4.2.10 Integración entre Azure DevOps y Azure Machine Learning 35
4.2.11 Modos de fallo y mitigaciones 35
4.2.12 Evidencias de reporting y trazabilidad operativa 39
4.2.13 Integración con MLflow y control total desde Teradata 40
4.3 Estándar de despliegue y productivización 41
4.3.1 Objetivo del template y alcance real 41
4.3.2 Estructura técnica y estándares reutilizables 42
4.3.3 Recursos de Azure ML: pipelines, componentes y artefactos 43
4.3.4 Arquitectura CI/CD basada en YAML 46
4.3.5 Estrategia de pruebas: seguridad y calidad del código 47

4.3.6 Documentación de la clase MLOpsLifeCycle 47
4.3.7 Gobernanza de modelos en Teradata: solutions, model-ids, self-promotion, manual model fallback y wrappers en MLflow 48
4.3.8 Evidencias operativas y trazabilidad 52
4.3.9 Control de cambios en la plantilla 52
4.3.10 Compatibilidad entre proyectos 53
4.4 Asset Allocation: validación de la transferibilidad del enfoque 54
4.4.1 Implementación sobre el estándar reusable 54
4.4.2 Arquitectura paralela para la gestión de perfiles de riesgo 54
4.4.3 Parametrización y configuración de perfiles 56
4.4.4 Contrato de configuración y control de cambios 56
4.4.5 Análisis de la orquestación y lógica de ejecución 56
4.4.6 Despliegues segmentados: DEV, PRE y PRO 59
4.4.7 Incidencias operativas y lecciones aprendidas 61
4.4.8 Conclusión del caso 62
4.4.9 Modelado de Costes de Transacción y Slippage 62

5 Resultados obtenidos 63
5.1 Cumplimiento de objetivos 63
5.1.1 OE1: operativizacion end-to-end de Chronos Engine 64
5.1.2 OE2: estandar reusable y arquitectura CI/CD 65
5.1.3 OE3: transferencia a Asset Allocation 65
5.1.4 OE4: medicion del impacto operativo 65
5.1.5 OE5: migracion a Azure ML SDK v2 66
5.1.6 OE6: gobernanza de modelos e inferencias en Teradata 66
5.2 Competencias alcanzadas 67
5.3 Resultados tangibles tras el desarrollo 67

6 Impacto, sostenibilidad y viabilidad 69
6.1 Implicaciones sobre salud y seguridad en el trabajo 69
6.2 Impacto en los ODS 70
6.3 Impacto social, medioambiental e industrial 71
6.3.1 Impacto en la gobernanza y cultura de datos 72
6.4 Criterios de sostenibilidad operativa a futuro 72

7 Memoria económica 74
7.1 Costes directos observados 74
7.2 Resumen de costes directos 75
7.3 Valoración de viabilidad y ROI 75

8 Conclusiones y trabajo futuro 76

8.1 Conclusiones 76
8.2 Valoración personal y competencias adquiridas 78
8.3 Líneas futuras 78

Bibliografía **83**

A Anexo A. Pliego de condiciones 85
A.1 Objeto del pliego 85
A.2 Descripcion del servicio a desarrollar 85
A.3 Recursos y medios requeridos 85
A.4 Requisitos tecnicos y funcionales 86
A.4.1 Requisitos no funcionales minimos 86
A.5 Pruebas y criterios de aceptacion 87
A.5.1 Pruebas minimas exigibles 87
A.5.2 Criterio general de aceptacion 87
A.6 Condiciones de ejecucion y organizacion 87
A.7 Condiciones legales, seguridad y cumplimiento 88
A.8 Entregables y condiciones de cierre 88

B Anexo B. Documentacion y guias de usuario 89
B.1 Documentacion por proyecto 89
B.1.1 Chronos Engine 89
B.1.2 Asset Allocation 89
B.1.3 Estandar de productivizacion mediante plantillas 89
B.2 Criterio de uso de las guias 90
B.3 Mapa documental por proyecto 90

C Anexo C. Glosario 91
C.1 Terminos clave 91
C.2 Nota de uso 93

Índice de figuras

1.1 Organigrama general de Laboral Kutxa. 2
1.2 Plataforma MLOps corporativa y esquema de entornos. 4
1.3 Cronograma de ejecución del proyecto. 10

2.1 Intersección de disciplinas que conforman MLOps (ML, DevOps y Data Engineering). . 11
2.2 Workflow genérico de un ciclo MLOps [1]. 14

4.1 Pipeline de entrenamiento de Chronos Engine. Se muestran los componentes principales (carga, preprocesado, entrenamiento por ventana, registro y *self-promotion*). Las flechas representan artefactos versionados (`data_path/output_path`) y triggers de promoción. 36
4.2 Pipeline de predicción diaria (inferencias, registro y backtesting). Se muestran los componentes principales (carga, preprocesado, inferencia, registro y reconstrucción histórica) y los puntos de consolidación de resultados. Las flechas representan artefactos versionados y triggers de ejecución. 36
4.3 Carga de inferencias desde Teradata y generación del reporte SumUp. 39
4.4 Relación entre capas funcionales de la plantilla y su despliegue operativo. 43
4.5 Componente de registro de modelo en la plantilla (MLflow + Teradata). 50
4.6 Componente de self-promotion para evaluación y promoción de versiones. 51
4.7 Componente de registro de inferencias en Teradata. 51
4.8 Componente de carga de datos desde Teradata (Select Teradata). 51
4.9 Tubería DevOps de Asset Allocation usando el estándar genérico. 54
4.10 Pipeline de entrenamiento calculando nueve perfiles simultáneamente. 55
4.11 Inferencia paralela de nueve perfiles y consolidación final del reporte. 57
4.12 Un ejemplo de la tabla resultante final que genera la inferencia agregada. 60
4.13 El HTML consolidado que reporta Asset Allocation al terminar todo el flujo. 61

7.1 Análisis de costes acumulados en el grupo de recursos del entorno MLOps. 74
7.2 Detalle de coste del workspace *ml workspace 2* en Azure Cost Analysis (nombres corporativos censurados). 74

Índice de tablas

1.1 Indicadores operativos base y objetivo del proyecto. 4
1.2 Definición SMART, evidencia y aceptación por objetivo específico. 7
1.3 Trazabilidad de planificación: hitos, objetivos y evidencias. 9

2.1 Roles habituales en MLOps y artefactos asociados. 13
2.2 Artefactos mínimos a versionar en un ciclo MLOps. 15
2.3 Mecanismos de gobernanza en MLOps y riesgo mitigado. 15
2.4 Señales mínimas de observabilidad en MLOps. 16
2.5 Comparativa de niveles de madurez MLOps en Google y Microsoft. 17
2.6 Comparativa técnica de plataformas MLOps en entornos enterprise. 18
2.7 Capas funcionales de la plataforma MLOps en Azure. 20
2.8 Lo que resuelve cada pieza de Azure Machine Learning. 23
2.9 Matriz de síntesis: problema, evidencia, decisión y resultado esperado. 25

3.1 Normativa y marcos de referencia: requisito, implicación y evidencia técnica. 28
3.2 Riesgos del proyecto y plan de contingencia (matriz ampliada). 29

4.1 Contratos de datos operativos en Chronos Engine. 32
4.2 Principales piezas de Azure ML montadas para Chronos. 33
4.3 Decisiones clave en Chronos: de la restricción a la consecuencia operativa. 33
4.4 Modos de fallo frecuentes y mitigaciones en Chronos. 37
4.5 Artefactos de reporting en Chronos y su uso operativo. 40
4.6 Taxonomía de las características (features) de Chronos Engine. 40
4.7 Perímetro funcional de la plantilla: qué resuelve y qué no resuelve. 42
4.8 Capas técnicas del estándar de productivización y límite de cambio recomendado. . . . 43
4.9 Pipelines principales del estándar y qué hace cada uno. 44
4.10 Cómo fluyen los archivos entre los componentes. 46
4.11 Catálogo funcional de `MLOpsLifeCycle` y contribución a la gobernanza en Teradata. . . 50
4.12 Evidencias operativas y riesgo mitigado en la plantilla. 52
4.13 Criterios de compatibilidad en la plantilla MLOps. 53
4.14 Parámetros clave del contrato de configuración en Asset Allocation. 56
4.15 Comparativa de transferibilidad: Chronos Engine vs Asset Allocation. 62

5.1 Matriz de objetivos y evidencias del Trabajo de Fin de Máster (TFM). 64
5.2 Comparativa de esfuerzo operativo entre Chronos y Asset Allocation. 66
5.3 Desglose de competencias alcanzadas y evidencias asociadas. 67

6.1 ODS priorizados, metas y relación causal con el proyecto. 70
6.2 Indicadores operativos propuestos para seguimiento de impacto en ODS. 70
6.3 Plan de actuación para mejorar el impacto en ODS. 71
6.4 Prácticas de gobernanza y su impacto organizativo. 72
6.5 Indicadores mínimos para sostenibilidad operativa. 73

7.1 Coste directo observado del proyecto (periodo septiembre–marzo). 75

8.1 Backlog de líneas futuras con viabilidad operativa. 79

A.1 Recursos minimos requeridos por bloque operativo. 86
A.2 Pruebas minimas y evidencia de aceptacion. 87

B.1 Mapa documental de apoyo por proyecto. 90

Glosario de acrónimos y términos

TFM Trabajo de Fin de Máster

ODS Objetivo de Desarrollo Sostenible

GJH Garapen Jasangarriko Helburua

ML Machine Learning

MLOps Machine Learning Operations

IA Inteligencia Artificial

CI Integración Continua

CD Despliegue Continuo

CI/CD Integración Continua/Despliegue Continuo

IRS Interest Rate Swap

JSON JavaScript Object Notation

LOB Large Object

MRM Model Risk Management

RBAC Control de Acceso Basado en Roles

NSG Network Security Group

VPN Virtual Private Network

RGPD Reglamento General de Protección de Datos

AML Azure Machine Learning

SDK Software Development Kit

CLI Command-Line Interface

AIA Artificial Intelligence Act

RMF Risk Management Framework

YAML YAML Ain't Markup Language

ACR Azure Container Registry

MLflow MLflow

SP Stored Procedure

PAA Protective Asset Allocation

BL Black-Litterman

CDS Credit Default Swap

SQL Structured Query Language

CSV Comma Separated Values

E2E End to End

ROI Retorno de la Inversión

OE Objetivo Específico

DEV Entorno de desarrollo

PRE Entorno de preproducción

PRO Entorno de producción

RFP Renta Fija Privada

1. Introducción

Este TFM se plantea para abordar una necesidad operativa específica: pasar de modelos funcionales en entorno analítico a modelos operables de forma estable en producción. Aunque la diferencia pueda parecer limitada en términos conceptuales, en la práctica concentra una parte relevante de los problemas de implantación: dependencias no controladas, ejecuciones manuales, baja trazabilidad y dificultad de reproducibilidad.

El trabajo se desarrolla en Laboral Kutxa, dentro del área de Tesorería y Mercados de Capitales, y se centra en aplicar una estrategia de MLOps sobre AML para mejorar la forma de desplegar, mantener y gobernar modelos financieros [2–4].

Este documento se estructura en ocho secciones dentro de la introducción: *Laboral Kutxa*, *Tesorería y Mercados*, *Contexto*, *Problemática y motivación*, *Antecedentes*, *Objetivos*, *Alcance* y *Planificación*. Esta secuencia permite presentar primero el marco organizativo del proyecto, después el problema y el punto de partida, y finalmente los compromisos y límites de ejecución del TFM.

1.1 Laboral Kutxa

Laboral Kutxa se fundó en 1956 para prestar servicios financieros a los empleados de la empresa Fagor. Desde entonces, ha evolucionado y ahora ofrece una amplia gama de productos y servicios tanto a particulares como a empresas. Estos servicios incluyen cuentas corrientes, cuentas de ahorro, tarjetas de crédito, préstamos, hipotecas, seguros y fondos de inversión.

La empresa tiene su sede en Mondragón, País Vasco, y cuenta con una extensa red de sucursales en toda España. Además, Laboral Kutxa forma parte de la cooperativa MONDRAGON.

La cooperativa de crédito se dedica a satisfacer las necesidades financieras de sus socios y clientes, principalmente particulares y empresas. Integrada en la Corporación MONDRAGON, canaliza su vocación de promoción social mediante el apoyo a entidades corporativas y extiende su compromiso social con el desarrollo económico y sociocultural, especialmente en el País Vasco, teniendo en cuenta la lengua y la cultura vascas (LABORAL Kutxa, s.f.). La siguiente imagen muestra el organigrama de la organización. Esta labor se llevó a cabo en el departamento de tesorería y mercado. [5]

Como se muestra en la Figura 1.1, la estructura organizativa articula las áreas corporativas y el encaje del departamento donde se desarrolló el TFM.

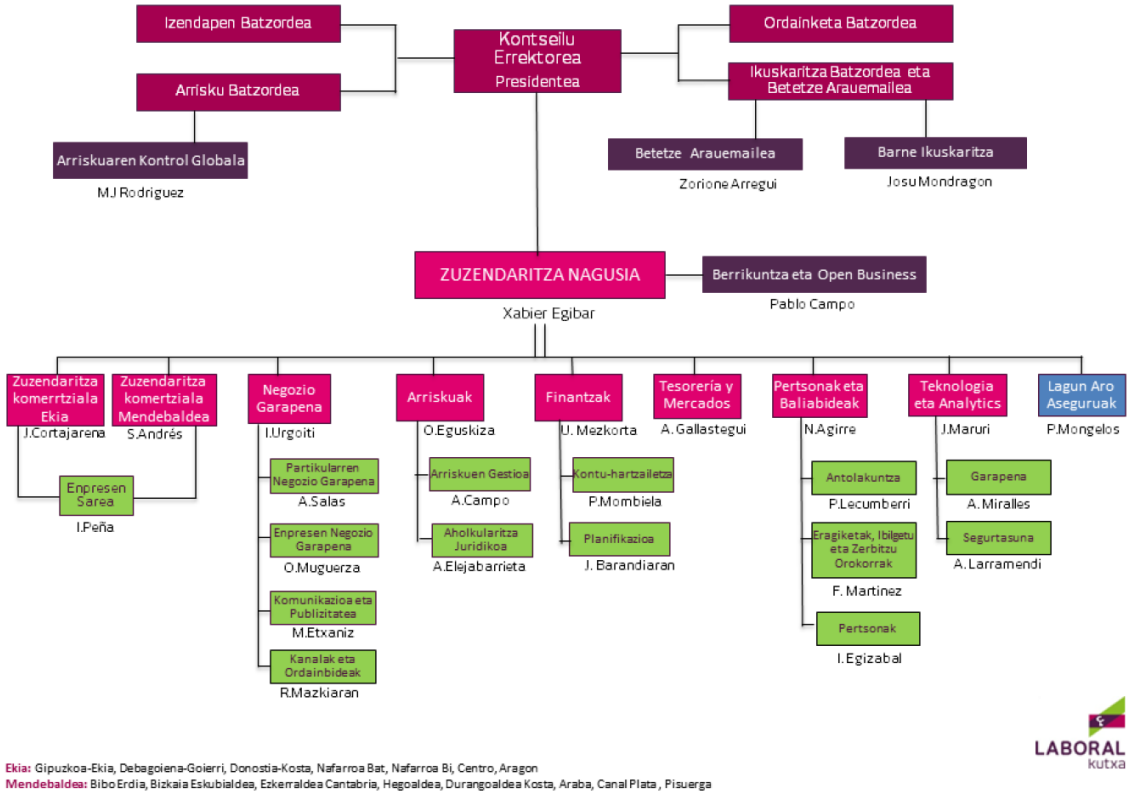

Figura 1.1: *Organigrama general de Laboral Kutxa.*

1.2 Tesorería y Mercados

El Departamento de Tesorería y Mercados, como su nombre indica, se divide en dos subdepartamentos. Las principales funciones de estos departamentos son:

- Gestión de la liquidez a corto plazo en euros y liquidez estructural.
- Cobertura del ratio de caja.
- Gestión de la liquidez a corto plazo en moneda extranjera.
- Gestión del riesgo de balance.
- Gestión de la cobertura de tipos de interés del balance.
- Gestión de carteras en diferentes tipos de activos:
    - Deuda pública.
    - Renta fija privada.
    - Renta variable.
    - Activos privados.
    - Alternativos.
- Contratación de operaciones con clientes:
    - Valores de renta fija.

  - Negociación de divisas.
  - Derivados de tipos de interés.
- Valoración y control del riesgo en fondos garantizados y BPA.
- Gestión de la financiación mayorista a largo plazo y financiación con garantías descontables en el banco central.

Existe un tercer departamento, de nueva creación, que apoya a los otros dos departamentos mediante el desarrollo de inteligencia artificial y modelos cuantitativos. Este trabajo se realizó en esta sección y tiene como objetivo apoyar la gestión de cartera de la tesorería, específicamente en renta fija privada.

1.3 Contexto

En los últimos años, el uso de Inteligencia Artificial (IA) y *machine learning* en sectores regulados, como el financiero, ha aumentado de forma sostenida. Debe distinguirse entre entrenar modelos y operarlos con control en un entorno real. La literatura de MLOps subraya este punto: el valor no reside solo en la métrica del modelo, sino en el ciclo de vida técnico que lo sostiene [1, 6].

En este contexto, Laboral Kutxa necesita que los resultados de sus modelos sean periódicos, auditables y mantenibles en el tiempo. Esto implica disponer de pipelines claros, contratos de entrada/salida bien definidos, versionado de artefactos y un mecanismo consistente de despliegue por entornos. El objetivo operativo es reducir la dependencia de tareas manuales y reforzar la reproducibilidad [2–4].

Con esta base, el TFM se plantea como un trabajo de ingeniería aplicada. El objetivo no es competir por un modelo nuevo, sino mejorar de forma real la capa de productivización para que varios modelos puedan convivir bajo un mismo estándar.

1.4 Problemática y motivación

El problema de negocio puede sintetizarse de la siguiente manera: cuando un proceso depende en exceso de ejecuciones manuales, cualquier cambio menor incrementa el esfuerzo, retrasa la entrega y dificulta la auditoría. En un entorno financiero, esta situación constituye un riesgo operativo.

El problema técnico asociado era la falta de un marco común para productivizar modelos: modularización irregular, diferencias entre proyectos, poca estandarización de pipelines y trazabilidad incompleta entre entrenamiento, registro e inferencia. Esta brecha entre desarrollo y producción es conocida en la industria y hace que muchos modelos no lleguen a operarse de forma sostenible. [7]

La motivación del TFM surge de esta necesidad: establecer una forma de trabajo aplicable de manera sostenida y no limitada a un caso puntual. En términos académicos, el enfoque resulta pertinente

porque integra arquitectura, ingeniería de software, cloud y MLOps en un caso real con restricciones empresariales.

En este sentido, el trabajo es relevante porque aborda un problema frecuentemente pospuesto: la calidad de la operación. Cuando esta dimensión falla, la calidad predictiva resulta insuficiente. Por ello, el foco se sitúa en construir una base técnica que reduzca fricción, mejore la gobernanza y sea reutilizable en nuevos casos.

Para que esta problemática sea evaluable en términos operativos, se define un conjunto mínimo de indicadores base y objetivo. La intención no es forzar una cuantificación no verificable, sino fijar cómo se medirá el avance en despliegues futuros y en auditorías internas.

Tabla 1.1: *Indicadores operativos base y objetivo del proyecto.*

Indicador	Situación de partida	Situación objetivo tras el estándar
Tiempo de despliegue por caso	Elevado y dependiente de ajustes manuales	Arranque reproducible con plantilla y contratos comunes
Trazabilidad de ejecuciones	Parcial y heterogénea entre proyectos	Evidencia homogénea de versión, run y artefactos
Esfuerzo manual por release	Alto en validaciones e integración	Reducción por automatización CI/CD y validaciones tempranas
Recuperación ante incidencias	Ad hoc y dependiente de conocimiento tácito	Fallback y procedimiento de recuperación documentado

1.5 Antecedentes

Como antecedentes, ya existía una plataforma MLOps en Azure que permitía ejecutar pipelines en AML, con entornos diferenciados (DEV, PRE y PRO) y una gobernanza de promoción de cambios controlada por perfiles *Admin Data Scientist*.

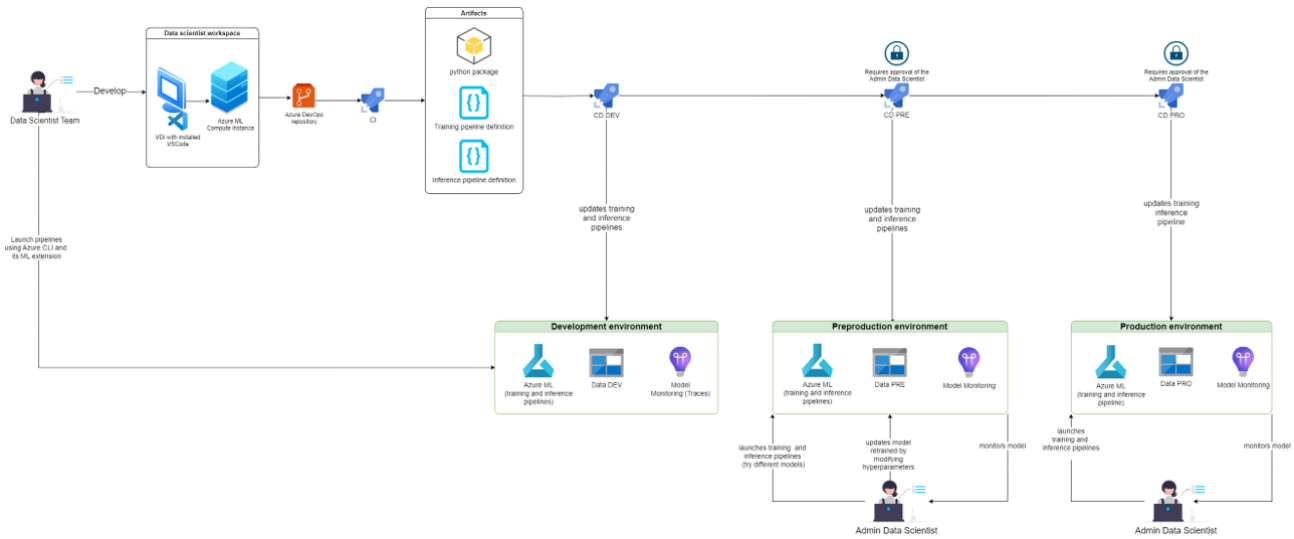

Figura 1.2: *Plataforma MLOps corporativa y esquema de entornos.*

Aunque existían modelos productivizados previamente, carecían de un orden técnico unificado. Sin embargo, este trabajo previo facilitó la identificación de componentes reutilizables y la integración inicial con infraestructuras como Azure Structured Query Language (SQL) Database y Teradata.

1.5.1 Trabajo previo y aportaciones de la memoria 2024–2025

El antecedente directo del TFM es la memoria de 2024–2025, donde se diseñó e implementó una metodología MLOps para la productivización de modelos financieros en Azure Machine Learning. Ese trabajo definio una estrategia de automatizacion del ciclo de vida que cubria preprocesamiento, entrenamiento, inferencia y monitorizacion, reforzando la trazabilidad mediante versionado de modelos, datos, codigo y artefactos.

En dicha memoria se documenta la evolucion de la plataforma ADI (Analisis de Datos para Inversion) como paso previo, con un foco inicial en la ingesta automatizada de datos financieros y su disponibilidad para ciencia de datos. Este trabajo previo incluyo el analisis de fuentes alternativas, el diseno de una base de datos de ingesta y la implementacion de procesos automaticos en Azure para consolidar datos en almacenamiento cloud, con el objetivo de reducir dependencia de descargas manuales y mejorar reutilizacion de datasets.

La memoria previa estructuro su desarrollo en dos fases: una fase de data engineering, con arquitectura de ingesta, almacenamiento y automatizacion, y una fase MLOps prototipo orientada a un modelo de prediccion de situacion economica. En esta segunda fase se introdujeron procedimientos de versionado de datos y se simulo el despliegue de pipelines en un entorno controlado mediante contenedores Docker, consolidando una primera prueba funcional de operativa MLOps.

Ademas, la memoria 2024–2025 detalla la arquitectura de la plataforma en Azure, el uso coordinado de Azure Machine Learning y Azure DevOps, y la implementacion de un flujo de CI/CD orientado a promover codigo entre entornos. Se documenta tambien la productivizacion de varios modelos financieros (LKM, Index Tracking, Señales y Largo Plazo) y el diseno de una base de datos de monitorizacion en Azure SQL para almacenar predicciones y metricas, con el objetivo de facilitar auditoria y analitica de rendimiento.

1.5.2 Estado inicial de la plataforma al inicio del TFM

Cuando comienza este TFM, la plataforma MLOps ya disponia de un *workspace* operativo en Azure, con separacion por entornos (DEV, PRE, PRO), recursos base desplegados y un flujo de despliegue que movia el codigo entre entornos con aprobaciones de gobernanza. En el plano de datos, la memoria previa refleja la existencia de *datastores* y *data assets* versionados en Azure Blob Storage, con convenciones de rutas y versionado por carpetas que permitian trazar entradas y salidas de los pipelines.

Desde el punto de vista de modelos, existia al menos un modelo productivizado (LKM) con flujos de entrenamiento, inferencia y monitorizacion, ademas de informes de salida en formato PDF para

consumo del negocio. Ese punto de partida permitia validar ejecuciones reales y extraer patrones reutilizables en la estructura de componentes, contratos de entrada/salida y registros de MLflow.

Por ultimo, el diseno de la base de datos de monitorizacion en Azure SQL y su implementacion preliminar aportaban un esquema de almacenamiento de predicciones y metricas que este TFM podia integrar con modelos adicionales, reforzando la trazabilidad operativa y la posibilidad de construir indicadores de rendimiento longitudinales.

1.6 Objetivos

Objetivo general. Diseñar, implementar y validar, en el periodo septiembre–marzo del TFM, una solucion basada en MLOps sobre AML que permita modularizar, parametrizar y operativizar modelos financieros existentes, mejorando la trazabilidad, la reproducibilidad y la mantenibilidad del ciclo de vida.

1.6.1 Objetivos específicos

- **Objetivo Específico (OE)1 — Operativizar Chronos Engine end-to-end:** estandarizar entrenamiento, inferencia, backtesting y reporting en AML, garantizando trazabilidad por `run_id` y versionado de datos/modelos.
- **OE2 — Extraer un estandar reutilizable (plantilla MLOps):** definir una estructura de repositorio, contratos de datos, componentes y pipelines base, y una arquitectura CI/CD reproducible entre DEV/PRE/PRO.
- **OE3 — Validar la transferibilidad del estandar en un segundo caso:** implementar Asset Allocation sobre la plantilla y demostrar compatibilidad de contratos, despliegue y gobernanza con minimo retrabajo.
- **OE4 — Medir el impacto operativo del estandar:** comparar esfuerzo/tiempo antes-después con un metodo explicito (tareas incluidas, supuestos y limitaciones) y reportar el resultado.
- **OE5 — Migrar a Azure ML SDK v2:** adaptar herramientas y scripts de gestion de recursos y despliegue a `azure.ai.ml`, garantizando compatibilidad con el ciclo MLOps.
- **OE6 — Gobernanza de modelos e inferencias en Teradata:** integrar el registro y el seguimiento del ciclo de vida en procedimientos corporativos para asegurar trazabilidad de modelos y ejecuciones.

Para reforzar el carácter **SMART** de los objetivos, la Tabla 1.2 explicita para cada OE su criterio de aceptación, la evidencia esperada y la relación con riesgo/mitigación en el Capítulo 3.

Tabla 1.2: *Definición SMART, evidencia y aceptación por objetivo específico.*

OE	Específico/medible	Temporal	Criterio de aceptación	Evidencia y vínculo de riesgo
1	Operativizar Chronos E2E con trazabilidad de run y versiones	Sept.–marzo	Pipelines de train/infer/backtest/report ejecutables y trazados	Tablas y figuras de Chronos en desarrollo y resultados; riesgos operativos en Tabla 3.2
2	Definir plantilla reusable con contratos y CI/CD homogéneo	Sept.–marzo	Estructura de plantilla desplegable con contratos estables	Evidencias en sección de template y tablas de contratos; mitigación de deriva técnica en Tabla 3.2
3	Validar transferencia a Asset Allocation con mínimo retrabajo	Ene.–marzo	Despliegue funcional sobre la misma base técnica	Figuras de pipeline Asset Allocation y tabla comparativa con Chronos
4	Medir impacto operativo del estándar	Ene.–marzo	Comparativa explícita de esfuerzo y límites de medición	Tabla 5.2 y amenazas a validez en capítulo de resultados
5	Migrar utilidades a Azure ML SDK v2	Sept.–marzo	Scripts funcionales sobre `azure.ai.ml`	Evidencia en repositorios y secciones de desarrollo/resultados
6	Integrar gobernanza de modelos e inferencias en Teradata	Oct.–marzo	Registro de ciclo de vida con evidencia consultable	Tabla 4.11 y componentes de registro en desarrollo

Criterio de exito. Cada objetivo se considera cumplido si existe evidencia tecnica verificable en repositorio y ejecuciones: pipelines definidos, componentes operativos, contratos de artefactos, registro de modelos/inferencias y salidas de reporting.

1.6.2 Competencias tecnicas del TFM y criterio de cumplimiento

Esta subseccion recoge las competencias tecnicas acordadas con el Director para trabajarlas de forma explicita durante el proyecto. Su inclusion permite alinear los objetivos academicos del TFM con evidencias verificables de implementacion y con criterios concretos de evaluacion.

- **M2N104 – Diseñar, desarrollar e implementar un proceso de analisis de datos avanzado para responder a la naturaleza de los datos y el objetivo de la tarea a ejecutar.**
  Criterio de cumplimiento: la competencia se considera alcanzada si se evidencia un flujo reproducible de preparacion, entrenamiento, inferencia y validacion adaptado al problema financiero, con decisiones metodologicas justificadas y trazabilidad de entradas, parametros y resultados en ejecucion.
- **M2N110 – Definir, diseñar e implementar arquitecturas escalables, flexibles y resistentes que aborden los problemas existentes y aceleren el despliegue de las diferentes aplicaciones.**
  Criterio de cumplimiento: la competencia se considera alcanzada si se implementa una arquitectura de productivizacion modular y reusable, desplegable por entornos, con separacion de responsabilidades (logica, componentes y orquestacion) y mecanismos de resiliencia operativa (promocion controlada, *fallback* y gobernanza tecnica).
- **M2N111 – Diseñar y automatizar los procesos de gestion de pruebas, cambios, despliegue y actualizaciones de las soluciones empresariales optimizando el ciclo de vida del software.**

Criterio de cumplimiento: la competencia se considera alcanzada si se demuestra automatizacion efectiva del ciclo de vida mediante pipelines de CI/CD, versionado de artefactos y componentes, y evidencia de despliegues y actualizaciones repetibles con control de cambios y trazabilidad de ejecucion.

1.6.3 Competencias transversales y criterio de cumplimiento

Ademas de las competencias tecnicas, el TFM incorpora competencias transversales orientadas a la comunicacion academica, la integracion profesional y la evaluacion integral del impacto del proyecto. Estas competencias se consideran cumplidas cuando existe evidencia clara en la memoria, en la defensa y en la forma de ejecucion del trabajo en entorno real.

- **CTFM01 – Redacta correctamente el informe del Trabajo Final de Master y presenta y defiende con claridad el resultado de dicho trabajo.**
  Criterio de cumplimiento: se considera alcanzada si la memoria mantiene estructura formal, coherencia tecnica, trazabilidad de resultados y defensa clara de decisiones, alcance y conclusiones.
- **CTFM02 – El alumno/la alumna se ha integrado optimamente en la empresa, colaborando con el resto de las personas de su entorno, y ha demostrado alto nivel de resolucion y autonomia en el desarrollo del Trabajo Fin de Master.**
  Criterio de cumplimiento: se considera alcanzada si se evidencia trabajo coordinado con equipos de negocio y tecnologia, gestion autonoma de tareas y resolucion efectiva de incidencias durante la ejecucion del proyecto.
- **CTFM03 – En el proyecto ha comprendido y tomado en cuenta las implicaciones sociales, de salud y seguridad, ambientales, economicas e industriales que conlleva el proyecto.**
  Criterio de cumplimiento: se considera alcanzada si la memoria explicita impactos y riesgos del proyecto (normativos, operativos, economicos y sociales), junto con medidas de control y criterios de uso responsable en contexto empresarial.

1.7 Alcance

El alcance del TFM incluye la parte de productivizacion: modularizacion de logica, definicion de componentes AML, orquestacion de pipelines, registro de artefactos y documentacion tecnica para operacion y mantenimiento.

No entra en alcance rehacer por completo los modelos cuantitativos base ni cambiar politicas corporativas de plataforma o *compliance*. Tampoco se busca cubrir todos los posibles casos de negocio del area, sino validar el enfoque en los casos definidos en esta memoria.

Como supuestos de trabajo, se considera disponibilidad de infraestructura cloud corporativa, acceso a entornos de ejecucion, permisos necesarios y colaboracion funcional para validar integraciones. Si alguno de estos supuestos falla, impacta directamente en calendario y profundidad de validacion.

Las dependencias externas mas criticas han sido permisos de acceso, disponibilidad de datos en ventanas concretas y coordinacion con equipos de plataforma para resolver incidencias de entorno.

1.8 Planificación

La planificación se ha estructurado en tres fases incrementales para mitigar riesgos, como se muestra en la Figura 1.3:

1. **Fase 1 (Chronos)**: Operativización del primer caso real E2E.
2. **Fase 2 (Template)**: Extracción de estándares y construcción de la base reusable.
3. **Fase 3 (Asset Allocation)**: Validación de la transferibilidad del estándar.

El orden anterior corresponde a la cronología real del TFM y estructura la narrativa del resto del documento: primero se resuelve un caso productivo, después se formaliza el estándar y, finalmente, se valida su reutilización.

Para reforzar el hilo objetivo → ejecución → evidencia, la Tabla 1.3 conecta cada fase con entregables verificables.

Tabla 1.3: *Trazabilidad de planificación: hitos, objetivos y evidencias.*

Fase	Objetivos	Entregables	Evidencias en memoria
Fase 1: Chronos	OE1, OE5	Pipelines E2E, componentes, reporting, SDK v2 operativo	Figuras de pipeline Chronos, tabla de contratos, sección de resultados OE1/OE5
Fase 2: Template	OE2, OE6	Plantilla reusable, contratos estables, lifecycle en Teradata	Tablas de capas/contratos, Tabla 4.11, componentes de registro
Fase 3: Asset Allocation	OE3, OE4	Despliegue transferido y comparativa de esfuerzo	Figuras de pipeline Asset Allocation, Tabla 5.2, tabla comparativa Chronos vs Asset

Con los objetivos, el alcance y la planificación establecidos, el siguiente capítulo presenta el estado del arte en MLOps y las bases técnicas que sustentan el enfoque adoptado. Esta transición conecta el problema planteado con las decisiones de arquitectura aplicadas en la fase de desarrollo.

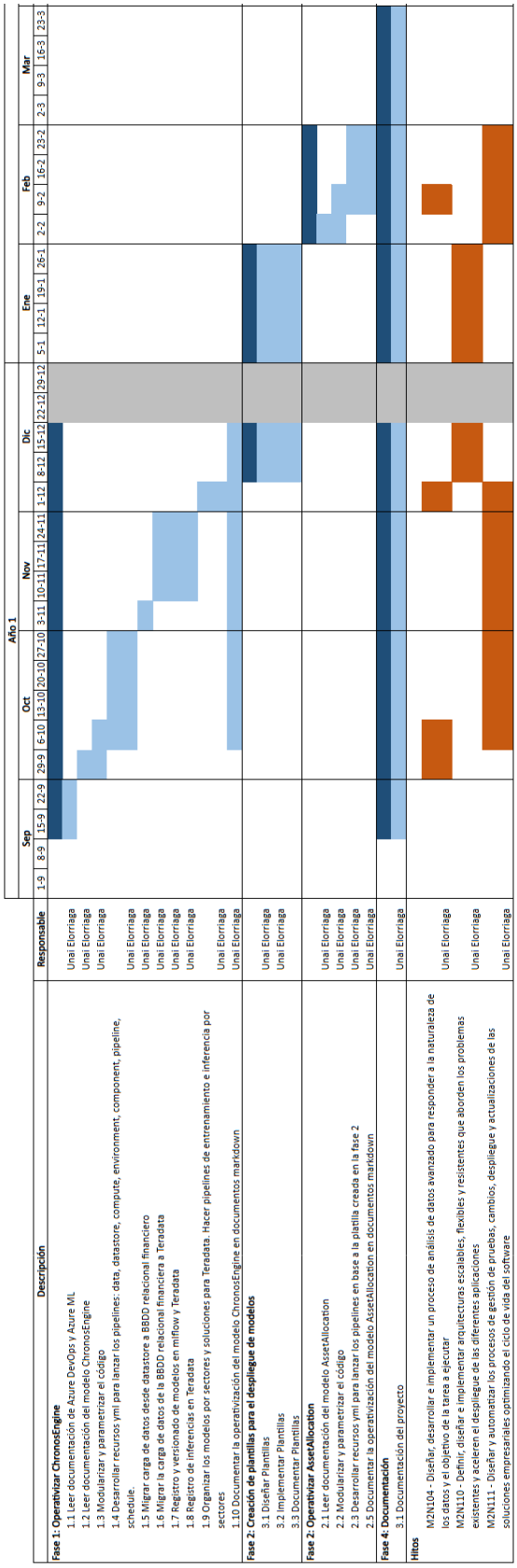

Figura 1.3: *Cronograma de ejecución del proyecto.*

2. Estado del arte

Este capítulo establece la base técnica y metodológica del TFM. El objetivo es explicar por qué llevar un modelo de Machine Learning (ML) a producción en un banco es, ante todo, un reto de ingeniería de software. En un entorno financiero, la calidad del modelo depende no solo de su precisión, sino de su operatividad diaria, la trazabilidad de datos y la mantenibilidad del sistema [1, 6, 7].

2.1 Fundamentos de MLOps

MLOps integra ingeniería de software, *DevOps* y gestión de datos en el ciclo de vida de los modelos de ML. El enfoque pasa de experimentación aislada a operación fiable de servicios en producción [1, 2]. Esto implica priorizar reproducibilidad, monitorización, seguridad y mantenibilidad.

Como se ilustra en la Figura 2.1, MLOps no es una disciplina aislada, sino que articula precisamente esa intersección: el modelado estadístico, la excelencia en operaciones y la ingeniería de datos deben converger en un mismo flujo controlado para ser efectivos en producción.

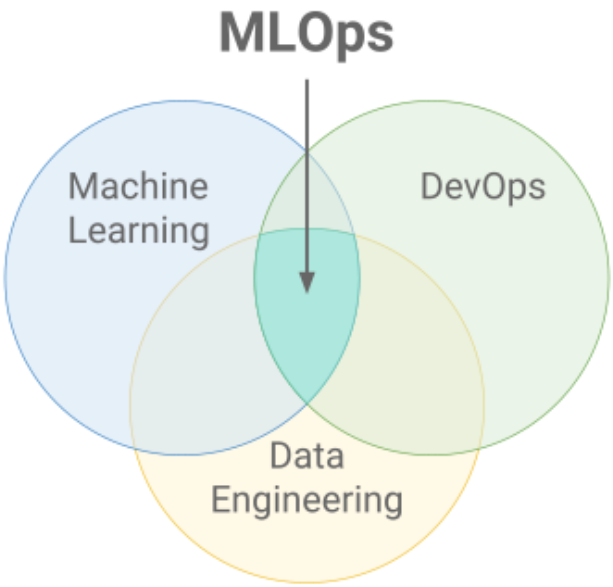

Figura 2.1: *Intersección de disciplinas que conforman MLOps (ML, DevOps y Data Engineering).*

La documentación de Microsoft y la literatura técnica alertan sobre la deuda técnica en sistemas de ML cuando no se documenta y automatiza entrenamiento y despliegue [7, 8]. Sin estas prácticas, aumentan los procesos manuales, la falta de trazabilidad y la fragilidad ante cambios de librerías. Por ello, MLOps exige diseñar la arquitectura desde el inicio.

La literatura reciente subraya que MLOps no es solo un conjunto de herramientas, sino un sistema socio-técnico donde la coordinación entre ciencia de datos, ingeniería de software y operaciones condiciona la fiabilidad del resultado [1, 6]. Esto obliga a definir responsabilidades explícitas y a construir flujos de trabajo donde los cambios en datos, código o configuración sigan el mismo estándar de control.

En sistemas ML, pequeñas variaciones en datos de entrada o en parámetros de entrenamiento pueden producir efectos operativos no lineales. La deuda técnica aparece cuando esas variaciones no quedan versionadas ni auditadas, dificultando reproducibilidad y diagnóstico [7]. Por ello, la trazabilidad de datasets, configuraciones y artefactos no es un extra: es una condición para operar modelos con garantías.

La noción de calidad en MLOps también va más allá de pruebas unitarias. Se requieren validaciones de datos, contratos de entradas/salidas y chequeos de coherencia entre entrenamiento e inferencia, especialmente en dominios regulados [8]. Esta perspectiva amplía la disciplina hacia un control integral del ciclo de vida y refuerza la necesidad de automatizar evidencias.

Si miramos cómo se organizan los equipos punteros, la literatura coincide en cuatro pilares prácticos para operar modelos con fiabilidad [1, 8]:

- Automatizar el recorrido crítico de entrenamiento, inferencia y evaluación.
- Versionar código, configuración, datos de entrada y modelos resultantes.
- Separar código de negocio y código de plataforma.
- Aplicar prácticas de DevOps adaptadas a datos, incluyendo *Continuous Integration* (CI), *Continuous Delivery* (CD), *Continuous Training* (CT) y *Continuous Monitoring* (CM) [2].

El proceso completo de MLOps es iterativo y se divide en tres áreas principales [2]:

1. **Diseño de la aplicación**: definición de requisitos, datos disponibles y arquitectura base.
2. **Desarrollo y experimentación**: pruebas de algoritmos, limpieza de variables y validación de aprendizaje.
3. **Operaciones ML**: empaquetado, pruebas y despliegue en producción mediante Git y pipelines automáticos.

En el sector financiero, estas prácticas permiten responder a auditorías: cuándo se ejecutó un reporte, con qué datos y por qué el modelo tomó una decisión.

2.1.1 Roles, responsabilidades y artefactos

En MLOps, la calidad operativa depende de separar responsabilidades y dejar explícitos los artefactos que cada rol produce y consume. La literatura señala que los fallos recurrentes aparecen cuando esa frontera es difusa y no existe un estándar compartido de evidencias [1, 6]. La Tabla 2.1 resume un esquema práctico de roles y entregables que reduce ambigüedad y facilita auditoría.

Tabla 2.1: *Roles habituales en MLOps y artefactos asociados.*

Rol	Responsabilidad	Artefactos típicos
Científico de datos	Diseño y validación del modelo	Notebooks, dataset de entrenamiento, métricas base
Ingeniería ML	Empaquetado y reproducibilidad	Scripts de entrenamiento/inferencia, *requirements*, tests
Ingeniería de plataforma	Infraestructura y orquestación	Pipelines, configuración de recursos, *infra as code*
Operación/IT	Control de cambios y despliegue	Aprobaciones, registros de despliegue, evidencias de ejecución
Gobernanza/Compliance	Trazabilidad y auditoría	Registros de versiones, evidencias de promoción, reportes de riesgo

La existencia de artefactos verificables por rol permite detectar divergencias entre entrenamiento e inferencia y reduce el tiempo de diagnóstico cuando aparecen fallos de datos o de configuración [7, 8]. Esto es especialmente relevante en proyectos donde las decisiones se revisan a posteriori con criterios regulatorios.

2.2 Ciclo de vida MLOps

El ciclo de vida cubre desde la ingesta hasta la monitorización en producción (ingesta, preprocesado, entrenamiento, evaluación, registro, predicción y salida) [3, 4]. Como se observa en la Figura 2.2, las fases están interconectadas y requieren transferencia continua de versiones, parámetros y resultados.

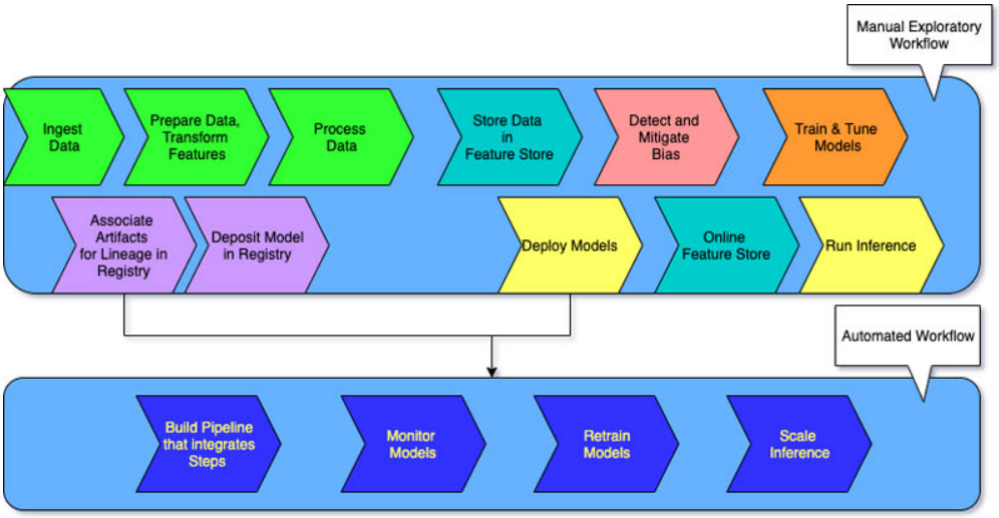

Figura 2.2: *Workflow genérico de un ciclo MLOps [1].*

Microsoft divide la madurez de estos ciclos en varios niveles. En el Nivel 0, el flujo es manual: el equipo de ciencia de datos ejecuta notebooks y entrega artefactos al equipo de *IT*. Los niveles superiores buscan automatización: un `git push` desencadena CI/CD, empaquetado, pruebas, contenedores, entrenamiento continuo y evaluación del modelo antes de producción [2].

En la práctica, el ciclo no termina en el despliegue. La fase de operación incorpora monitorización de rendimiento, gestión de versiones y criterios de reentrenamiento para mantener estabilidad en el tiempo [1, 8]. Esto introduce un bucle de retroalimentación donde las métricas de rendimiento y los cambios en datos gobiernan decisiones técnicas, y donde la observabilidad es el puente entre la ejecución diaria y la mejora incremental.

Asimismo, el ciclo de vida requiere que los artefactos estén trazados en un registro central: datos, modelos, métricas y configuraciones deben poder rehidratarse para auditoría y análisis de incidentes [7, 9]. Este requisito conecta directamente con prácticas de registro y versionado adoptadas en plataformas MLOps modernas.

Para operar con automatización, se requiere *observabilidad*. Es necesario conocer datos de entrada, variables, precisión del modelo y ubicación de resultados. Herramientas como MLflow registran estos elementos en cada ejecución [9].

La observabilidad se complementa con criterios de reentrenamiento. Un modelo puede mantener métricas aceptables en un periodo corto y degradarse cuando cambian las condiciones de mercado. Por eso, los ciclos MLOps maduros incorporan umbrales de alerta y políticas de actualización que se ejecutan de forma controlada [2, 8].

Además, los cambios deben gestionarse con control de versiones para datos y modelos. Esto permite *rollback* cuando el rendimiento se degrada. En sistemas avanzados, los modelos se comparan mediante *Champion vs Challenger*, manteniendo el vigente hasta que el candidato demuestra mejor desempeño [2, 8].

Este esquema requiere que la comparación no se limite a una métrica aislada. En contextos regulados, se recomienda combinar métricas de error, estabilidad y cumplimiento para evitar promociones basadas en mejoras marginales que no son sostenibles [8].

Finalmente, MLOps exige separación de roles: el desarrollo se realiza en DEV y la automatización promueve cambios a PRO con trazabilidad de aprobaciones.

2.2.1 Artefactos críticos y versionado

Un ciclo operativo controlado requiere identificar qué artefactos deben versionarse y dónde se almacenan. No basta con versionar el código: en entornos reales también deben quedar registrados los datasets de entrada, las configuraciones, las métricas y los reportes generados [7, 9]. La Tabla 2.2 sintetiza los artefactos mínimos para garantizar reproducibilidad.

Tabla 2.2: *Artefactos mínimos a versionar en un ciclo MLOps.*

Artefacto	**Ejemplo**	**Propósito**
Datos de entrada	Snapshot *uri_folder*	Reproducir entrenamiento e inferencia
Configuración	`YAML` de parámetros	Aislar cambios de lógica y de entorno
Modelo	`model.joblib` / registry	Trazar versión exacta en producción
Métricas	`metrics.json`	Comparar rendimiento entre versiones
Reportes	PDF/HTML final	Evidencia operativa para negocio

2.2.2 Gobernanza y auditoría en MLOps

En entornos regulados, la gobernanza no es un complemento, sino un requisito operativo. Las normativas de protección de datos y los marcos de IA exigen trazabilidad, control de cambios y capacidad de auditoría sobre modelos en producción [10, 11]. En MLOps, esto se traduce en registrar de forma consistente qué modelo se ejecutó, con qué datos, bajo qué configuración y con qué resultados.

La Tabla 2.3 resume mecanismos de gobernanza que suelen implementarse en plataformas empresariales y el riesgo operativo que ayudan a mitigar. El objetivo no es añadir burocracia, sino garantizar evidencia verificable ante incidencias o revisiones internas [2, 8].

Tabla 2.3: *Mecanismos de gobernanza en MLOps y riesgo mitigado.*

Mecanismo	**Evidencia generada**	**Riesgo mitigado**
Registro de versiones	Historial de modelos y métricas	Promociones sin trazabilidad
Control de cambios	Aprobaciones por entorno	Despliegues no autorizados
Linaje de datos	Snapshot y rutas de entrada	Imposibilidad de reproducir resultados
Auditoría de inferencias	Registro de outputs	Falta de evidencia ante reclamaciones

2.2.3 Observabilidad operativa y niveles de servicio

La observabilidad no se limita a logs técnicos; debe traducirse en señales operativas que permitan evaluar estabilidad, calidad y capacidad de recuperación. En plataformas MLOps, estas señales suelen agruparse en métricas de ejecución, métricas de datos y métricas de resultado [4, 9].

La Tabla 2.4 propone un conjunto mínimo de señales que pueden usarse como base para acuerdos de nivel de servicio internos. El objetivo es poder responder con evidencia a preguntas como: ¿cuánto tarda un ciclo de inferencia?, ¿qué porcentaje de ejecuciones falla?, ¿qué cambios afectan a la calidad del output? [8].

Tabla 2.4: *Señales mínimas de observabilidad en MLOps.*

Señal	**Fuente**	**Uso operativo**
Duración de ejecución	Logs de pipeline	Detectar cuellos de botella
Tasa de fallo	Historial de jobs	Priorizar mejoras de resiliencia
Consistencia de datos	Validaciones de entrada	Evitar inferencias con datos incompletos
Calidad del output	Métricas y reportes	Decidir reentrenamiento

2.2.4 Niveles de madurez: De Google a Microsoft

La industria ha definido modelos de madurez para ubicar el grado real de automatización y gobernanza de un ciclo MLOps. En la práctica, Google y Microsoft proponen taxonomías con nombres distintos, pero con una progresión comparable: desde procesos manuales a operación estandarizada con automatización y control de ciclo de vida [1, 2, 12].

Google describe una evolución en tres niveles. El **Nivel 0** corresponde a un flujo mayoritariamente manual (entrenamiento y despliegue con alta intervención humana). El **Nivel 1** introduce automatización de *pipeline* para entrenamiento continuo y reproducibilidad de pasos, aunque con intervención humana relevante en promoción/operación. El **Nivel 2** apunta a automatización más completa del ciclo (orquestación, integración y despliegue gestionado con menor fricción operacional) [12].

Microsoft desglosa una escala más fina en cinco niveles: **Nivel 0** (sin MLOps estructurado), **Nivel 1** (DevOps sin MLOps), **Nivel 2** (entrenamiento automatizado), **Nivel 3** (despliegue automatizado de modelos) y **Nivel 4** (operación integrada con monitorización, gobernanza y bucles de mejora bajo supervisión) [2].

La Tabla 2.5 resume ambos marcos con foco operativo.

Tabla 2.5: *Comparativa de niveles de madurez MLOps en Google y Microsoft.*

Empresa	Nivel	Enfoque	En qué consiste
Google	Nivel 0	Manual	Entrenamiento, validación y despliegue con scripts/notebooks y fuerte dependencia del equipo; baja reproducibilidad operacional.
Google	Nivel 1	CT básico	Pipeline de entrenamiento automatizado, trazabilidad de artefactos y reentrenos más repetibles; la promoción suele mantener control humano.
Google	Nivel 2	Automatización avanzada	Integración más completa entre datos, entrenamiento, despliegue y operación, con menor fricción para iterar y escalar casos.
Microsoft	Nivel 0	Sin MLOps	Flujo ad hoc, sin estándar de lifecycle ni contratos operativos consistentes.
Microsoft	Nivel 1	DevOps sin MLOps	Hay control de código y despliegue, pero el ciclo ML aún no está plenamente automatizado/-gobernado.
Microsoft	Nivel 2	Entrenamiento automatizado	Jobs/pipelines de entrenamiento reproducibles con registro técnico de ejecuciones y artefactos.
Microsoft	Nivel 3	Despliegue automatizado	Promoción y publicación de modelos más sistemáticas entre entornos, con menor dependencia de tareas manuales.
Microsoft	Nivel 4	Operación gobernada	Ciclo más integrado: monitorización, trazabilidad, políticas de promoción y controles operativos con supervisión humana.

2.3 Herramientas y tecnologías relevantes

En la práctica corporativa, el estado del arte de MLOps se materializa en tres bloques tecnológicos: orquestación de pipelines, registro y versionado de modelos, y operación de despliegue por entornos. En este trabajo, estos bloques se implementan sobre Azure Machine Learning, MLflow, Teradata y Azure DevOps, por lo que resulta útil revisar qué problemas resuelve cada pieza y qué trade-offs introduce su combinación.

2.3.1 Arquitecturas de referencia en plataformas gestionadas

Las plataformas gestionadas de MLOps han convergido en un patrón arquitectónico común: recursos declarativos, componentes reutilizables, pipelines orquestados y servicios de registro/serving acoplados a CI/CD. En ese marco, Azure Machine Learning, AWS SageMaker y Vertex AI ofrecen capacidades similares en bloques funcionales, aunque con diferencias en portabilidad, curva operativa y acoplamiento de proveedor [3, 13, 14].

En paralelo, Kubeflow y MLflow representan enfoques más abiertos. Kubeflow maximiza control y portabilidad sobre Kubernetes, a costa de mayor complejidad de operación. MLflow aporta trazabilidad y gobierno transversal, pero requiere un orquestador externo para cubrir el ciclo completo [9, 15]. En entornos enterprise, la elección depende de la capacidad organizativa para operar la complejidad.

Según la Tabla 2.6, las principales alternativas muestran trade-offs consistentes con la literatura:

Tabla 2.6: *Comparativa técnica de plataformas MLOps en entornos enterprise.*

Plataforma	Pipelines	Registry-Tracking	CI/CD	Despliegue	Infra.	Ventaja / limitación
Azure ML	Nativo (AML Pipelines)	Nativo + MLflow	Alta con Azure DevOps/-GitHub	Endpoints gestionados y batch	Gestionada (Azure)	Integración enterprise alta; menor portabilidad si se explotan recursos específicos
AWS SageMaker	Nativo (SageMaker Pipelines)	Nativo	Alta con CodePipeline/-CodeBuild	Endpoints gestionados y batch	Gestionada (AWS)	Muy buen encaje en AWS; mayor dependencia del stack AWS
Vertex AI	Nativo (Vertex Pipelines)	Nativo	Alta con Cloud Build/Vertex	Endpoints gestionados y batch	Gestionada (GCP)	Integración fuerte en GCP; adopción más costosa fuera de su ecosistema
Kubeflow	Nativo (Kubeflow Pipelines)	Según implementación	Variable según plataforma DevOps	Serving sobre Kubernetes	Autogestionada	Máximo control y portabilidad; mayor complejidad operativa
MLflow	Requiere orquestador externo	Nativo (tracking + registry)	Tool-agnostic	Requiere integración adicional	Agnóstica	Trazabilidad transversal muy buena; no cubre toda la orquestación por sí solo

En este TFM, Azure Machine Learning se adopta por alineación corporativa previa y disponibilidad operativa. Esta decisión sitúa la comparativa de plataformas como análisis de contexto, no como ejercicio de selección ad hoc. La discusión relevante pasa a ser cómo diseñar sobre la plataforma elegida una arquitectura que minimice deuda técnica y facilite transferencia entre casos.

2.4 Metodologias agiles

Esta seccion se incluye con un objetivo contextual y comparativo: sintetizar metodologias de trabajo habituales en proyectos tecnologicos para justificar, en el capitulo de desarrollo, la eleccion aplicada en este TFM. De este modo, se diferencia entre marco conceptual general y aplicacion operativa del proyecto [16, 17].

Las metodologias agiles aportan un marco de trabajo iterativo e incremental, basado en ciclos cortos, retroalimentacion frecuente y adaptacion continua ante cambios de requisitos o restricciones. Su valor en proyectos de ingenieria de software es la capacidad de reducir incertidumbre, priorizar el valor entregado y favorecer la mejora continua en cada iteracion [16, 17].

Scrum es un marco ligero para gestionar trabajo complejo mediante iteraciones temporizadas (*sprints*) y un conjunto acotado de roles, eventos y artefactos. La disciplina se apoya en la inspeccion y adaptacion: planificacion de sprint, seguimiento diario, revision de incremento y retrospectiva, con un *Product Backlog* priorizado y un *Sprint Backlog* para ejecutar el compromiso del ciclo [16]. Sus tres roles (*Product Owner*, *Scrum Master* y equipo de desarrollo) buscan balancear vision de producto, eliminacion de impedimentos y capacidad tecnica para entregar incrementos verificables [16].

Desde el punto de vista operativo, Scrum incorpora un *Definition of Done* que establece los criterios minimos de calidad para aceptar un incremento. Esta practica refuerza la coherencia tecnica del producto y evita acumulacion de deuda tecnica, especialmente relevante cuando el ciclo de vida incluye despliegues repetidos y evidencias de ejecucion [16].

Kanban complementa esta perspectiva con un enfoque centrado en el flujo. El trabajo se visualiza en un tablero, se establecen limites de *Work In Progress* (WIP) y se gestionan politicas explicitas para optimizar la estabilidad y la predecibilidad del proceso. Su orientacion a flujo continuo resulta especialmente util cuando conviven tareas de desarrollo, mantenimiento y operacion [17].

El enfoque Kanban permite medir y mejorar el rendimiento del sistema mediante indicadores como tiempo de ciclo, tiempo de entrega y tasa de finalizacion. La limitacion de WIP reduce multitarea, facilita la identificacion de cuellos de botella y mejora la capacidad de respuesta ante incidencias, lo que resulta critico en entornos con pipelines programados y restricciones de ventana operativa [17].

En entornos MLOps, estas metodologias facilitan que las entregas se realicen de forma incremental y verificable, manteniendo trazabilidad de tareas, validaciones y cambios en pipelines. La combinacion de iteraciones cortas (Scrum) y gestion del flujo (Kanban) encaja con la necesidad de integrar desarrollo, despliegue y operacion de modelos con un control disciplinado de riesgos [16, 17].

En proyectos con fuerte dependencia de datos y validaciones, la aplicacion de practicas agiles ayuda a mantener la coherencia entre definicion de requisitos, ejecucion tecnica y aprendizaje continuo. El resultado es una organizacion del trabajo que prioriza valor operativo y control de calidad, sin perder capacidad de adaptacion ante cambios de datos, modelos o infraestructura [16, 17]. Esta base se retoma en el capitulo de desarrollo para describir la implementacion concreta seguida en la ejecucion del proyecto.

2.5 Plataforma MLOps en Azure

Antes de abordar buenas prácticas, resulta necesario situar Azure para MLOps como un ecosistema completo. En banca, el valor operativo surge de integrar gestión de cambios (Git), seguridad de acceso, separación por entornos (DEV, PRE, PRO) y ejecución en la nube [2, 4].

La memoria de referencia enfatiza que no basta con entrenar un modelo si no existen reglas claras para gestión de datos, métricas y despliegue estable. Azure se utiliza como un conjunto de servicios con responsabilidades definidas.

La subsección 3.1 de la memoria divide la plataforma en tres capas: (i) **Azure DevOps** para control de código y despliegue entre entornos; (ii) **Azure Resource Manager** para gestión de costes y permisos; y (iii) **Azure Machine Learning** (AML) para ejecutar pipelines y registrar modelos.

La Tabla 2.7 resume cómo encajan estas piezas. Esta separación funcional evita mezclar responsabilidades entre gobierno de cambios, gestión de recursos y ejecución ML, reduciendo ambigüedad operativa.

Tabla 2.7: *Capas funcionales de la plataforma MLOps en Azure.*

Capa	Servicios principales	Responsabilidad operativa
Gobierno de cambio	Azure DevOps (Repos, Pipelines, Library)	Control de versiones, despliegue por entornos, variables compartidas y automatización CI/CD.
Gobierno de recursos	Azure Portal + Resource Groups + identidades	Gestión del ciclo de vida de recursos, asignación de permisos (RBAC), presupuestos y separación entre DEV/PRE/PRO.
Plano de ML	Azure Machine Learning Workspace + Compute + Datastores	El plano operativo principal: recursos de cómputo, registro de modelos y artefactos, y orquestación de flujos de ML de inicio a fin.
Seguridad y contraseñas	Azure Key Vault + Service Principals	Servicio centralizado de gestión segura de credenciales y certificados, evitando su exposición en el código.

Según la taxonomía tecnológica descrita en la sección 2 de la memoria, una arquitectura MLOps madura debe resolver al menos los siguientes bloques: experiment tracking, model registry, artifact store, container registry, orquestador, despliegue y monitorización. En la plataforma de Azure adoptada en este TFM, estos bloques se aterrizan del siguiente modo:

- **experiment tracking y model registry:** Azure ML con integración MLflow para registrar parámetros, métricas, artefactos y versiones de modelo en un mismo flujo operativo;
- **artifact store y data plane:** Azure Storage/Blob y datastores de AML para gestionar entradas, salidas e intermedios de pipeline con trazabilidad de rutas y versiones;
- **container registry:** Azure Container Registry para imágenes de ejecución y despliegue reproducible de entornos;
- **orquestación:** pipelines de Azure ML como grafo de dependencias con contratos explícitos de entradas/salidas;
- **despliegue y operación:** combinación de jobs batch programados, endpoints gestionados (cuando se requiere serving online) y automatización CI/CD en Azure DevOps;
- **monitorización:** telemetría técnica (Application Insights/monitoring de jobs) y monitorización funcional de datos/rendimiento integrada en los propios flujos de proyecto.

La subsección 3.1 también aporta un marco útil para clarificar responsabilidades de Azure ML frente a Azure DevOps. Azure ML concentra el workspace y sus dependencias operativas (Storage, ACR, Application Insights, Key Vault), además de recursos administrados de cómputo (Compute Instance y Compute Cluster) y activos de alto nivel (environments, experiments, pipelines, datasets/data assets, models y endpoints). Azure DevOps, por su parte, organiza el ciclo de cambio con Repos, Pipelines y Library, y puede complementarse con Boards, Test Plans y Artifacts para planificación, validación y distribución de paquetes [18, 19].

La memoria de referencia sintetiza una secuencia operativa alineada con este trabajo: desarrollo colaborativo en Git y cómputo de desarrollo, Integración Continua (CI) para calidad y empaquetado, Despliegue Continuo (CD) por entornos con aprobación manual en pasos clave y validación final en

Azure ML. Esta secuencia y la separación DEV/PRE/PRO reducen el riesgo operativo y explicitan responsabilidades.

2.5.1 Arquitectura de red y seguridad perimetral

En un banco como Laboral Kutxa, la exposición de IP pública se evita. La seguridad requiere una arquitectura de red segmentada basada en el patrón *Hub-and-Spoke* [4].

El núcleo de esta estrategia es **Azure Private Link**. El Workspace de Azure ML opera con IP privada dentro de la red interna, evitando exposición pública.

Para asegurar el flujo, se aplican tres reglas:

- **Aislamiento de cómputo**: Los clústeres no tienen IP pública y se conectan mediante *Private Endpoints*.
- **Modelo Zero Trust**: Azure Container Registry (ACR) y Storage Account están protegidos por firewall y solo aceptan tráfico desde Workspace o Azure DevOps.
- **Conectividad híbrida**: El acceso se realiza mediante Virtual Private Network (VPN) o *ExpressRoute*.

Esta arquitectura exige configuración adicional (DNS privadas, rutas de red y Network Security Group (NSG)), pero permite cumplir requisitos regulatorios y de seguridad.

En la arquitectura de Laboral Kutxa, los entornos se separan físicamente: Resource Group específico para DEV, PRE y PRO, evitando impactos cruzados.

La filosofía es mover **el código** entre entornos, no modelos entrenados. El código pasa de DEV a PRE para validación y, si supera los controles, se promueve a PRO para entrenar el modelo final.

2.5.2 Azure DevOps y Library

Azure DevOps coordina el ciclo de cambios, definiendo qué *commit* se despliega en cada entorno y validando criterios mínimos de calidad [18].

La *Library* centraliza variables por entorno y evita condicionales en el código, inyectando configuración en despliegue [2, 18].

En Laboral Kutxa, el flujo de trabajo usa *feature branch* y *pull request* a *main*. La Integración Continua (CI) valida formato y empaquetado, y el Despliegue Continuo (CD) promueve a DEV, PRE y PRO con autorizaciones y validaciones.

2.5.3 Resource Group y recursos base en Azure

Un *Resource Group* agrupa los recursos de un proyecto en Azure y unifica permisos, facturación y ciclo de vida [20]. Por ello se define un Resource Group por entorno.

El elemento central es el *Workspace* de Azure Machine Learning, que integra experimentos, datasets, componentes y clústeres de cálculo [19].

El Workspace se despliega junto a recursos auxiliares necesarios: *Storage Account* para datos, ACR para imágenes y Application Insights para registros de error. Laboral Kutxa estandariza este conjunto por plantilla para mantener DEV y PRO alineados.

2.5.4 Azure Datastore y Key Vault

En AML, un *datastore* abstrae el acceso a bases de datos o almacenamiento externo [21]. Esto evita rutas absolutas en el código y facilita reutilización entre entornos.

Azure Key Vault gestiona credenciales de forma segura [22]. Los permisos se resuelven mediante identidades de servicio, evitando credenciales embebidas en el código. La documentación de MLOps recomienda mantener secretos fuera del repositorio [2]. En Laboral Kutxa se marcan como secretos los valores sensibles en grupos de variables y se evita su inclusión en artefactos publicados, reduciendo exposición en logs y repositorios.

2.5.5 Compute Instance y clústeres de cálculo

Azure distingue entre *Compute Instance* (entorno interactivo de desarrollo) y clústeres de ejecución (cómputo bajo demanda para entrenar) [23].

Esta separación evita dependencias locales: los pipelines de PRO deben ejecutarse en clústeres reproducibles basados en `requirements.txt`.

En Laboral Kutxa, la instancia interactiva se limita a DEV, se asigna a un usuario concreto, opera en red privada y se apaga por inactividad para controlar costes y evitar operaciones directas en producción.

2.5.6 AML: Environments, Components y Pipelines

Los *environments* definen versiones exactas de Python y librerías, actuando como control de cambios y reproducibilidad [4, 24].

Los *components* encapsulan funciones específicas y declaran entradas y salidas [3, 25]. Esta modularidad reduce acoplamientos y mejora mantenibilidad.

La definición de entradas y salidas requiere, como mínimo, los siguientes criterios:

- Tipado explícito de entradas y salidas (`CSV`, número, carpeta).
- Separación entre parámetros de negocio y parámetros de sistema.
- Nombres de salida consistentes (por ejemplo, `modelo_entrenado`).
- Registro de metadatos (autor, fecha, versión de código).

Los *orchestrators* (pipelines) definen el orden y las dependencias entre componentes en archivos `YAML` [3].

Con este enfoque, los pipelines se construyen mediante contratos entre componentes. En Laboral Kutxa se utiliza además despliegue por plantillas para separar infraestructura y lógica de datos, reduciendo riesgos de cambios entre componentes.

La Tabla 2.8 sintetiza la función de cada activo dentro de la capa de ejecución ML.

Tabla 2.8: *Lo que resuelve cada pieza de Azure Machine Learning.*

Activo AML	**Pregunta que resuelve**	**Aporte al desarrollo**
Environment	*¿Con qué software se ejecuta?*	Congela versiones de librerías para garantizar reproducibilidad.
Component	*¿Qué hace cada paso?*	Encapsula una función y declara entradas y salidas.
Pipeline	*¿En qué orden y bajo qué reglas?*	Orquesta el flujo automático y deja trazas en la interfaz.
Datastore	*¿Dónde están los datos?*	Estandariza acceso sin rutas absolutas en el código.
Compute	*¿Dónde calculo?*	Proporciona cómputo bajo demanda durante la ejecución.

2.6 Buenas prácticas

Este apartado sintetiza prácticas generales de MLOps que ayudan a operar modelos con fiabilidad y a reducir deuda técnica. No se pretende cubrir todo el estado del arte, sino concentrar las prácticas que sostienen un ciclo de vida reproducible y reutilizable [1, 6–8].

2.6.1 Separación de lógica de negocio y plataforma

La lógica de negocio y la infraestructura no deben evolucionar acopladas. Separar el código que implementa el modelo de los componentes que orquestan su ejecución reduce fricción, facilita pruebas y permite reutilizar la infraestructura sin reescribir el core del modelo [1, 6, 7].

2.6.2 Modularización y parametrización del código

La modularización convierte el pipeline en una composición de componentes con responsabilidad única. La parametrización desplaza cambios de configuración fuera del código y evita modificaciones repetitivas. Esta combinación mejora mantenibilidad, permite variantes de ejecución controladas y acelera la trazabilidad de cambios [1, 8].

2.6.3 Contratos explícitos y trazabilidad entre componentes

Los pipelines robustos declaran entradas, salidas y metadatos de cada componente. Documentar datasets y modelos con especificaciones consistentes y asociar versiones a cada ejecución permite reproducibilidad y auditoría cuando los resultados se revisan a posteriori [8, 26, 27].

2.6.4 Logging y evidencias de ejecución

El registro de métricas, artefactos y configuraciones por ejecución convierte la operación en un proceso verificable. Sin evidencias, es imposible explicar desviaciones o reconstruir resultados. Por ello, los entornos MLOps incorporan logging estructurado, trazas de pipeline y almacenamiento persistente de outputs [7–9, 28].

2.6.5 Automatización y control de cambios

Las prácticas de *CI/CD* y la segregación por entornos controlan qué cambios llegan a producción. Automatizar pruebas y despliegues reduce errores humanos y asegura que las versiones promovidas han pasado validaciones mínimas [8]. Estas prácticas son un requisito operativo en ciclos MLOps maduros [1, 2, 8].

2.7 Síntesis del estado del arte y gap abordado

A partir de la revisión anterior, el estado del arte deja dos conclusiones consistentes. La primera es que la productivización de ML no puede evaluarse solo por calidad predictiva: exige arquitectura operable, contratos explícitos y gobernanza de ciclo de vida [1, 2, 7]. La segunda es que la transferencia entre proyectos no aparece de forma espontánea; depende de estandarizar capas técnicas y reglas de despliegue.

El gap que aborda este TFM se sitúa precisamente en esa transición entre caso puntual y base reutilizable. En lugar de limitarse a productivizar un único modelo, el trabajo conecta tres fases: una primera operativización real (Chronos), la extracción de un estándar reusable (template) y su validación en un segundo caso (Asset Allocation). Esta secuencia permite contrastar si las recomendaciones del estado del arte se sostienen cuando cambian dominio, perfiles y flujos operativos dentro de un marco corporativo homogéneo.

Por tanto, el aporte diferencial no es una nueva taxonomía de MLOps, sino una implementación trazable de principios conocidos en un entorno corporativo concreto. En capítulos posteriores se detalla cómo esos principios se materializan en contratos, pipelines, gobernanza y evidencia operativa.

La Tabla 2.9 resume de forma trazable el paso de la evidencia teórica a decisiones de diseño en esta memoria.

Tabla 2.9: *Matriz de síntesis: problema, evidencia, decisión y resultado esperado.*

Problema	Evidencia en literatura	Decisión de diseño	Sección de implementación	Evidencia de resultado
Deuda técnica en sistemas ML	Riesgo por falta de contratos y versionado [7, 8]	Contratos YAML y trazabilidad de artefactos	Cap. desarrollo (Chronos/Template)	Tablas de contratos y lifecycle en resultados
Dificultad de operar modelos en regulado	Necesidad de gobernanza y auditoría [11, 29]	Integración MLflow + Teradata con acciones explícitas	Cap. desarrollo (gobernanza)	Registro de versiones, inferencias y fallback
Baja transferibilidad entre proyectos	MLOps requiere estandarización reusable [1, 2]	Plantilla común con CI/CD homogéneo	Cap. desarrollo (template + Asset)	Validación de transferencia en segundo caso

2.8 Bases Financieras Mínimas

Para comprender las decisiones de orquestación, es necesario introducir conceptos básicos de bonos y riesgo que justifican el control de métricas financieras en MLOps. Las definiciones de puntos básicos, duración, benchmark y riesgo de liquidez se recogen de forma breve en el Apéndice C para mantener este capítulo centrado en los conceptos usados explícitamente en el desarrollo.

2.8.1 CDS y la dinámica de los spreads

Un *Credit Default Swap* (Credit Default Swap (CDS)) refleja el coste de asegurar el riesgo de impago de una entidad [30, 31]. El *spread* de CDS es un indicador sensible al riesgo percibido por el mercado [32].

En Chronos Engine, el spread se utiliza como señal clave para predicción y filtrado. Su seguimiento permite identificar cambios de riesgo diarios y distinguir entre variaciones estructurales y ruido de mercado [33–35].

2.8.2 Perfiles de gestión y control del riesgo

En el proyecto se utilizan perfiles BP_ y GDC_, definidos por negocio para distinguir estrategias de riesgo.

Desde el punto de vista MLOps, la reutilización de pipelines es esencial: en lugar de definir pipelines por perfil, se utiliza una tubería común parametrizada por configuración, reduciendo mantenimiento [6].

2.8.2.1 MPT y la frontera eficiente

Para el proyecto de Asset Allocation, la base conceptual es la Teoría de Carteras de Markowitz. El objetivo es equilibrar rentabilidad esperada y riesgo, considerando la correlación entre activos [34, 36].

El modelo busca posicionar los perfiles de inversión cerca de la frontera eficiente, es decir, carteras con máxima rentabilidad para un nivel de riesgo dado. Si un perfil se aleja de esa zona, el pipeline de rebalanceo ajusta la distribución.

En consecuencia, los conceptos financieros introducidos se integran con un enfoque de operación trazable, donde contratos, pipelines y mecanismos de gobernanza permiten sostener la ejecución en entorno corporativo con criterios verificables [2].

Los conceptos financieros revisados justifican la necesidad de trazabilidad y control: conocer versiones de datos y modelos, parametrizar perfiles de riesgo y reconstruir estados históricos para auditoría. En este contexto, MLOps es un requisito operativo.

A partir de este marco teórico y financiero, el capítulo siguiente aborda el marco regulatorio que condiciona la implantación en banca. La finalidad de esta secuencia no es describir de forma exhaustiva el ecosistema MLOps, sino identificar los principios y restricciones que justifican las decisiones de diseño aplicadas en Chronos, en la plantilla reusable y en su validación con Asset Allocation.

3. Marco regulatorio

Este capítulo aterriza una parte que en proyectos de Inteligencia Artificial (IA) suele generar problemas si no se define desde el principio: qué se puede hacer, bajo qué condiciones y con qué controles. En un contexto como el de este TFM, no es suficiente con que el pipeline funcione; también tiene que estar alineado con protección de datos, seguridad y gobierno operativo.

3.1 Normativa y regulaciones aplicables al proyecto

La base regulatoria principal viene de la normativa de protección de datos en la UE: el Reglamento General de Protección de Datos (RGPD) (Reglamento *(EU) 2016/679*) [10] y su aterrizaje en España mediante la LOPDGDD [37]. En la práctica, esto afecta a cómo se diseña la operación técnica, no solo a cómo se redacta la documentación.

También hay que considerar el nuevo marco europeo de sistemas de IA (*AI Act*) [11]. Aunque este proyecto no se plantea como un sistema de decisión autónoma, sí aplica una lógica de gobierno basada en la trazabilidad, control humano y capacidad de auditoría.

Para materializar esta gobernanza, se han implementado dos mecanismos críticos:

- **Ficha de modelo y metadatos operativos**: Cada modelo registrado queda asociado a metadatos de uso previsto, versión y métricas, garantizando que la documentación nazca vinculada al artefacto técnico y a su contexto de ejecución.
- **Trazabilidad de ejecución**: La arquitectura permite reconstruir ejecuciones pasadas enlazando la versión de modelo, los artefactos de entrenamiento y la evidencia de inferencias registradas en la base de datos corporativa [9].

La Tabla 3.1 traduce cada marco normativo a implicaciones de implementación y evidencia técnica de cumplimiento dentro del proyecto.

Tabla 3.1: *Normativa y marcos de referencia: requisito, implicación y evidencia técnica.*

Norma/Marco	Requisito clave	Implicación práctica	Evidencia en la solución
RGPD / LOPDGDD	Minimización y trazabilidad de tratamiento	Control de accesos, segregación de entornos y custodia de secretos	Uso de identidades, variables seguras y trazabilidad de ejecución en pipelines
AIA (UE 2024/1689)	Gobierno, trazabilidad y supervisión humana de sistemas de IA	Registro verificable de versiones y decisiones de promoción	Integración MLflow + Teradata con acciones de lifecycle y evidencia por ejecución
SR 11-7 (MRM)	Gestión de riesgo de modelo y validación independiente	Separación de responsabilidades y criterios explícitos de promoción/fallback	Componentes de registro, self-promotion y fallback documentados en desarrollo
NIST AI RMF 1.0	Gobernar, mapear, medir y gestionar riesgos de IA	Matriz de riesgos por categoría y controles preventivos/detectivos	Matriz ampliada de riesgos y plan de contingencia del proyecto

Además, el marco de referencia en el sector bancario es la guía **SR 11-7** de la Reserva Federal (MRM) [38], cuyos principios de gestión de riesgo de modelo se han integrado en la plataforma:

- **Tres Líneas de Defensa**: Segregación de responsabilidades entre el equipo de desarrollo (1ª línea), validación técnica mediante MLflow (2ª línea) y auditoría de procesos en Azure DevOps (3ª línea).
- **Linaje de Datos (*Data Lineage*)**: Trazabilidad completa de las transformaciones sufridas por el dato mediante identificadores únicos de versión (`Data_Version_ID`) que se propagan desde la extracción hasta la inferencia.

3.2 Riesgos del proyecto y plan de contingencia

Una vez definido el marco regulatorio, el siguiente paso es convertirlo en gestión de riesgo operativa. En este proyecto los riesgos más importantes no fueron teóricos: aparecieron en permisos cloud, dependencias de entorno, disponibilidad de datos y continuidad de ejecución.

Como se resume en la Tabla 3.2, se ha trabajado con una matriz de riesgo ampliada para priorizar mitigaciones y decidir planes de contingencia antes de promover cambios de entorno.

Tabla 3.2: *Riesgos del proyecto y plan de contingencia (matriz ampliada).*

Categoría	Riesgo	Prob.	Sev.	Control preventivo	Control detectivo	Propietario	Contingencia
Seguridad	Permisos insuficientes en recursos cloud	M	A	Revisión de roles y mínimo privilegio antes de despliegue	Fallo temprano de conexión en jobs y validación de permisos	Plataforma/IT	Revertir ejecución, corregir permisos y relanzar con validación previa
Técnico	Dependencias rotas entre entorno local y runtime cloud	M	A	Versionado estricto de environment e imagen	Errores de build y smoke tests de pipeline	Equipo MLOps	Bloquear promoción y corregir image-n/`requirements`
Datos	Disponibilidad parcial de datos de origen	M	A	Validación de fuentes y ventanas de carga	Alertas por filas faltantes y chequeos de completitud	Equipo de datos	Reintentos planificados y salida degradada trazable
Operativo	Sobrecoste por ejecuciones no controladas	B-M	M	Política de schedules por entorno y control de compute	Monitorización periódica de coste cloud	MLOps/FinOps	Pausar jobs no críticos y ajustar frecuencia/recursos
Cumplimiento	Error funcional en pipeline productivo	B-M	A	Pruebas por componente y contratos de artefactos	Evidencia de ejecución y validación de outputs	Equipo funcional + MLOps	Fallback manual, rollback de versión y análisis de causa raíz

Este enfoque de contingencia está alineado con prácticas recomendadas de MLOps: diseñar para recuperar rápido cuando algo falla, no solo para que ”todo salga bien-[4, 9].

3.3 Evolución normativa y horizonte de cumplimiento

El despliegue del AIA en la UE es progresivo y condiciona la hoja de ruta de gobernanza. La Comisión Europea establece hitos de aplicación escalonados, lo que permite planificar controles por fases y evitar un enfoque de cumplimiento reactivo [11, 39].

En términos prácticos para esta memoria, el corto plazo se centra en trazabilidad verificable de ciclo de vida y documentación de evidencias; el medio plazo, en formalizar controles de riesgo por tipo de uso; y el largo plazo, en consolidar monitorización y revisión periódica con criterios homogéneos entre proyectos. Esta secuencia conecta directamente con las líneas futuras del Capítulo 8 y reduce el riesgo de deuda de cumplimiento.

Como cierre, la parte regulatoria y la parte técnica no se tratan como bloques separados. En este TFM, ambas se conectan en una misma idea: operar modelos con control, evidencia y responsabilidad durante todo el ciclo de vida.

Con este marco de cumplimiento y riesgo establecido, el capítulo siguiente entra en el desarrollo y la operativización de los modelos. La narrativa pasa de las condiciones de control a la ejecución técnica que materializa esos requisitos.

4. Desarrollo y operativización de modelos

Este capítulo describe el núcleo técnico del proyecto, estructurado en tres fases progresivas. En primer lugar, se detalla la operativización de Chronos Engine como caso real de producción inicial. A continuación, se presenta la estandarización del proceso mediante la plantilla MLOps (repositorio `TYM.MLOpsTemplate`), concebida como una base técnica reutilizable para facilitar el despliegue y la gobernanza de nuevos modelos. Finalmente, se expone la operativización de Asset Allocation como caso de validación de la transferibilidad del enfoque desarrollado.

Mapa de lectura para perfiles mixtos. Si el lector no técnico quiere una visión operativa, puede centrarse en las subsecciones de motivación, flujo diario y evidencias (tablas de contratos, comparativas y figuras de pipeline). Si el objetivo es validar detalle de implementación, conviene revisar también las subsecciones de componentes, orquestación y gobernanza en Teradata. Para una lectura rápida orientada a evaluación, la ruta mínima recomendada es: Chronos (flujo y evidencias), plantilla reusable (qué resuelve/qué no resuelve) y Asset Allocation (transferibilidad y límites).

4.1 Metodologias agiles

Los conceptos de Scrum/Kanban se introducen en la seccion 2.4; aqui se describe su aplicacion concreta. Tras la revision metodologica del estado del arte, la ejecucion del proyecto se organizo con iteraciones cortas y un *backlog* priorizado, donde las unidades de trabajo se definieron como entregables verificables (componentes, contratos YAML, pipelines y evidencia de despliegue). En cada ciclo se planificaron tareas, se revisaron avances y se ajusto el alcance segun resultados y necesidades de negocio [16].

El *Definition of Done* se utilizo como criterio operativo para aceptar incrementos solo cuando existian evidencias minimas: pipelines funcionales, contratos de entrada/salida definidos y trazabilidad de ejecucion. En paralelo, el seguimiento diario se gestiono con un tablero Kanban para limitar *work in progress* y visualizar bloqueos vinculados a permisos, datos o ejecuciones fallidas, facilitando la coordinacion entre desarrollo, despliegue y validacion [16, 17].

4.2 Chronos Engine: operativización del modelo de pricing RFP

Chronos Engine es el primer caso real de operativización del TFM. Se desarrolló como proyecto de referencia para validar que los principios de MLOps funcionan en un entorno bancario. El objetivo de negocio del modelo es anticipar la evolución del mercado de deuda privada (usando el índice iTraxx). Desde el punto de vista ingenieril, el objetivo fue automatizar el ciclo completo: ingesta, entrenamiento, registro de modelos, inferencia y generación diaria de informes [2–4].

4.2.1 Motivación y contexto operativo

Chronos se eligió por su complejidad: combina varios modelos, procesa 12 sectores financieros y debe integrarse con bases de datos corporativas. El requisito de negocio era abandonar ejecuciones manuales en notebooks y garantizar trazabilidad y auditoría. El reto técnico se centró en estandarizar los contratos de datos, parametrizar la orquestación para evitar duplicidades y conectar Azure con MLflow y Teradata, que es la base de datos principal de Laboral Kutxa.

El modelo opera en horizontes diario, mensual y trimestral. Incluye una rama *Fair Value* de predicción numérica y una rama *Mid Term* de clasificación por escenarios probabilísticos.

La predicción se ejecuta por sectores (Tecnología, Sanidad, Energía, entre otros). Esta segmentación condicionó el diseño: los pipelines comparten el mismo código base y difieren únicamente en parámetros. El resultado converge en un informe resumen (*SumUp*) que consolida las 12 predicciones para el equipo financiero.

La separación entre lógica matemática y configuración fue clave. Se externalizaron fechas de validación y parámetros de infraestructura en variables de despliegue para reducir cambios en el código de Python y mantener trazabilidad entre entornos.

4.2.2 Cómo se usa en el día a día

En operación diaria, Chronos se utiliza como un flujo rutinario y no como una ejecución ad hoc. El recorrido operativo típico es el siguiente:

1. Validación de disponibilidad de datos y ventana operativa del día.
2. Lanzamiento de inferencia parametrizada por entorno y solución.
3. Carga y preprocesado con validación de contratos de entrada.
4. Ejecución de inferencia por sectores y consolidación de resultados.
5. Registro de inferencias y metadatos de ejecución en la capa de gobierno.
6. Generación automática de reporte de negocio y publicación en ruta acordada.
7. Revisión de incidencias, si existen, y activación de contingencia cuando procede.

Este flujo permite que el equipo de negocio reciba resultados diarios con trazabilidad técnica, mientras que el equipo de plataforma mantiene evidencia de versiones, parámetros y artefactos asociados a cada ejecución.

4.2.3 Modularización y desacoplamiento de la lógica analítica

La estructura técnica evita un único script monolítico y separa el problema en módulos funcionales:

- Ingesta SQL para extracción de datos corporativos.
- Preprocesado y cálculos previos (lags del iTraxx).
- Entrenamiento de modelos *Fair Value* y *Mid Term*.
- Inferencia operativa y *backtesting*.
- Reporting automático en PDF.
- Integración con Teradata para gestión de versiones.

La modularización se apoya en contratos explícitos de entrada y salida. Si el componente de limpieza produce un `CSV`, el componente de entrenamiento consume el mismo formato. Esto acota responsabilidades y facilita el diagnóstico cuando se produce un fallo [3].

4.2.4 Contratos de datos y validaciones operativas

Para evitar errores silenciosos, Chronos incorpora validaciones previas en cada salto de componente. Estas validaciones no pretenden sustituir las pruebas unitarias, sino garantizar que los artefactos que circulan por el pipeline cumplen el contrato esperado antes de avanzar [4, 8].

La Tabla 4.1 resume ejemplos de contratos y sus chequeos asociados. El objetivo es reducir fallos tardíos y mantener trazabilidad de entradas y salidas en cada etapa.

Tabla 4.1: *Contratos de datos operativos en Chronos Engine.*

Etapa	Contrato	Validación aplicada
Ingesta	Dataset histórico completo	Comprobación de columnas obligatorias y fechas mínimas
Preprocesado	Dataset limpio sin nulos críticos	Validación de nulos y consistencia temporal
Entrenamiento	`output_train_path` válido	Verificación de artefactos `.joblib` y métricas
Registro	Modelo y métricas coherentes	Registro en MLflow con identificadores consistentes
Inferencia	Predicciones completas	Chequeo de filas/sectores esperados

4.2.5 Configuración de infraestructura en Azure Machine Learning

A nivel de infraestructura, Chronos despliega casi medio centenar de pipelines en AML. Como muestra la Tabla 4.2, se trata de un conjunto de recursos coordinados y no de un único script.

Tabla 4.2: *Principales piezas de Azure ML montadas para Chronos.*

Pieza	Cantidad	Para qué sirve
Pipelines (YAML)	44	Es el plano de montaje para entrenar, predecir, testear y reportar según el sector.
Componentes (YAML)	25	Las unidades de trabajo sueltas: un YAML para limpiar, otro para entrenar, etc.
Máquinas (Compute)	2	Un clúster para trabajo pesado y otra máquina pequeña para construir contenedores Docker.
Disco duro (Datastore)	1	El Storage Account donde guardamos los `CSVs` y modelos generados.
Entorno (Environment)	1 base	La receta de Python hipercontrolada con las versiones exactas de las librerías.

4.2.6 Estrategia de despliegue y el dilema de los 12 sectores

La operación de Chronos se articula mediante un pipeline de Azure DevOps. En lugar de mantener varios archivos de despliegue, se utiliza un único YAML parametrizado con `action`. Con `train` se ejecuta el entrenamiento, con `infer` se realiza la inferencia diaria y con `report` se genera el informe operativo.

La división en 12 sectores exigía evitar duplicidades. La estrategia adoptada fue la reutilización estricta: el código y los componentes base son comunes, y las diferencias se introducen por parametrización en la definición del pipeline de AML.

En un entrenamiento de un sector concreto, el flujo interno incluye: extracción segura desde Teradata, preprocesado, entrenamiento paralelo de las ramas *Fair Value*, *MIDAS* y *Mid Term*, registro de modelos y métricas en MLflow, y evaluación de promoción frente al modelo vigente, con actualización de estado en base de datos cuando procede.

La Tabla 4.3 resume decisiones de diseño relevantes en formato restricción-alternativa-criterio-consecuencia para mantener trazabilidad de por qué se eligió cada enfoque.

Tabla 4.3: *Decisiones clave en Chronos: de la restricción a la consecuencia operativa.*

Restricción real	Alternativa considerada	Criterio de elección	Consecuencia operativa
12 sectores con alta repetición de lógica	Pipelines independientes por sector	Minimizar duplicidad y mantenimiento	Orquestación paralela parametrizada con menor retrabajo
Dependencia de credenciales corporativas	Secretos embebidos en scripts	Seguridad y trazabilidad de acceso	Uso de identidades/Key Vault y menor exposición
Necesidad de reporte diario consolidado	Un reporte por sector	Consumo de negocio y legibilidad	Informe SumUp unificado para lectura operativa
Promoción de versiones con control	Promoción manual no estandarizada	Gobernanza reproducible y auditable	Registro + self-promotion/fallback con evidencia

4.2.7 Orquestación masiva y el reto de los 12 sectores

La arquitectura de Chronos se apoya en ejecución paralela mediante **ParallelJob** [3]. En lugar de lanzar 12 pipelines independientes, el orquestador particiona los datos de entrada por sector y distribuye las cargas en los nodos disponibles del clúster.

Esta estrategia reduce el tiempo total de ejecución al aproximarlo al del sector más costoso, evitando ejecuciones secuenciales. En inferencia diaria se aplica el mismo patrón, usando el modelo *champion*. Para evitar la generación de 12 informes separados, la acción `report` activa el pipeline *SumUp*, que consolida las inferencias en un único informe de síntesis.

4.2.8 Ventanas operativas y control de ejecución

La ejecución de Chronos requiere coordinar ventanas de datos, disponibilidad de cómputo y horarios de consumo por negocio. Para evitar ejecuciones fuera de ventana o con datos incompletos, el despliegue se apoya en parámetros de calendario y en validaciones previas de carga [4].

En la práctica, el diseño prioriza la consistencia: si los datos no cumplen los criterios mínimos, el pipeline bloquea la ejecución o entra en modo controlado de reintento. Esta disciplina evita generar informes con datos parciales y reduce retrabajo operativo, manteniendo trazabilidad de cuándo y por qué se pospone una ejecución.

4.2.9 Arquitectura técnica del pipeline y componentes operativos

A continuación se resume la función de los principales componentes operativos del pipeline:

4.2.9.1 Ingesta y preprocesamiento de series temporales

- **Componente de ingesta**: Paso inicial de extracción. Las credenciales se resuelven desde Key Vault corporativo y el `output` es un `uri_folder` con el histórico SQL. El componente utiliza un conector SQL estándar.
- **Componente de preprocesado iTraxx**: Elimina nulos, alinea fechas con huecos de mercado y sincroniza las series con el índice iTraxx, generando un conjunto de datos limpio para el entrenamiento.

4.2.9.2 Entrenamiento de modelos con algoritmos robustos

El componente de entrenamiento de *Fair Value* recibe la ruta de datos limpios, el sector, los porcentajes de partición temporal y los identificadores para trazabilidad en MLflow y Teradata. Como salida, genera un `output_train_path` con el modelo entrenado, métricas y parámetros. Internamente, aplica validación temporal, ejecuta búsqueda de hiperparámetros y selecciona el algoritmo con mejor rendimiento, registrando métricas relevantes y persistiendo el modelo en formato `.joblib`.

4.2.9.3 Gestión de artefactos y persistencia en Datastores

- **Registro de modelo**: Registra el modelo y sus métricas en MLflow y actualiza el estado en Teradata mediante procedimientos almacenados, asegurando trazabilidad de la versión activa.
- **Self-promotion**: Compara el rendimiento del modelo entrenado con el modelo vigente. Si no supera el umbral definido, se descarta la promoción; si lo supera, se actualiza la versión activa en Teradata.
- **Registro de inferencias**: En inferencia, inserta las predicciones en Teradata mediante carga masiva para su consumo por cuadros de mando y equipos de análisis [9].

4.2.10 Integración entre Azure DevOps y Azure Machine Learning

El despliegue de Chronos se apoya en un pipeline corporativo de CI/CD. El parámetro de acción permite ejecutar entrenamiento, inferencia o generación de informes sin duplicar pipelines.

Antes de ejecutar en AML, el pipeline valida formato con Ruff y ejecuta pruebas unitarias con Pytest. Superadas estas validaciones, DevOps invoca la plantilla central para el registro de componentes y el lanzamiento de pipelines.

El enfoque multisegmento permite ejecutar en paralelo doce pipelines de entrenamiento, uno por sector. Esta paralelización reduce tiempos de ciclo y evita mantener definiciones redundantes.

Como se observa en la Figura 4.1, el pipeline de entrenamiento centraliza la ejecución por sector. Se utiliza `user_identity` para operar con credenciales corporativas en lugar de secretos compartidos.

Como evidencia documental, en Chronos Engine se ha redactado un `README.md` con el flujo de ejecución y pruebas principales, un `docs.md` con criterios operativos de AML/MLflow y troubleshooting, y un `CHANGELOG.md` que deja trazadas decisiones técnicas relevantes (por ejemplo, el ajuste del motor de reporting). Estas piezas permiten reconstruir decisiones y reproducir el ciclo de vida con mayor rigor.

Como se muestra en la Figura 4.2, la inferencia diaria integra el registro en Teradata, la reconstrucción histórica para backtesting y la preparación de reportes.

4.2.11 Modos de fallo y mitigaciones

La operación diaria de Chronos mostró que los fallos más costosos no siempre son algorítmicos. Suelen aparecer en dependencias de entorno, credenciales o cambios de configuración no versionados. Para reducir impacto, se definieron mitigaciones operativas que se integran en el flujo de despliegue [2, 4].

La Tabla 4.4 resume fallos típicos y su tratamiento. El foco está en detectar pronto, evitar propagación entre entornos y dejar evidencia de recuperación.

Figura 4.1: *Pipeline de entrenamiento de Chronos Engine. Se muestran los componentes principales (carga, preprocesado, entrenamiento por ventana, registro y self-promotion). Las flechas representan artefactos versionados (`data_path/output_path`) y triggers de promoción.*

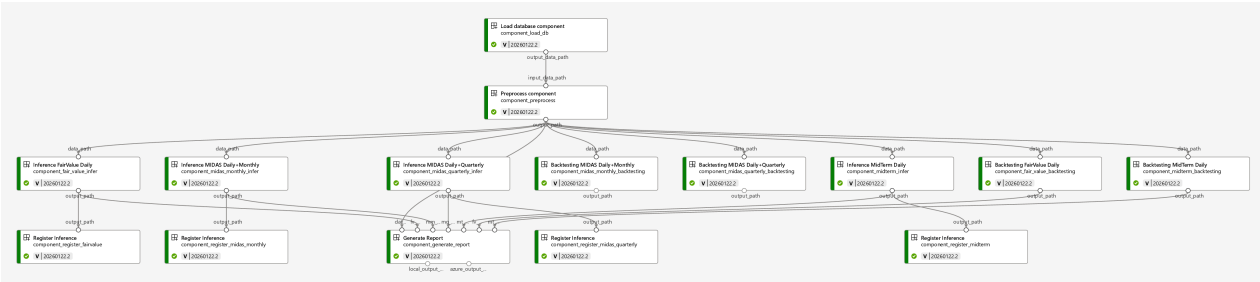

Figura 4.2: *Pipeline de predicción diaria (inferencias, registro y backtesting). Se muestran los componentes principales (carga, preprocesado, inferencia, registro y reconstrucción histórica) y los puntos de consolidación de resultados. Las flechas representan artefactos versionados y triggers de ejecución.*

Tabla 4.4: *Modos de fallo frecuentes y mitigaciones en Chronos.*

Fallo típico	Impacto	Mitigación operativa
Credenciales inválidas	Bloqueo de ingesta	Uso de Key Vault y reintento controlado
Cambios de librería	Incompatibilidad de entorno	Versionado estricto en *environment*
Desfase de datos	Inferencia incompleta	Validación de ventanas temporales
Artefactos incompletos	Registro fallido	Chequeo de `joblib` y métricas antes de registrar

4.2.11.1 Detalle técnico: lógica de entrenamiento para Fair Value

El entrenamiento se integra con la clase `MLOpsLifeCycle` para abrir un *run* en MLflow desde el inicio. Se aplica escalado ajustado únicamente con datos históricos para evitar fugas de información, y se persisten artefactos intermedios en el *Datastore* (datos limpios y escaladores) para trazabilidad y reejecución [4, 9].

El entrenamiento utiliza validación temporal con `TimeSeriesSplit` y una búsqueda de hiperparámetros para seleccionar el mejor modelo entre distintas familias. El modelo ganador se encapsula en un `SklearnWrapper` y se registran parámetros y métricas en MLflow, incluyendo un `hparams.json` que facilita auditoría y reproducibilidad.

4.2.11.2 Métricas de evaluación y validación matemática

En la fase de evaluación se registran varias métricas complementarias en MLflow para evitar conclusiones basadas en un único indicador. Se utiliza MAE para el error medio absoluto en puntos básicos, RMSE para penalizar errores grandes y MAPE para interpretar el error relativo cuando los niveles de precio varían significativamente. La comparación entre MAE y RMSE aporta información sobre la presencia de *outliers* y guía mejoras en la limpieza de datos y en la configuración del pipeline.

4.2.11.3 Detalle técnico: clasificación en series Mid Term

Chronos incorpora una rama de clasificación multiclase denominada **Mid Term**. El objetivo se transforma mediante un desplazamiento temporal y se asignan etiquetas discretas según umbrales definidos con negocio, pasando de una variable continua a categorías de rentabilidad.

La evaluación utiliza un *scorer* específico que combina el *macro F1* con una penalización proporcional al tamaño del error de clasificación. Este enfoque reduce el impacto de errores severos y favorece modelos más conservadores en escenarios de alto riesgo [8].

4.2.11.4 Detalle técnico: análisis retrospectivo (Backtesting)

El proceso de *Backtesting* no reentrena el modelo. Su objetivo es simular el comportamiento histórico del modelo *champion* y documentar su rendimiento sobre periodos pasados.

Para ello, se recupera el modelo con la etiqueta `@champion` desde MLflow y se reutiliza el `scaler` almacenado durante el entrenamiento [9]. El pipeline genera métricas de error agregadas, produce visualizaciones de la regresión y las registra como artefactos para auditoría y trazabilidad histórica.

4.2.11.5 Generación de informes operativos y SumUp

El pipeline finaliza con una fase de *Data-to-Document* para convertir resultados técnicos en un informe operativo. Se consolidan las salidas de *Fair Value*, *MIDAS* y *Mid Term* y se valida la coherencia temporal antes de generar el reporte.

Las visualizaciones producidas durante el pipeline se integran en plantillas HTML y se compilan en un PDF de dos páginas: la primera resume desviaciones del precio inferido y la segunda sintetiza la clasificación *Mid Term* con niveles de confianza.

4.2.11.6 Orquestación de informes consolidados: SumUp

Con 12 sectores, el paso final no genera 12 informes independientes. El componente **SumUp** recibe los 12 PDFs y los combina en un único documento mediante `PyPDF2`.

Como se muestra en la Figura 4.3, el pipeline carga las inferencias consolidadas desde Teradata y ejecuta el componente de generación del reporte SumUp, dejando trazabilidad del artefacto final.

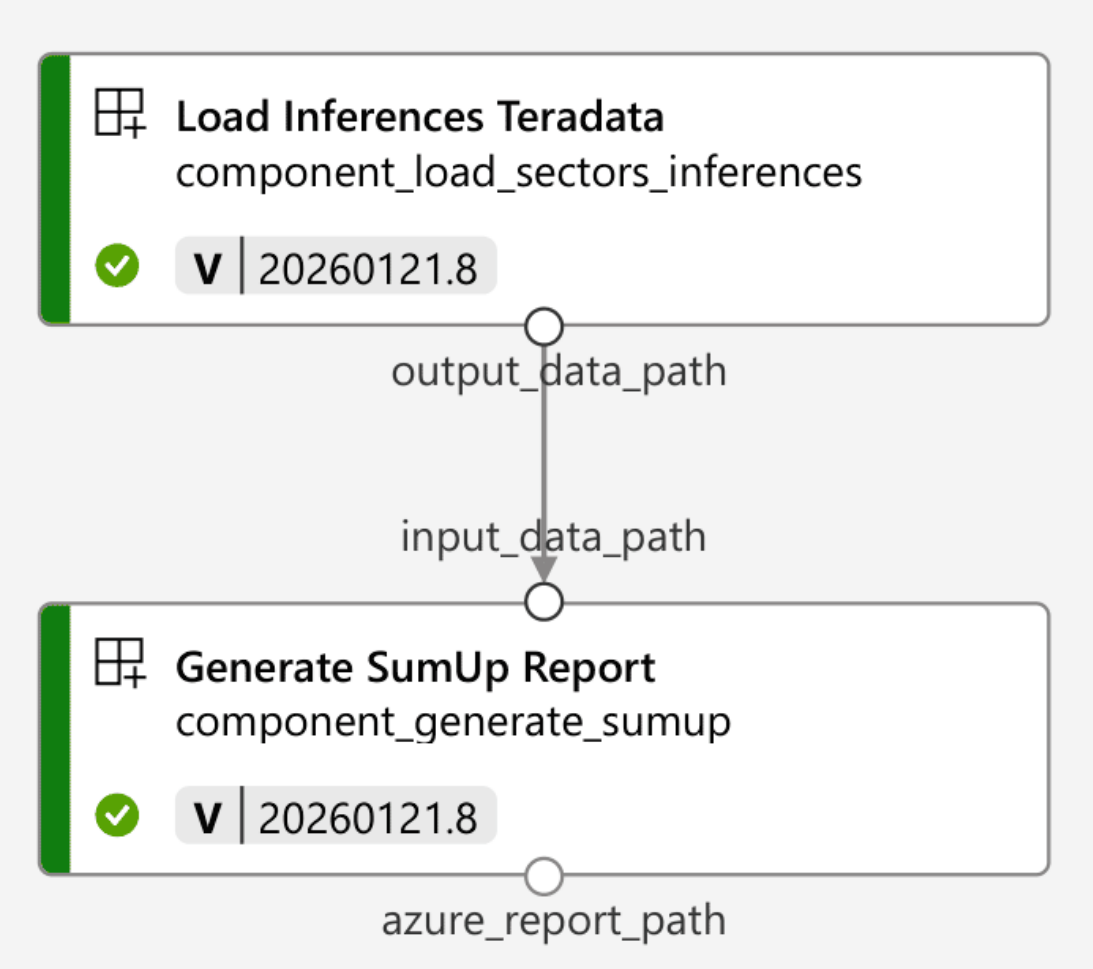

Figura 4.3: *Carga de inferencias desde Teradata y generación del reporte SumUp.*

Además, ejecuta un análisis de *Top Movers* para destacar los sectores con mayor desviación entre el precio observado y el *Fair Value*. Estos sectores se reflejan en la portada del informe consolidado para facilitar la lectura por parte del equipo de tesorería.

El PDF final se copia al almacenamiento corporativo con sello temporal `YYYY-MM-DD`. Esta acción activa los flujos automatizados de distribución, garantizando el envío del informe a los equipos correspondientes.

4.2.12 Evidencias de reporting y trazabilidad operativa

El reporting no se considera un paso accesorio, sino parte del ciclo de vida gobernado. Los informes de Chronos son la salida que consume negocio y, por tanto, deben estar trazados con la misma disciplina que los modelos y métricas. Esta trazabilidad se implementa como artefactos versionados en MLflow y en el almacenamiento corporativo, garantizando recuperación histórica y auditoría [9].

La Tabla 4.5 resume los artefactos clave del reporting, su uso y el beneficio operativo asociado. El objetivo es asegurar que cada informe puede vincularse a una ejecución, una versión y un conjunto de datos concretos.

Tabla 4.5: *Artefactos de reporting en Chronos y su uso operativo.*

Artefacto	Uso principal	Beneficio
Informe *SumUp* (PDF)	Consumo diario por Tesorería	Visión consolidada y trazable
Gráficos intermedios	Evidencia técnica de resultados	Auditoría y diagnóstico posterior
Metadatos de ejecución	*run_id*, fecha, versión	Reconstrucción de contexto
Registro de inferencias	Persistencia en Teradata	Consumo por analítica corporativa

4.2.13 Integración con MLflow y control total desde Teradata

Un elemento diferenciador de la arquitectura es la integración entre MLflow y Teradata.

MLflow gestiona el registro de modelos, métricas y artefactos, mientras que la clase `MLOpsLifeCycle` comunica a Teradata, mediante procedimientos almacenados, la versión activa, caducidad y fechas de vigencia. De este modo, la gobernanza se mantiene en los sistemas corporativos, sin depender exclusivamente de componentes cloud [9].

4.2.13.1 Algoritmo de entrenamiento y lógica secuencial

La secuencia de entrenamiento se puede describir de forma resumida: carga de configuración, lectura de datos desde el Datastore, separación de variables, escalado, validación temporal, búsqueda de hiperparámetros, registro de métricas en MLflow y persistencia de modelos y parámetros. Esta lógica asegura reproducibilidad y trazabilidad sin necesidad de detallar el código fuente en la memoria.

4.2.13.2 Definición y documentación de características de entrada

La calidad de Chronos no depende solo del algoritmo, sino tambien de la amplitud y diversidad de datos integrados desde Teradata. A continuación, se detallan las principales familias de variables que maneja el modelo para cada uno de los 12 sectores:

Tabla 4.6: *Taxonomía de las características (features) de Chronos Engine.*

Familia	Descripción Técnica	Importancia
Swaps de Crédito (CDS)	Spreads de 5 y 10 años para entidades del mismo sector.	Principal señal de riesgo.
Curva de Tipos	Tasas IRS (Interest Rate Swap) a 2, 5, 10 y 30 años.	Sensibilidad al coste del dinero.
Indicadores Macro	Inflación esperada (BEI), PIB trimestral y datos de empleo.	Ciclo económico global.
Renta Variable	Niveles de índices bursátiles (Eurostoxx, S&P500, IBEX) y su volatilidad (VIX).	Sentimiento de mercado.
Materias Primas	Precios del Petróleo (Brent) y Oro.	Costes de producción industrial.
Variables Propias	Medias móviles exponenciales (EMA) y retornos logarítmicos de la empresa.	Inercia histórica del precio.

Estas variables aportan interpretabilidad al modelo. Si se anticipa una caída en el precio de un bono, la importancia de variables permite vincular la predicción con factores observables (por ejem-

plo, cambios en Brent). Sin trazabilidad, la operación del modelo pierde capacidad de explicación y control.

4.3 Estándar de despliegue y productivización

Tras la productivización de Chronos, el siguiente objetivo fue evitar que cada nuevo modelo requiriera un arranque técnico desde cero. En ese contexto se construye el estándar reusable de productivización: una base común para desplegar soluciones de ML con criterios homogéneos en CI/CD, AML, contratos de artefactos y gobierno del ciclo de vida [2–4]. Este estándar se implementa en el repositorio `TYM.MLOpsTemplate`.

El propósito de esta sección no es presentar un nuevo modelo de negocio, sino explicar la transición desde un proyecto puntual hacia un estándar reusable. Mientras una solución aislada se optimiza para un caso concreto, una plataforma reusable prioriza consistencia, mantenibilidad y trazabilidad entre casos distintos.

En la práctica, este estándar estandariza cinco bloques: estructura de repositorio, recursos de Azure Machine Learning (pipelines, componentes y entornos), patrón canónico de componente (YAML + entrypoint + orchestrator) e integración de lifecycle entre MLflow y Teradata. Este diseño reduce variabilidad de arranque y riesgo operativo en nuevos proyectos, porque convierte decisiones implícitas en contratos explícitos.

Como soporte de adopción se han redactado tres documentos complementarios: un `README.md` para visión general y arquitectura, un `USER-GUIDE.md` con pasos de implementación y criterios de finalización, y un `AZURE-DOCS.md` con la referencia técnica de CI/CD y recursos AML. Esta documentación evita dependencias tácitas y facilita transferencia entre equipos.

4.3.1 Objetivo del template y alcance real

El objetivo del template es estandarizar la productivización de modelos, no resolver por sí solo el problema de negocio final. En términos operativos, la plantilla define un flujo estandarizado de despliegue y gobierno sobre el que cada proyecto implementa su lógica funcional específica.

Según la evidencia del repositorio (`README.md`, `USER-GUIDE.md` y `AZURE-DOCS.md`), la cobertura nativa incluye: estructura base por capas, pipeline CI/CD corporativo, recursos AML versionados, contratos de componentes y ciclo de vida con MLflow/Teradata. Ese alcance está alineado con patrones de plataforma MLOps en entornos empresariales [2, 3].

Lo que no cubre sin extensión es justamente la lógica de negocio específica: transformaciones concretas de dominio, entrenamientos reales, reglas de inferencia y diseño final de reportes. Esta frontera evita mezclar responsabilidades y permite que varios casos compartan infraestructura sin forzar el mismo modelo analítico.

Para facilitar adopción por perfiles mixtos, conviene diferenciar dos niveles de uso del estándar:

- **Uso mínimo**: desplegar estructura base, un pipeline de entrenamiento y otro de inferencia con contratos estables y trazabilidad de ejecución.
- **Uso completo**: activar ciclo de vida con registro, promoción/fallback, reporting y gobierno integrado en Teradata.

Esta distinción evita expectativas erróneas: el estándar acelera el arranque técnico, pero no sustituye el diseño del modelo ni las decisiones funcionales del negocio.

La Tabla 4.7 explicita el perímetro de responsabilidad para separar claramente qué aporta la plantilla y qué debe implementar cada proyecto derivado.

Tabla 4.7: *Perímetro funcional de la plantilla: qué resuelve y qué no resuelve.*

Bloque	Cobertura de la plantilla
Estructura de repositorio y contratos	**Sí resuelve**. Proporciona estructura base y contratos reutilizables para componentes y pipelines.
Despliegue por entornos y CI/CD	**Sí resuelve**. Define flujo homogéneo para DEV/PRE/PRO con validaciones tempranas.
Registro y gobernanza de lifecycle	**Sí resuelve**. Aporta integración MLflow/Teradata para trazabilidad de versión y acciones operativas.
Lógica de negocio del modelo	**No resuelve**. Debe implementarse por cada caso de uso (features, algoritmo, reglas de decisión).
Definición de KPIs de negocio	**No resuelve**. La definición y validación dependen del contexto funcional del proyecto.

Para considerar que la plantilla es útil y adoptable en nuevos proyectos, se han usado tres criterios prácticos: (1) que el arranque técnico sea repetible por configuración, (2) que los contratos entre componentes sean estables y auditables, y (3) que la integración de lifecycle mantenga trazabilidad de versión y decisiones de promoción. Estos criterios son los que después se validan con Asset Allocation en la siguiente sección del capítulo.

4.3.2 Estructura técnica y estándares reutilizables

La arquitectura del estándar de productivización se organiza por capas para separar lo que cambia mucho (lógica de negocio) de lo que debe mantenerse estable (infraestructura y contratos). Esta separación es coherente con recomendaciones de ingeniería ML orientadas a reducir deuda técnica y acoplamiento entre experimentación y operación [6, 7].

Como se muestra en la Tabla 4.8, cada capa tiene un rol claro y un nivel de cambio recomendado.

Tabla 4.8: *Capas técnicas del estándar de productivización y límite de cambio recomendado.*

Capa	Elemento	Responsabilidad	Cambio habitual
DevOps	Pipeline corporativo de CI/CD	Stages por entorno y automatización de despliegue.	Bajo
Recursos AML	Definiciones declarativas	Compute, datastore, environment, components, pipelines y schedules.	Bajo-Medio
Glue layer	Adaptadores de componentes	Adaptación entre contratos AML y lógica de negocio mediante entrypoints y orchestrators.	Medio
Core logic	Lógica de negocio	Implementación funcional de preprocess, train, infer, backtest y report según dominio.	Alto
Contratos template	Plantillas base	Interfaces abstractas para imponer consistencia mínima entre proyectos derivados.	Bajo

Tal y como se observa en la Figura 4.4, la estructura de carpetas refleja esa separación de responsabilidades y marca de forma visual donde se espera intervenir en un proyecto nuevo.

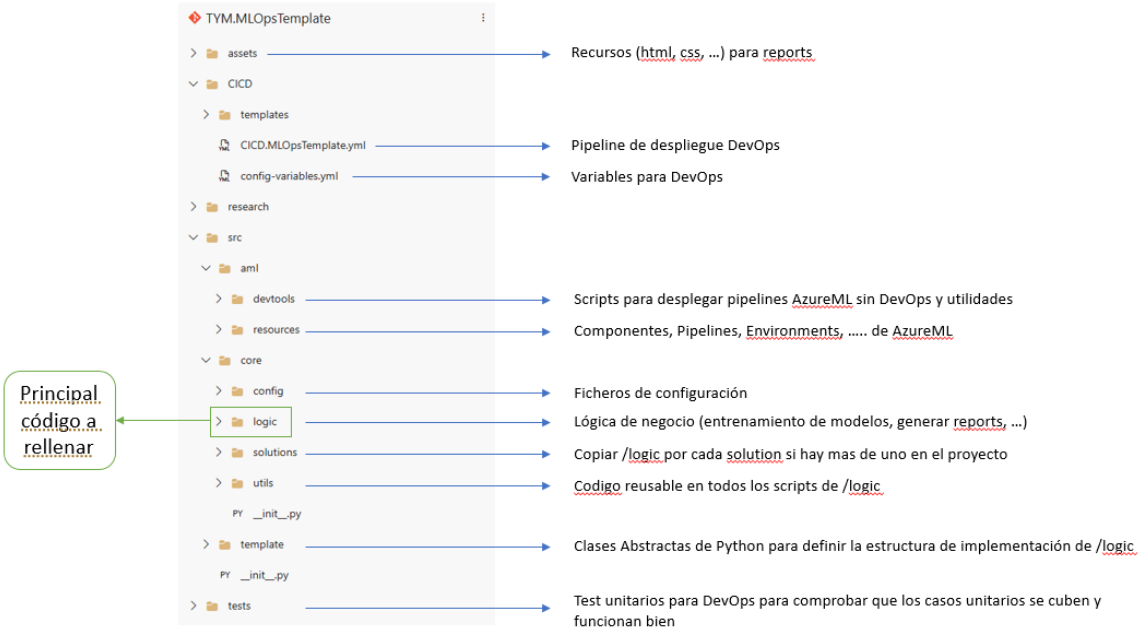

Figura 4.4: *Relación entre capas funcionales de la plantilla y su despliegue operativo.*

Sobre esa base, cualquier proyecto derivado debe respetar tres contratos técnicos mínimos: contrato de entrada/salida de componentes, coherencia entre YAML-`argparse`-orchestrator, y trazabilidad de ejecución por entorno. Mantener estos contratos es lo que permite reutilizar la plataforma sin romper la operación.

4.3.3 Recursos de Azure ML: pipelines, componentes y artefactos

La plantilla define un inventario acotado y declarativo de recursos: dos unidades de cómputo (ejecución y build de imágenes), un datastore, un entorno con Dockerfile y la orquestación de tres pipelines principales y once componentes reutilizables [3, 4].

Como se muestra en la Tabla 4.9, los tres pipelines principales de entrenamiento, inferencia y backtesting cubren el ciclo de vida principal del modelo.

Tabla 4.9: *Pipelines principales del estándar y qué hace cada uno.*

Pipeline	Flujo de componentes	Objetivo
Entrenamiento	Ingesta de datos $Azure$SQL → preprocesado → entrenamiento → registro → self-promotion	Entrenar el modelo, registrarlo y evaluar promoción frente al modelo vigente.
Inferencia	Ingesta de datos $Azure$SQL → preprocesado → inferencia → registro de inferencias $Teradata$ → reporte	Inferir datos nuevos y generar el reporte operativo.
Backtest	Ingesta de datos $Azure$SQL → preprocesado → backtest → reporte	Evaluar el rendimiento histórico del modelo mediante backtesting.

El diseño opera bajo `user_identity`. Cuando el pipeline se ejecuta en Azure, utiliza credenciales corporativas del usuario, evitando el uso de contraseñas compartidas.

A continuación se describe la función de cada uno de los once componentes que integran los pipelines:

4.3.3.1 Capa de Ingesta: `azure_sqldb` y `teradata_load_data`

Los componentes de ingesta seleccionan el origen (Azure o Teradata) según la ubicación de los datos.

- **Entrada (Inputs)**: Los parámetros de conexión se resuelven desde Azure Key Vault y se inyectan en tiempo de ejecución mediante `orchestrator.py`.
- **Salida (Outputs)**: Carpeta `uri_folder` con los datos en bruto.
- **Ejecución**: El script `TeradataDataLoader` abre conexión asíncrona, ejecuta consultas financieras y descarga resultados en streaming al Data Lake. Los datos se almacenan como *CSV* intermedios para aislar el resto del pipeline de incidencias en la base de datos.

4.3.3.2 Capa de Adecuación: `preprocess`

Este componente realiza la limpieza y normalización de datos.

- **Entrada**: Carpeta de datos brutos.
- **Salida**: Datos limpios y preparados para entrenamiento e inferencia.
- **Ejecución**: `preprocess.py` elimina nulos, corrige inconsistencias y aplica escalado. Si se utiliza un `scaler`, se persiste como `joblib` para garantizar consistencia entre entrenamiento e inferencia.

4.3.3.3 Capa de Computación: `train`, `infer` y `backtest`

Esta capa ejecuta el entrenamiento, la inferencia y el backtesting.

- **Pipeline de entrenamiento**: Ejecuta el entrenamiento y genera artefactos de modelo, métricas e hiperparámetros.
- **Pipeline de inferencia y backtesting**: Recuperan el modelo registrado en Azure ML, ejecutan inferencia o backtesting y generan reportes operativos.

4.3.3.4 Orquestación: `orchestrator.py` y `SklearnWrapper`

El *orchestrator* centraliza la configuración y la trazabilidad del entrenamiento. Cuando el componente `train` se inicia, realiza:

1. **Carga de hiperparámetros**: Lectura del archivo de configuración de modelos con parámetros del negocio.
2. **Inyección de dependencias**: Inicialización del `Trainer` con rutas de datos y parámetros.
3. **Encapsulamiento**: Persistencia del modelo mediante `SklearnWrapper`, preservando preprocesadores y dependencias de entorno [9].
4. **Trazabilidad**: Apertura de sesión en MLflow y registro de métricas e hiperparámetros.

4.3.3.5 Shadow Mode y Evaluación de Promoción

Para pasar un modelo a producción, el componente `selfpromotion` aplica un **Shadow Mode**. El orquestador compara el rendimiento del modelo vigente (champion) con el modelo candidato (challenger) en Teradata, usando umbrales de mejora y tolerancia definidos. Si la mejora no es significativa, el modelo se registra sin promoción y queda en modo sombra. La decisión se comunica mediante procedimientos almacenados que actualizan el estado en las tablas de gobernanza.

4.3.3.6 Capa de Gobernanza: El control automático de calidad

La capa de gobernanza establece controles automáticos de calidad y trazabilidad.

- `registry.yml`: Registra el modelo en MLflow y Teradata, estableciendo un *checkpoint* obligatorio para su trazabilidad.
- `selfpromotion.yml`: Decide si el modelo candidato supera al vigente y actualiza el alias `@champion` cuando procede.
- `manual_model_fallback.yml`: Permite revertir a una versión anterior en caso de incidencias operativas.

4.3.3.7 Capa de Consumo: El PDF en el escritorio y datos en Teradata

La fase de consumo asegura que los resultados llegan a los sistemas corporativos y al informe operativo.

- **Registro en Teradata**: `teradata_register_inference` inserta resultados en las tablas corporativas mediante `MLOpsLifeCycle`.
- **Reportes**: `report.py` integra tablas y gráficos en un PDF o HTML, almacenado en el Datastore con sello temporal.

La separación de responsabilidades se mantiene entre definición de recursos, lógica de ejecución y orquestación. Este desacoplamiento facilita la portabilidad del núcleo de negocio si la plataforma cloud cambiara.

4.3.3.8 Infraestructura como Código (IaC): Terraform y Bicep

Para que la plantilla sea desplegable en cualquier entorno sin errores humanos, la infraestructura se define mediante **Infraestructura como Código (IaC)** [2, 4].

En lugar de crear recursos desde el portal, se usan archivos de **Terraform** y **Bicep** que definen:

- **Cómputo**: Instancias estándar con autoescalado a cero.
- **Seguridad**: Reglas de firewall (NSG), *Private Endpoints* y permisos RBAC.
- **Servicios auxiliares**: Azure Key Vault, Container Registry (ACR) y Application Insights.

Esto permite desplegar entornos de MLOps de forma reproducible mediante automatización en DevOps.

La Tabla 4.10 resume el flujo de datos entre componentes. La transferencia de información se implementa mediante carpetas inmutables en disco (`uri_folder`), evitando dependencias de variables de entorno no controladas.

Tabla 4.10: *Cómo fluyen los archivos entre los componentes.*

Productor	Artefacto	Tipo AML	Consumidor
Ingesta $Azure$SQL	CSV con los datos brutos	`uri_folder`	Preprocesado
Preprocesado	Dataset limpio y scaler (`.joblib`)	`uri_folder`	Entrenamiento / Inferencia / Backtest
Entrenamiento	Modelo y metadatos de ejecución	`uri_folder`	Registro
Registro	Señal de validación operativa	`uri_file`	Self-promotion
Inferencia	JSON con las predicciones diarias	`uri_folder`	Registro de inferencias, Reporte
Backtest	Informe histórico detallado	`uri_folder`	Reporte
Reporte	PDF final para el usuario	`uri_folder`	Consumo externo

Con esta base, tanto Chronos como Asset Allocation comparten el mismo esquema de infraestructura y contratos de pipeline, lo que reduce la variabilidad operativa [2].

4.3.4 Arquitectura CI/CD basada en YAML

Para asegurar consistencia operativa, la plantilla incluye una arquitectura de **Integración y Despliegue Continuo (CI/CD)** en Azure DevOps. El archivo `azure-pipelines.yml` define el flujo de trabajo.

Este flujo se organiza en tres bloques principales que se ejecutan en cada envío a la rama principal:

1. **Validación (CI)**: Ejecuta linters (`ruff`) y pruebas. Si falla, el pipeline se detiene.
2. **Infraestructura**: Verifica Workspace y clústeres con Command-Line Interface (CLI) v2 de Azure y aplica cambios declarativos.
3. **Ejecución de jobs**: Lee `pipeline.yml` y lanza entrenamientos o inferencias con la imagen Docker y los datos del Datastore definidos.

4.3.5 Estrategia de pruebas: seguridad y calidad del código

La plantilla establece una estrategia de pruebas con **Pytest** para reducir errores en producción.

4.3.5.1 Pruebas unitarias y uso de mocks

Dado que los componentes dependen de Azure y Teradata, se emplea **mocking** para simular respuestas de base de datos [2]. Esto permite validar la lógica en integración local antes de ejecutar en infraestructura corporativa.

4.3.5.2 Pruebas de humo del pipeline

Además de pruebas unitarias, se ejecutan pruebas de humo que validan sintaxis de YAML y coherencia de rutas de entrada/salida antes de iniciar clústeres de ejecución.

4.3.6 Documentación de la clase MLOpsLifeCycle

La clase `MLOpsLifeCycle` actúa como interfaz entre el código de negocio y la infraestructura de gobernanza. Sus métodos principales afectan al ciclo de vida del modelo:

- `init_run(experiment_name)`: Abre conexión con MLflow e identifica entorno (DEV/PRE/PRO), recuperando `run_id` cuando el job se lanza desde Azure ML.
- `log_metrics(metrics_dict)`: Registra métricas y filtra aquellas consideradas críticas para su envío a Teradata.
- `save_model(model, path)`: Serializa el modelo con versiones de librerías, alertando ante incompatibilidades futuras.
- `get_champion_model()`: Recupera el modelo con la etiqueta `@champion` para inferencia.

4.3.6.1 Lógica del componente Registry

El componente *Registry* gestiona estados y trazabilidad. Su lógica interna incluye:

1. **Verificación de artefactos**: Comprueba presencia de `.joblib`, `scaler` y `hparams.json`.

2. **Registro en Teradata**: Ejecuta el procedimiento almacenado corporativo para vincular la versión con la tabla de gobernanza.
3. **Marcado temporal**: Asigna una marca de tiempo precisa para trazabilidad operativa.

Este nivel de detalle asegura que cada modelo en producción está documentado y es rastreable.

4.3.7 Gobernanza de modelos en Teradata: solutions, model-ids, self-promotion, manual model fallback y wrappers en MLflow

La gobernanza de modelos se concentra en la clase `MLOpsLifeCycle`, que actúa como punto único de integración entre MLflow y Teradata. Esta centralización evita duplicar lógica de lifecycle en componentes dispersos y mejora auditabilidad en registro, promoción, inferencia y fallback [9].

La identificación operativa se apoya en tres ejes: `SolutionId` (solución), `model_id` (modelo) y `version` (revisión registrada). En el registro, `model_id` se normaliza para MLflow (reemplazo de `/` por `.`), y después se persisten metadatos técnicos en Teradata mediante la Stored Procedure (SP) `MlOps.LifeCycle`. Con esto, cada decisión queda trazada por entorno, run y versión.

En la práctica, la relación entre estos identificadores funciona así:

- `SolutionId`: identifica el caso de uso (por ejemplo, una solución concreta de pricing o asignación);
- `model_id`: identifica una familia de modelo dentro de esa solución;
- `version`: identifica la revisión exacta promovida o descartada en cada iteración.

Esta jerarquía permite responder preguntas operativas sin ambigüedad: qué solución se ejecutó, qué modelo concreto generó la salida, qué versión se promocionó y bajo qué métrica. Es el punto que hace posible auditoría real cuando hay que justificar una predicción o analizar una incidencia de producción.

En la implementación concreta del template, `MLOpsLifeCycle` no es solo un helper de registro; actúa como fachada de gobierno con una API explícita para cada evento del ciclo de vida. A nivel de código, la clase define claves canónicas del payload (por ejemplo `ModelAction`, `SolutionId`, `ModelId`, `ModelVersion`, `MainPromotionMetric`, `DeltaMainMetric`, `Maximize`) y centraliza la llamada a la SP corporativa mediante `CALL MlOps.LifeCycle('<json>')`. Esta normalización reduce divergencia entre proyectos porque evita construir payloads ad hoc en cada componente.

Además, la clase incorpora resolución de entorno para auditoría (lab/dev/pre/qa/pro) y usa secretos de Key Vault para conexión a Teradata y contexto de workspace. Esto permite que el mismo código funcione en local y en Azure ML con mínimos cambios de configuración, manteniendo traza homogénea de *run* y entorno.

Entrando al detalle funcional del fichero `mlops_life_cycle.py`, la gobernanza no se limita a registrar modelos: también estandariza inicialización de contexto, selección de versión para inferencia, registro de outputs operativos y persistencia de evidencia técnica (métricas, parámetros y artefactos).

Esta cobertura amplia es relevante para Teradata porque evita que cada pipeline implemente su propia versión de "gobierno" y asegura que los eventos críticos pasan por una única pasarela de control [9].

El flujo de gobierno empieza en inicialización con `__init__(...)` y `_init_mlflow(...)`. Ahí se decide si la ejecución está en contexto Azure ML o en contexto local, se fija tracking de MLflow, se instancia el cliente de registry y, si aplica, se abre conexión a Teradata usando secretos corporativos. Operativamente, este bloque garantiza que cualquier acción posterior (registro, promoción o inferencia) tenga contexto consistente de experimento, run y credenciales.

Después, la clase separa utilidades internas de acciones de negocio. `_registry_name(...)` normaliza nombres para registry, `_get_latest_model_version(...)` resuelve última versión efectiva y `_call_procedure(...)` encapsula la invocación a la SP con manejo de result sets y trazas de ejecución. Esta separación importa porque concentra en un único punto el contrato técnico con Teradata: serialización del payload, ejecución SQL y lectura de respuesta tabular.

Sobre ese núcleo, los métodos de gobierno de ciclo de vida implementan políticas concretas:

- `model_registry(...)` registra en MLflow (*pyfunc* o *sklearn*), consolida métricas/parámetros desde child runs y persiste metadatos de versión en Teradata con acción `ModelRegistry`.
- `evaluate_selfpromotion(...)` dispara evaluación de autopromoción con acción `AutoPromotion`, métrica principal, delta mínimo y sentido de optimización (maximizar/minimizar).
- `inference_champion_solution(...)` consulta champion por `SolutionId` en Teradata (acción `InferenceChampionSolution`) y carga exactamente esa versión desde registry.
- `inference_registry_latest_version(...)` habilita estrategia alternativa sin champion/challenger: usar la última versión del registry.
- `manual_model_fallback(...)` ejecuta contingencia explícita (acción `ManualModelFallback`) para volver a un estado operativo estable.
- `register_inference_teradata(...)` inserta huella de inferencia (fecha, modelo, versión y output JSON) en tabla operacional de Teradata.

Además, la clase incorpora una capa de observabilidad técnica en MLflow que también refuerza gobernanza. Incluye métodos de etiquetado, logging y recuperación de artefactos (por ejemplo, `set_pipeline_tag(...)`, `log_pipeline_metrics(...)`, `log_artifact_file(...)` y `load_artifact_from_run(...)`), junto con variantes equivalentes para parámetros, ficheros e imágenes. Aunque estos métodos no llaman directamente a la SP, sí son clave para auditoría porque dejan evidencia recuperable del contexto exacto de ejecución y de los artefactos usados en train/infer [9].

Tabla 4.11: *Catálogo funcional de `MLOpsLifeCycle` y contribución a la gobernanza en Teradata.*

Bloque	Método(s)	Propósito operativo
Inicialización y contexto	`__init__`, `_init_mlflow`, `start_run`, `close_connection`	Configura tracking local/Azure, cliente MLflow, conexión Teradata y ciclo de apertura/cierre para ejecución segura de pipelines.
Normalización y versión	`_registry_name`, `_get_latest_model_version`, `load_pyfunc_model`	Estandariza nombres de registry, resuelve versión efectiva y carga modelos por versión, stage o latest sin ambigüedad operativa.
Puente Teradata	`_call_procedure`	Serializa payload JSON, ejecuta `CALL MlOps.LifeCycle(...)` y devuelve resultados tabulares para decisiones de promoción/inferencia.
Registro de modelos	`model_registry` (`ModelRegistry`)	Registra modelo (*pyfunc/sklearn*), agrega métricas e hiperparámetros de child runs y persiste `SolutionId`, `ModelId`, `ModelVersion`, `AMLWorkspace`, `JobRun`, métricas JSON e hiperparámetros JSON.
Autopromoción	`evaluate_selfpromotion` (`AutoPromotion`)	Ejecuta regla champion/challenger con métrica principal, delta mínimo y criterio de maximización/minimización para promover o bloquear versiones.
Selección para inferencia	`inference_champion_solution` (`InferenceChampionSolution`)	Obtiene champion por solución en Teradata y carga esa versión exacta desde registry para inferencia gobernada.
Selección alternativa	`inference_registry_latest_version`	Permite operar por latest del registry cuando el caso no requiere política champion/challenger.
Contingencia	`manual_model_fallback` (`ManualModelFallback`)	Fuerza retorno a estado estable cuando se detecta degradación o incidencia en la versión activa.
Huella de inferencia	`register_inference_teradata`	Persiste salida operativa (timestamp, modelo, versión y output JSON) en tabla de inferencias para auditoría posterior.
Trazabilidad técnica	`set_pipeline_tag`, `log_pipeline_params`, `log_pipeline_metrics`, `log_pipeline_plots`, `log_artifact_file`, `log_artifacts_dir`, `log_dict_artifact`, `log_image_artifact`, `log_bytes_as_artifact`	Registra evidencia de ejecución en MLflow (tags, parámetros, métricas y artefactos) para reproducibilidad y diagnóstico.
Recuperación de evidencia	`load_artifact_from_run`, `load_pickle_from_run`	Rehidrata artefactos y objetos de runs previos para reproducir inferencias, validar transformaciones y auditar cadenas de dependencia.

Como se muestra en la Tabla 4.11, la gobernanza queda definida como contrato de acciones y no como lógica distribuida en notebooks o scripts puntuales. Este enfoque aporta dos ventajas directas: minimiza riesgo de inconsistencia entre proyectos y permite auditar por tipo de evento de lifecycle.

Como se muestra en la Figura 4.5, el componente de registro encapsula el paso de salida de entrenamiento a gobierno de versión. Ese trigger de registro es el que habilita la fase de promoción semiautomática.

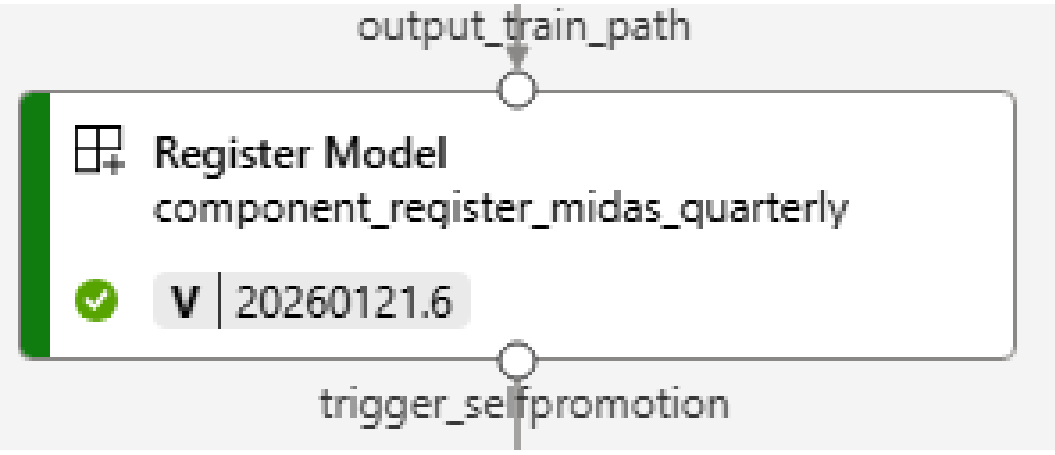

Figura 4.5: *Componente de registro de modelo en la plantilla (MLflow + Teradata).*

Tal y como se observa en la Figura 4.6, la autopromoción se implementa como etapa explícita de pipeline: compara challenger/champion según métrica principal y umbral de mejora. Si el criterio no se cumple o hay incidencia operativa, la plantilla mantiene componente de contingencia `manual_model_fallback` para recuperar estado estable.

Figura 4.6: *Componente de self-promotion para evaluación y promoción de versiones.*

Además del registro de versiones de modelo, la gobernanza incluye el registro de resultados de inferencia en Teradata para mantener trazabilidad de salida operacional por ejecución. Como se muestra en la Figura 4.7, el componente `register_inference` persiste los outputs de inferencia y deja huella auditable del resultado publicado.

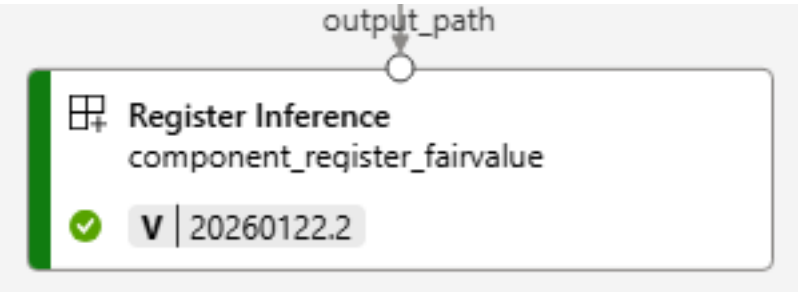

Figura 4.7: *Componente de registro de inferencias en Teradata.*

La capa de gobernanza también incorpora carga de datos desde Teradata como paso de entrada estandarizado. Tal y como se observa en la Figura 4.8, el componente `select_teradata` encapsula la consulta y entrega el artefacto de salida para el resto del pipeline, manteniendo separación entre acceso a datos y lógica de negocio.

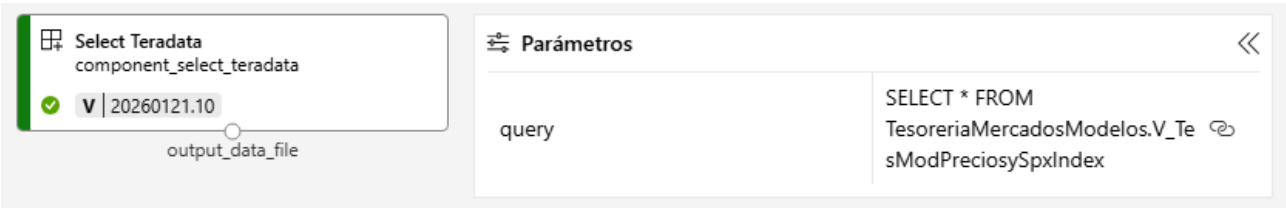

Figura 4.8: *Componente de carga de datos desde Teradata (Select Teradata).*

El uso de wrappers `pyfunc` (por ejemplo `SklearnWrapper`) unifica la interfaz operacional del modelo registrado: permite conservar preprocesador/hiperparámetros junto al modelo y desacopla el formato de registro de la lógica interna de predicción. Esta estrategia facilita portabilidad entre train e infer y simplifica mantenimiento cuando cambia el algoritmo.

Otro detalle técnico relevante del registro es cómo se consolidan métricas e hiperparámetros antes de persistir. `model_registry` busca los child runs del run raíz en MLflow y compone `MetricsJSON` y `HyperParamsJSON` para almacenarlos en Teradata junto con `AMLWorkspace`, `JobRun` y fecha de expiración opcional. Con esto, la ficha de versión no depende de un único componente, sino del contexto completo de ejecución.

En inferencia, la clase también resuelve una necesidad práctica de operación: decidir origen del modelo sin duplicar lógica por proyecto. Si el caso trabaja con champion/challenger, se usa `inference_champion_solution`; cuando no existe esa política, se usa la estrategia de última versión del registro. Esta bifurcación es especialmente útil para explicar por qué distintos proyectos (Chronos vs Asset Allocation) aplican políticas de promoción distintas manteniendo la misma base técnica de lifecycle.

En conjunto, el esquema de lifecycle de la plantilla ofrece tres controles clave para operación corporativa: trazabilidad de versiones, reglas explícitas de promoción y mecanismo de recuperación ante fallo. Ese trío es el que convierte un pipeline funcional en un proceso gobernado de verdad.

4.3.8 Evidencias operativas y trazabilidad

La plantilla no solo define pipelines, también establece un esquema de evidencias para auditoría y recuperación. Cada acción del ciclo de vida produce un artefacto que queda registrado en MLflow y, cuando aplica, en Teradata. Esta doble persistencia reduce ambigüedad en la versión vigente y facilita reconstrucción de decisiones [9].

La Tabla 4.12 sintetiza qué evidencias se generan y qué riesgo operativo mitigan.

Tabla 4.12: *Evidencias operativas y riesgo mitigado en la plantilla.*

Acción	Evidencia generada	Riesgo mitigado
Registro de modelo	Versión y métricas en MLflow	Promoción sin trazabilidad
Auto-promoción	Resultado de comparación *champion/challenger*	Cambios sin criterio explícito
Registro de inferencias	Output persistido en Teradata	Imposibilidad de auditoría posterior
Fallback manual	Versión anterior registrada	Recuperación lenta ante incidencia

4.3.9 Control de cambios en la plantilla

Un riesgo recurrente en plantillas reutilizables es la deriva entre proyectos: pequeñas variaciones no documentadas terminan creando versiones incompatibles. Para evitarlo, la plantilla define un

conjunto mínimo de contratos que deben permanecer estables entre proyectos derivados (nombres de componentes, tipos de artefactos y estructura de YAML) [2].

Este control de cambios se refuerza con validaciones en CI/CD que verifican consistencia de rutas, existencia de componentes y coherencia de entradas/salidas antes de desplegar. El objetivo no es frenar la evolución del código, sino asegurar que cualquier variación queda trazada y puede auditarse en retrospectiva [4].

4.3.10 Compatibilidad entre proyectos

Para mantener la reutilización real, la plantilla necesita reglas de compatibilidad explícitas. La Tabla 4.13 resume criterios mínimos que permiten que proyectos distintos mantengan interoperabilidad y reduzcan costes de mantenimiento.

Tabla 4.13: *Criterios de compatibilidad en la plantilla MLOps.*

Contrato	Criterio	Impacto si se rompe
Entradas/salidas	Tipos `uri_folder` consistentes	Fallos en orquestación y trazabilidad
Nombres de componentes	Convención estable	Dificultad de mantenimiento y soporte
Configuración YAML	Parámetros obligatorios comunes	Ejecuciones inconsistentes entre entornos
Registro de versiones	Uso homogéneo de MLflow	Auditoría incompleta o ambigua

La pieza clave en la base de datos es el procedimiento almacenado `MlOps.LifeCycle`. Este SP centraliza la lógica de negocio del banco sobre el estado operativo del modelo. Recibe un objeto JavaScript Object Notation (JSON) que describe la acción y los metadatos, y realiza las siguientes operaciones atómicas:

- **Validación de Linaje**: Comprueba que el `AMLWorkspace` y el `JobRun` existen y son válidos antes de registrar nada.
- **Cálculo de Promoción**: Compara las métricas del JSON contra el Champion actual en las tablas de histórico. Si la mejora supera el umbral (`DeltaMainMetric`), actualiza el puntero de Champion automáticamente.
- **Persistencia de Auditoría**: Guarda una copia exacta del JSON recibido en una tabla de auditoría Large Object (LOB) para reconstrucción forense de decisiones.

Este diseño desacoplado es fundamental: Azure ML se encarga de la computación, pero la fuente de verdad sobre el modelo oficial reside en Teradata. Así, otros sistemas del banco solo consultan la base de datos para conocer la versión vigente, sin depender de detalles de infraestructura.

4.4 Asset Allocation: validación de la transferibilidad del enfoque

Asset Allocation se utiliza como caso de validación de la transferibilidad del estándar. El reto no era solo entrenar otro modelo, sino mantener los mismos contratos YAML, trazabilidad en MLflow, gobernanza en Teradata y entrega automática de informes a negocio [3, 4, 9]. La validación consistió en integrar el modelo en la misma tubería sin reescribir la plataforma.

4.4.1 Implementación sobre el estándar reusable

La implementación confirma que se reutiliza el estándar sin cambios estructurales relevantes:

- La misma organización de carpetas (pipelines, componentes genéricos, núcleo de código).
- El mismo orquestador CI/CD en DevOps para DEV, PRE y PRO.
- La misma trinidad algorítmica: YAML + script de Python + entorno.
- La clase `MLOpsLifeCycle` gobernando el guardado en bases de datos.

Como evidencia documental, en Asset Allocation se ha redactado un `README.md` que describe el flujo de entrenamiento e inferencia, la política de registro en MLflow y la razón de desactivar la autopromoción por falta de histórico comparativo. Esta documentación refuerza la trazabilidad del caso de transferencia.

Como se ve en la Figura 4.9, el flujo continuo en Azure DevOps es idéntico al estándar que diseñamos.

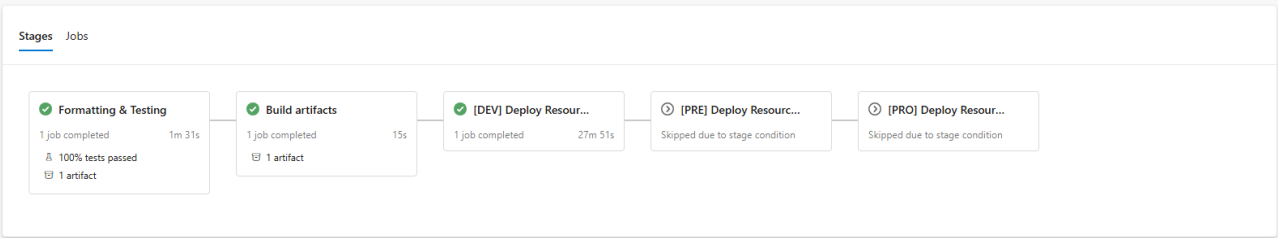

Figura 4.9: *Tubería DevOps de Asset Allocation usando el estándar genérico.*

4.4.2 Arquitectura paralela para la gestión de perfiles de riesgo

Operativamente, Asset Allocation gestiona nueve perfiles de riesgo en paralelo, desde perfiles líquidos (BP_S) hasta perfiles de gestión discrecional (GDC_XL).

En el pipeline de entrenamiento se utiliza un único componente de limpieza de datos que alimenta nueve componentes de entrenamiento simultáneos, con registros independientes en base de datos. Como se observa en la Figura 4.10, esta arquitectura permite calcular nueve estrategias en un único ciclo de ejecución.

En inferencia se mantiene el mismo patrón: una limpieza de datos, nueve predicciones, nueve registros en Teradata y un nodo final de consolidación de informes (Figura 4.11).

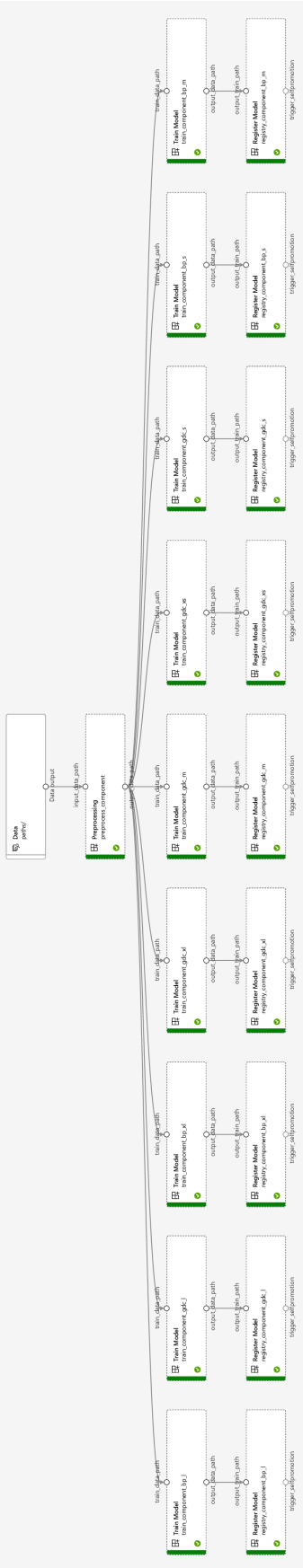

Figura 4.10: *Pipeline de entrenamiento calculando nueve perfiles simultáneamente.*

Para desacoplar el entrenamiento de la disponibilidad de la base de datos, los datos descargados se inyectan en modo solo lectura mediante montaje del datastore. Esta estrategia permite continuar entrenamientos con datos en caché cuando la base de datos no está disponible.

4.4.3 Parametrización y configuración de perfiles

La parametrización financiera se concentra en un archivo de configuración central, donde se definen los nueve perfiles con sus niveles de riesgo, objetivos y curvas de referencia.

El código base permanece estable: todos los perfiles utilizan el motor de momentum común y se ajustan mediante la configuración. Esta separación valida que el CI/CD opera sobre cambios declarativos sin requerir modificaciones en scripts de producción.

La ejecución genera artefactos técnicos (modelo, métricas e hiperparámetros) y salidas operativas (predicciones, tablas Comma Separated Values (CSV) y gráficos). La utilidad para negocio se materializa en los informes consolidados, como se aprecia en la Figura 4.12.

4.4.4 Contrato de configuración y control de cambios

El uso de `config_models_solution.yml` actúa como contrato de cambio: toda modificación relevante en perfiles se expresa en configuración, no en código. Esto reduce el riesgo de divergencias entre entornos y permite auditar qué cambió y cuándo [2].

La Tabla 4.14 resume parámetros clave y su justificación operativa. El objetivo es mantener trazabilidad sin bloquear la evolución del modelo.

Tabla 4.14: *Parámetros clave del contrato de configuración en Asset Allocation.*

Parámetro	Función	Riesgo controlado
`lookback_days`	Ventana de cálculo de momentum	Evitar sobreajuste a ruido corto
`rebalance_freq`	Frecuencia de cambios	Control de coste de transacción
`risk_free_rate`	Base de ratios de riesgo	Coherencia en métricas comparables
`constraints_group`	Límites regulatorios por activo	Cumplimiento de restricciones legales

4.4.5 Análisis de la orquestación y lógica de ejecución

Un único módulo ejecuta la limpieza de datos y nueve ramas paralelas resuelven el entrenamiento para perfiles BP y GDC dentro del pipeline de entrenamiento.

El despliegue reutiliza la sintaxis del estándar original adaptada a carteras financieras. En el YAML de entrenamiento se definen identificadores de experimento, solución y modelo; el orquestador ejecuta el proceso de entrenamiento con esos parámetros.

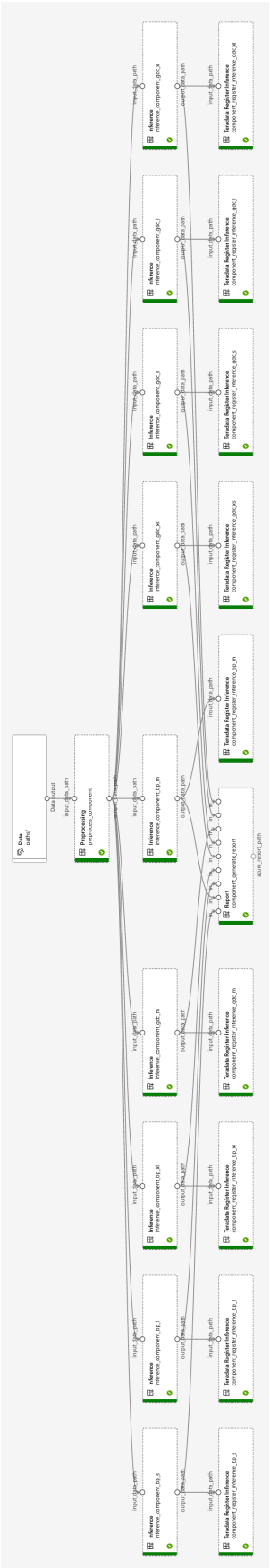

Figura 4.11: *Inferencia paralela de nueve perfiles y consolidación final del reporte.*

4.4.5.1 Detalle técnico: implementación basada en reglas financieras

El modelo *ADM_Prop_Momentum* no es un algoritmo de predicción estocástico, sino un esquema determinista basado en reglas de momento. En este contexto, el entrenamiento no optimiza parámetros mediante gradientes, sino que aplica restricciones regulatorias y criterios de riesgo para calcular pesos óptimos y generar los artefactos `model.joblib` y `metrics.json`.

Este caso refuerza que la arquitectura MLOps es agnóstica al tipo de algoritmo. El mismo contrato YAML permite operar tanto modelos deterministas como modelos de aprendizaje automático sin cambios en la infraestructura [2].

4.4.5.2 Lógica algorítmica: Dual Momentum y PAA

El motor financiero implementa estrategias de momentum adaptativo. En perfiles agresivos se aplica **Dual Momentum**, que combina un filtro de momentum relativo entre activos y un filtro de momentum absoluto para excluir activos con rendimiento negativo [40].

En perfiles defensivos se utiliza Protective Asset Allocation (PAA). El algoritmo calcula el porcentaje de activos con momentum negativo y, si supera el umbral de protección definido, reorienta el capital hacia activos refugio de forma proporcional [41]. El umbral y el factor de agresividad se configuran en los YAML del proyecto.

4.4.5.3 Implementación del modelo Black-Litterman

Para perfiles GDC, el modelo contempla Black-Litterman (BL). A diferencia del enfoque media-varianza, BL permite incorporar opiniones de los gestores sobre la evolución del mercado [42].

La implementación combina retornos de equilibrio con opiniones y objetivos definidos por el gestor, incorporando incertidumbre en dichas opiniones para generar una distribución coherente de activos.

Esta integración aporta una síntesis entre evidencia histórica y criterio experto, manteniendo coherencia estadística.

4.4.5.4 Métricas de control de riesgo y solvencia

En este repositorio, el desempeño se evalúa por el equilibrio entre rentabilidad y riesgo. El pipeline de inferencia calcula tres métricas de riesgo sobre la serie temporal resultante:

1. **Sharpe Ratio**: Exceso de retorno por unidad de volatilidad, comparado con la tasa libre de riesgo.
2. **Sortino Ratio**: Similar al Sharpe, pero penalizando únicamente la volatilidad de las caídas.
3. **Maximum Drawdown (MDD)**: Máxima caída desde un pico previo hasta un valle, indicador crítico de tolerancia al riesgo.

Estas métricas se registran junto a la versión del modelo y el perfil de riesgo correspondiente, permitiendo comparar ejecuciones entre periodos sin perder trazabilidad de configuración. La coherencia de este registro es clave para interpretar cambios en la cartera sin atribuirlos erróneamente a variaciones de mercado o de datos.

4.4.5.5 Diseño de Datos y Contratos de Configuración

La robustez de Asset Allocation se apoya en una configuración declarativa: el comportamiento de los nueve perfiles se define en un contrato `YAML` que el pipeline inyecta en cada ejecución. Los parámetros clave acordados con negocio son:

- `lookback_days`: Cuántos días atrás mira el modelo para calcular las medias móviles de momentum.
- `rebalance_freq`: Si el modelo propone cambios cada día o cada mes, minimizando así las comisiones de transacción.
- `risk_free_rate`: El valor de referencia para el cálculo de los ratios anteriores.
- `constraints_group`: Diccionario que mapea categorías de activos (ej. .Equity US") con sus límites legales de inversión.

Esta arquitectura desacoplada permite ajustar el comportamiento de la cartera modificando archivos de configuración versionados, sin alterar la lógica de Python.

4.4.5.6 El reto de las Restricciones: Redistribución Iterativa

Uno de los retos de programación fue la lógica de aplicación de restricciones. En banca existen límites regulatorios (por ejemplo, porcentaje máximo por activo). Cuando el algoritmo propone pesos iniciales, la función realiza una redistribución iterativa para cumplir restricciones individuales y grupales y mantener la suma total en 1.0.

Tras generar el modelo cuantitativo, el nodo de **Registro** guarda el `model_id` y asienta la versión en Teradata. En inferencia, el pipeline ejecuta los nueve perfiles e inserta la distribución sugerida de cartera mediante carga masiva en la base de datos, cerrando el ciclo con un `component_report.yml` unificado.

En el ciclo de vida, la autopromoción *champion/challenger* se mantuvo desactivada por falta de histórico suficiente. Se operó con la última versión registrada como regla de selección.

4.4.6 Despliegues segmentados: DEV, PRE y PRO

El pipeline orquestador central sigue el estándar definido. El despliegue se organiza en las siguientes etapas:

1. **Formateo y pruebas**: Se instala Python 3.10, se ejecuta `ruff check` y se lanzan pruebas unitarias con `pytest --cov`. Si falla, el despliegue se detiene.
2. **Empaquetado (Build)**: Se genera un artefacto con el código del proyecto para asegurar consistencia entre validación y despliegue.
3. **Ejecución de pipeline**: Se selecciona el entorno (`DEV`, `PRE`, `PRO`) y se aplican los parámetros de configuración correspondientes.

La configuración de recursos y conexiones corporativas se centraliza en un archivo de variables por entorno. La separación entre secretos, parámetros de infraestructura y código funcional facilita la aprobación de despliegues en producción.

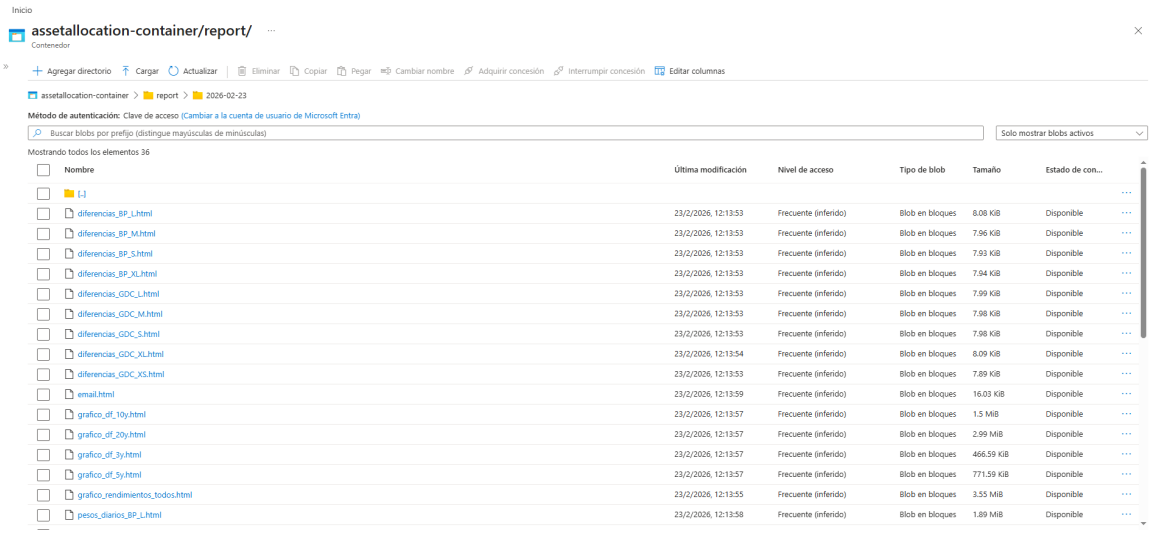

Figura 4.12: *Un ejemplo de la tabla resultante final que genera la inferencia agregada.*

Como se observa en la Figura 4.13, el reporte se entrega también en formato HTML para su consumo interno.

LABORAL KUTXA

Informe de Asset Allocation

Fecha de generacion: 2026-02-20

TFM - Asset Allocation

Resumen de asignacion

Comparativa entre modelo, benchmark y desviaciones por perfil.

Asset Allocation (Modelo)

	S	M	L	XL	GDC_XS	GDC_S	GDC_M	GDC_L	GDC_XL
Monetario	15.0	40.00	20.0	20.000000	100.0	13.333333	20.0	10.000000	10.000000
Deuda_Publica	15.0	10.00	10.0	4.999999	0.0	13.333333	10.0	11.250001	9.999999
RF_Privada_grado_de_inversion	15.0	10.00	10.0	5.000000	0.0	13.333333	10.0	11.250001	10.000000
RF_High_Yield__RF_Emergente	35.0	10.00	10.0	5.000000	0.0	30.000000	10.0	11.250001	10.000001
RV_Europa	4.5	6.25	10.0	13.000000	0.0	7.500000	10.0	11.249999	12.000000
RV_EEUU	4.5	6.25	10.0	13.000000	0.0	5.000000	10.0	11.249999	12.000000
RV_Japon	4.5	6.25	10.0	13.000000	0.0	5.000000	10.0	11.249999	12.000000
RV_Emergente	2.0	5.00	10.0	13.000000	0.0	5.000000	10.0	11.249999	12.000000
RV_Otros	4.5	6.25	10.0	13.000000	0.0	7.500000	10.0	11.250001	12.000000

Asset Allocation vs Benchmark

	BP_S	BP_M	BP_L	BP_XL	GDC_XS	GDC_S	GDC_M	GDC_L	GDC_XL
Monetario	-5.0	25.00	1.250000e+01	2.000000e+01	100.0	13.333333	20.0	10.000000	10.000000
Deuda_Publica	-20.0	-15.00	2.500000e+00	-9.934107e-07	0.0	13.333333	10.0	11.250001	9.999999
RF_Privada_grado_de_inversion	-10.0	-10.00	-5.000000e+00	4.967054e-07	0.0	13.333333	10.0	11.250001	10.000000
RF_High_Yield__RF_Emergente	25.0	-10.00	-1.000000e+01	-5.000000e+00	0.0	30.000000	10.0	11.250001	10.000001
RV_Europa	1.2	-0.35	-6.500000e+00	-1.340000e+01	0.0	7.500000	10.0	11.249999	12.000000
RV_EEUU	0.8	-1.15	-8.500000e+00	-1.660000e+01	0.0	5.000000	10.0	11.249999	12.000000
RV_Japon	4.2	5.65	8.500000e+00	1.060000e+01	0.0	5.000000	10.0	11.249999	12.000000
RV_Emergente	1.3	3.60	6.500000e+00	7.400000e+00	0.0	5.000000	10.0	11.249999	12.000000
RV_Otros	2.5	2.25	1.776357e-14	-3.000000e+00	0.0	7.500000	10.0	11.250001	12.000000

Benchmark

	BP_S	BP_M	BP_L	BP_XL	GDC_XS	GDC_S	GDC_M	GDC_L	GDC_XL
Monetario	20.0	15.0	7.5	0.0	0	0	0	0	0
Deuda_Publica	35.0	25.0	7.5	5.0	0	0	0	0	0
RF_Privada_grado_de_inversion	25.0	20.0	15.0	5.0	0	0	0	0	0
RF_High_Yield__RF_Emergente	10.0	20.0	20.0	10.0	0	0	0	0	0
RV_Europa	3.3	6.6	16.5	26.4	0	0	0	0	0
RV_EEUU	3.7	7.4	18.5	29.6	0	0	0	0	0
RV_Japon	0.3	0.6	1.5	2.4	0	0	0	0	0
RV_Emergente	0.7	1.4	3.5	5.6	0	0	0	0	0
RV_Otros	2.0	4.0	10.0	16.0	0	0	0	0	0

Figura 4.13: *El HTML consolidado que reporta Asset Allocation al terminar todo el flujo.*

4.4.7 Incidencias operativas y lecciones aprendidas

Se identificaron tres áreas de fricción: dependencia de Azure Key Vault (si el secreto falla, se interrumpe el pipeline), baja cobertura de pruebas automatizadas y una referencia a *Backtest* en DevOps sin implementación operativa del flujo de backtesting end-to-end.

En Asset Allocation, el backtesting existe como cálculo integrado en la inferencia, pero no como pipeline independiente con gobierno completo. Por tanto, la transferibilidad se valida en entrenamiento, inferencia, reporting y gobierno, pero no en backtesting end-to-end. Esta diferencia se asume como una limitacion formal del caso de validacion, no como ausencia de capacidad tecnica.

La desactivación de la validación *champion/challenger* se consideró temporal. Con mayor histórico de ejecución, podrá activarse una selección automática basada en métricas; mientras tanto, se priorizó la trazabilidad de versiones y cambios.

La Tabla 4.15 sintetiza qué se reutilizó y qué se adaptó entre Chronos y Asset Allocation, para hacer explícita la evidencia de transferibilidad.

Tabla 4.15: *Comparativa de transferibilidad: Chronos Engine vs Asset Allocation.*

Dimensión	Qué se reutiliza	Qué se adapta / aprendizaje
Arquitectura CI/CD	Pipeline corporativo por entornos y validaciones	Ajuste de parámetros y variables específicas de carteras
Contratos YAML	Entradas/salidas y estructura de componentes	Nuevos contratos de configuración financiera por perfil
Gobernanza lifecycle	Registro en MLflow y persistencia en Teradata	Regla de promoción adaptada (sin auto-promoción por histórico limitado)
Reporting	Patrón de generación de artefacto final	Formato y contenido orientados a recomendaciones de asignación
Lección principal	La infraestructura reusable sí transfiere entre dominios financieros; la principal adaptación se concentra en reglas de negocio y configuración, no en la plataforma.	

4.4.8 Conclusión del caso

Asset Allocation aporta evidencia de la transversalidad del enfoque MLOps. Se han operativizado perfiles financieros, predicciones deterministas y métricas de riesgo bajo el mismo esquema de DevOps empleado en Chronos.

La estandarización del despliegue permite que la diversidad algorítmica no requiera cambios en la infraestructura, respaldando la hipótesis central del TFM en el marco corporativo analizado.

4.4.9 Modelado de Costes de Transacción y Slippage

Este bloque se presenta como una extension funcional del caso, no como el nucleo de validacion de transferibilidad del estandar. Su objetivo es complementar la interpretacion financiera del resultado operativo sin alterar la arquitectura MLOps evaluada en entrenamiento, inferencia, reporting y gobierno.

El modelo incorpora costes de transacción mediante una función de fricción, penalizando rebalanceos frecuentes [43]. El coste total incluye comisiones directas y *slippage*, que representa la diferencia entre el precio objetivo y el precio de ejecución.

En perfiles de baja liquidez, el *slippage* se incrementa en el YAML para reducir cambios bruscos y favorecer estabilidad en la inversión. Esta capacidad amplía el valor analítico del caso, aunque su nivel de integración operativa no se usa como evidencia principal de transferibilidad del estándar.

Como cierre del desarrollo, los resultados técnicos se sintetizan en un capítulo específico de resultados obtenidos, donde se vinculan objetivos y competencias con evidencias verificables. A partir de ahí se analizan impacto, sostenibilidad y viabilidad en términos operativos.

5. Resultados obtenidos

Este capitulo sintetiza los resultados obtenidos tras el desarrollo y los vincula con los objetivos y competencias planteados. Las evidencias se apoyan en los repositorios y en los artefactos operativos descritos en el Capítulo 4 y en sus secciones internas.

5.1 Cumplimiento de objetivos

El grado de cumplimiento de objetivos se evalua en terminos operativos, considerando la trazabilidad de ejecuciones, la reutilizacion de contratos y la transferibilidad entre casos. La Tabla 5.1 resume la relacion entre objetivos y evidencias verificables.

En lectura de madurez, los resultados sitúan la solución en una franja alta para el contexto corporativo del proyecto: equivalente a **Google Nivel 1–2** y **Microsoft Nivel 3–4**, al combinar pipelines reproducibles, CI/CD por entornos, registro de artefactos y gobierno de lifecycle en Teradata (Tabla 2.5, Tabla 4.9, Tabla 4.11 y Figura 4.7) [2, 9, 12].

Tabla 5.1: *Matriz de objetivos y evidencias del TFM.*

Objetivo	Evidencia	Repositorio
OE1 — Operativizar Chronos Engine end-to-end: estandarizar entrenamiento, inferencia, backtesting y reporting en AML, garantizando trazabilidad por `run_id` y versionado de datos/modelos.	Pipelines y componentes ejecutables	`TYM.RFP_pricing`
OE2 — Extraer un estandar reutilizable (plantilla MLOps): definir una estructura de repositorio, contratos de datos, componentes y pipelines base, y una arquitectura CI/CD reproducible entre DEV/PRE/PRO.	Plantilla reutilizable y CI/CD	`TYM.MLOpsTemplate`
OE3 — Validar la transferibilidad del estandar en un segundo caso: implementar Asset Allocation sobre la plantilla y demostrar compatibilidad de contratos, despliegue y gobernanza con minimo retrabajo.	Asset Allocation operativizado	`TYM.AssetAllocation`
OE4 — Medir el impacto operativo del estandar: comparar esfuerzo/tiempo antes-después con un metodo explicito (tareas incluidas, supuestos y limitaciones) y reportar el resultado.	Metodologia y tabla comparativa documentadas	Esta memoria
OE5 — Migrar a Azure ML SDK v2: adaptar herramientas y scripts de gestion de recursos y despliegue a `azure.ai.ml`, garantizando compatibilidad con el ciclo MLOps.	Devtools y utilidades SDK v2	`TYM.RFP_pricing`
OE6 — Gobernanza de modelos e inferencias en Teradata: integrar el registro y el seguimiento del ciclo de vida en procedimientos corporativos para asegurar trazabilidad de modelos y ejecuciones.	Ciclo de vida con registro y llamadas a Teradata	`TYM.MLOpsTemplate`

5.1.1 OE1: operativizacion end-to-end de Chronos Engine

La evidencia de OE1 se observa en la operativizacion completa de Chronos Engine, con contratos, pipelines y reporting documentados en el capitulo de desarrollo (Tabla 4.1, Figura 4.1, Figura 4.2 y Tabla 4.5). Estos artefactos confirman trazabilidad y versionado en AML con registro en MLflow [3, 9].

La estandarizacion se verifica tambien en la definicion de recursos y convenciones comunes del entorno de ejecucion (Tabla 4.2), lo que permite reproducir entrenamientos y ejecuciones sin depender de configuraciones locales. El reporting operativo y los mecanismos de recuperacion documentados en desarrollo aportan evidencia de que el ciclo de vida es auditable y coherente con las exigencias de gobierno del proyecto [4, 9].

Métrica y límite de validez. Como métrica operativa principal se usa el volumen estructurado de ejecución (44 pipelines y 25 componentes en Chronos). Esta métrica muestra cobertura de operativización, pero no mide por sí sola calidad predictiva o impacto económico, por lo que se interpreta como evidencia de industrialización técnica y no como indicador único de desempeño final.

5.1.2 OE2: estandar reusable y arquitectura CI/CD

La evidencia de OE2 se apoya en la plantilla reusable, su estructura por capas y la definicion de contratos y pipelines base (Tabla 4.8, Tabla 4.9 y Tabla 4.10), asi como en la gobernanza del ciclo de vida (Tabla 4.11). Estos resultados respaldan la capacidad de estandarizar productivizacion y CI/CD de forma reproducible en un marco corporativo homogéneo [2, 4].

Ademas, la documentacion y los mecanismos de registro y promocion (Figura 4.5, Figura 4.6 y Figura 4.7) evidencian un flujo controlado de versiones y despliegue, alineado con buenas practicas de trazabilidad y control de cambios en entornos corporativos [2, 9].

Métrica y límite de validez. Se toma como métrica de estandarización la existencia de 3 pipelines y 11 componentes base reutilizables. El límite es que la reutilización se contrasta en el contexto tecnológico de la organización; no se evalúa portabilidad completa a un stack cloud distinto.

5.1.3 OE3: transferencia a Asset Allocation

La evidencia de OE3 se valida en el despliegue de Asset Allocation sobre la misma columna vertebral tecnica, con pipelines propios y contratos compatibles (Figura 4.10, Figura 4.11 y Tabla 4.14). La compatibilidad de configuracion y gobierno respalda la transferibilidad del estandar dentro del ecosistema corporativo analizado [2].

La salida de reporting y su trazabilidad operacional (Figura 4.12 y Figura 4.13) completan la evidencia de que el estandar soporta otro dominio financiero sin redisenar la infraestructura, manteniendo el mismo esquema de contratos y evidencias [9]. No obstante, en este segundo caso la validacion no cubre un flujo de backtesting end-to-end como pipeline independiente, lo que se considera una limitacion formal del alcance de transferencia.

Métrica y límite de validez. La validación de transferencia se apoya en la operación de 2 pipelines y 11 componentes para 9 perfiles de cartera. La amenaza a validez es de escala: se demuestra transferencia en un segundo caso, pero no en múltiples equipos ejecutando variantes en paralelo.

5.1.4 OE4: medicion del impacto operativo

La medicion del impacto se plantea en unidades operativas comparables: semanas-persona de ejecucion, numero de componentes y pipelines implicados, y numero de iteraciones de despliegue en DEV/PRE/PRO. El objetivo no es fijar una causalidad estricta, sino explicitar el esfuerzo incremental cuando se reutiliza un estandar de productivizacion.

Se incluyen tareas de arquitectura, definicion de contratos, desarrollo de componentes, orquestacion, pruebas y validacion en entorno. Se excluyen permisos cloud, incidencias externas de plataforma y bloqueos administrativos que no dependen del estandar.

Tabla 5.2: *Comparativa de esfuerzo operativo entre Chronos y Asset Allocation.*

Caso	Alcance	Entregables	Esfuerzo	Dependencias y reutilizacion
Chronos Engine	Operativizacion end-to-end con pipelines por sector y reporting diario	Pipelines, componentes, contratos, registro y reporting	*Aprox.* 4 meses	Base completa de contratos y CI/CD; reutilizacion limitada (caso inicial)
Asset Allocation	Transferencia del estandar con perfiles paralelos y reporting consolidado	Pipelines, componentes parametrizados, contratos y registro	*Aprox.* 3 semanas	Reutiliza estructura, CI/CD y contratos; cambios en configuracion y logica financiera

La comparacion evidencia una reduccion del esfuerzo incremental cuando se reutiliza el estandar, manteniendo compatibilidad de contratos y gobernanza sin redisenar la columna vertebral tecnica [2, 4].

Esta medicion se reporta como evidencia operativa en terminos de esfuerzo relativo, sin atribuir causalidad estricta. Se limita a describir la diferencia observada entre un primer caso completo y una transferencia posterior bajo el mismo estandar.

Métrica y límite de validez. La métrica central es tiempo/esfuerzo relativo (aprox. 4 meses frente a aprox. 3 semanas). El límite es que hay factores exógenos (aprendizaje acumulado, disponibilidad de datos y permisos) que también influyen en la diferencia.

5.1.5 OE5: migracion a Azure ML SDK v2

La evidencia de OE5 se observa en la adopcion de `azure.ai.ml` y credenciales de identidad en las utilidades de despliegue y registro del repositorio de Chronos. Las herramientas de *devtools* y los utilitarios de conexion documentan el uso del SDK v2 para gestionar componentes, recursos y ejecuciones en AML.

Métrica y límite de validez. La métrica utilizada es funcional (compatibilidad operativa de utilidades y pipelines con SDK v2). La limitación es que no se ha realizado benchmark de rendimiento comparativo frente a SDK v1, al no formar parte del alcance.

5.1.6 OE6: gobernanza de modelos e inferencias en Teradata

La evidencia de OE6 se apoya en el ciclo de vida centralizado que integra MLflow con procedimientos en Teradata para registrar modelos, promociones e inferencias. Esta integracion aporta trazabilidad corporativa y persistencia de evidencias operativas dentro del entorno de datos institucional.

Métrica y límite de validez. La métrica de cumplimiento es estructural: existencia de acciones de lifecycle trazables (registro, promoción, inferencia, fallback) y persistencia de evidencia en base corporativa. La limitación es que no se cubre en esta memoria una auditoría longitudinal multi-anual de esos registros.

5.2 Competencias alcanzadas

La evidencia tecnica del proyecto permite vincular las competencias del master con resultados verificables en plataforma. En el ambito tecnico, el diseno de pipelines, la orquestacion y la trazabilidad en AML confirman el dominio de arquitectura y automatizacion en entornos MLOps [3, 4, 9].

En terminos de competencias tecnicas, la definicion de contratos y artefactos entre fases, la estandarizacion de recursos AML y la integracion de MLflow como registro central aportan evidencia de capacidad para disenar soluciones reproducibles, auditables y coherentes con las restricciones de un entorno regulado [2, 4, 9].

En competencias transversales, la elaboracion de documentacion operativa, el seguimiento de despliegues por entorno y la coordinacion con equipos de negocio evidencian la capacidad de estructurar trabajo, comunicar decisiones tecnicas y justificar resultados con evidencias verificables.

Tabla 5.3: *Desglose de competencias alcanzadas y evidencias asociadas.*

Competencia	Justificacion y evidencia
M2N104	Diseno de arquitecturas y flujos de datos reproducibles en AML, con contratos y artefactos entre fases (Tabla 4.1 y Tabla 4.10) que garantizan control de entradas/salidas y trazabilidad.
M2N110	Orquestacion de pipelines con versionado, registros y control de entornos; evidencia en Chronos y plantilla (Figura 4.1, Figura 4.2 y Tabla 4.9).
M2N111	Gobernanza del ciclo de vida y auditabilidad de ejecuciones mediante MLflow y persistencia en Teradata (Tabla 4.11 y Figura 4.7).
CTFM01	Capacidad de comunicar y documentar trabajo tecnico con guias operativas (`README.md`, `USER-GUIDE.md`, `AZURE-DOCS.md`).
CTFM02	Coordinacion con negocio y trazabilidad de la operacion por entornos; controles de promocion y despliegue descritos en el desarrollo.
CTFM03	Justificacion de resultados e impacto con criterios operativos verificables, apoyado en la comparativa de esfuerzo (Tabla 5.2).

5.3 Resultados tangibles tras el desarrollo

Como resultado operativo quedan activos y reutilizables los siguientes elementos:

- Trazabilidad y auditoría de ciclo de vida con registro en MLflow y persistencia en Teradata (Tabla 4.11 y Figura 4.7) [9].
- Reducción de tiempos de arranque por reutilización de plantilla `TYM.MLOpsTemplate` y CI/CD homogéneo (Tabla 4.8, Tabla 4.9 y Tabla 5.2) [2].
- Reducción de errores de integración gracias a contratos de datos y artefactos en YAML (Tabla 4.1 y Tabla 4.10) [2, 4].
- Reutilización demostrada en segundo caso de negocio sin rediseñar la infraestructura base (Tabla 4.15 y Figura 4.11).
- Documentación operativa (`README.md`, `USER-GUIDE.md`, `AZURE-DOCS.md`) como soporte de transferencia y mantenimiento.

Estos resultados tangibles se vinculan con los objetivos del TFM al aportar evidencias verificables de trazabilidad end-to-end, estabilidad contractual entre componentes y ejecución registrada en plataforma [2, 4].

Estos resultados permiten interpretar el impacto y la sostenibilidad desde una base objetiva. El capitulo siguiente analiza dichas implicaciones en salud, ODS, entorno industrial y viabilidad economica.

6. Impacto, sostenibilidad y viabilidad

Este capitulo conecta el trabajo tecnico del TFM con sus implicaciones practicas: condiciones de trabajo, impacto en ODS, efectos organizativos de la estandarizacion MLOps y criterios de sostenibilidad operativa. El objetivo no es hacer promesas de impacto abstractas, sino dejar trazabilidad de que cambia en la operacion y que limites tiene ese cambio en esta fase.

6.1 Implicaciones sobre salud y seguridad en el trabajo

El proyecto se ha desarrollado en un entorno digital con riesgo fisico directo bajo, pero con dos grupos de riesgos relevantes: ergonomicos y tecnico-operativos.

En el plano ergonomico, los riesgos principales son fatiga visual, sobrecarga postural y exposicion prolongada a pantalla, tipicos de tareas intensivas de desarrollo, validacion y redaccion tecnica. Aunque no son riesgos de alta gravedad inmediata, si pueden afectar productividad y calidad de trabajo cuando se acumulan durante periodos largos.

En el plano tecnico-operativo, los riesgos mas relevantes han sido errores de configuracion cloud, uso incorrecto de permisos y gestion de secretos deficiente en flujos CI/CD y AML. Este tipo de riesgo no impacta solo en el sistema. Tambien incrementa carga de trabajo, retrabajo y presion operativa sobre el equipo.

Como medidas preventivas aplicadas durante el TFM:

- segregacion por entornos (DEV, PRE, PRO) para reducir impacto de errores en produccion;
- principio de minimo privilegio en accesos y uso de secretos centralizados;
- validacion previa en desarrollo antes de promocionar cambios;
- estandarizacion de contratos de pipeline para reducir fallos por ambiguedad tecnica.

Este enfoque esta alineado con principios de mejora continua y control preventivo aplicados al contexto de trabajo digital [44].

6.2 Impacto en los ODS

La contribucion del trabajo a los ODS es indirecta y se evalua de forma cualitativa. No se plantea como impacto social o ambiental final cuantificado, sino como mejora de procesos tecnicos en trazabilidad, eficiencia operativa y gobernanza del ciclo de vida MLOps [45–47].

En ODS 9, la aportacion se relaciona con la consolidacion de una infraestructura analitica mas estable mediante contratos, componentes y pipelines reutilizables. En ODS 8, el efecto se vincula a la reduccion de retrabajo y a la mejora de condiciones de trabajo en equipos tecnicos por mayor estandarizacion de despliegues y operacion. En ODS 16, el valor principal es el refuerzo de transparencia interna mediante versionado, evidencias de ejecucion y control de promociones por entorno [45–47].

Para seguimiento futuro, se propone medir indicadores estables por trimestre (tiempo medio de despliegue, tasa de ejecuciones exitosas en pipeline y esfuerzo manual por release). Estos indicadores permitirian convertir la contribucion a ODS en evidencia operativa comparable entre iteraciones y vinculable a hitos de negocio relevantes [45, 47].

Para reforzar el encaje con la rúbrica, la Tabla 6.1 conecta cada ODS con metas concretas y una línea causal operativa verificable.

Tabla 6.1: *ODS priorizados, metas y relación causal con el proyecto.*

ODS	Meta de referencia	Aporte del proyecto	Cómo se verifica
ODS 9	9.5 (capacidades tecnológicas)	Estandariza industrialización de modelos con contratos y pipelines reutilizables	Tiempo de arranque por caso y estabilidad del despliegue
ODS 8	8.2 (productividad)	Reduce retrabajo técnico y dependencia de tareas manuales repetitivas	Esfuerzo manual por release y ratio de incidencias por integración
ODS 16	16.6 (instituciones eficaces y transparentes)	Refuerza trazabilidad de decisiones técnicas y evidencia auditable de ejecuciones	Porcentaje de ejecuciones con versión y artefactos trazables

Para dar consistencia a esta medicion, se propone un cuadro minimo de indicadores cualitativos y cuantitativos que no dependen de estimaciones externas. La Tabla 6.2 resume una propuesta de indicadores operativos vinculados a cada ODS relevante, que permitiria evaluar impacto sin introducir cifras no verificadas.

Tabla 6.2: *Indicadores operativos propuestos para seguimiento de impacto en ODS.*

ODS	Indicador	Justificación
ODS 9	Tiempo medio de despliegue por modelo	Captura capacidad de productivización y reutilización
ODS 8	Esfuerzo manual por release	Relaciona automatización con condiciones de trabajo
ODS 16	Tasa de ejecuciones trazables con versión	Evalúa transparencia y rendición de cuentas

La Tabla 6.3 traduce estos indicadores en un plan de actuación operativo y calendarizable.

Tabla 6.3: *Plan de actuación para mejorar el impacto en ODS.*

ODS	Actuación	Horizonte	Resultado esperado
ODS 9	Extender plantilla a nuevos casos con checklist de adopción	Corto/medio plazo	Menor variabilidad técnica de arranque y mayor reutilización
ODS 8	Automatizar controles de contratos y calidad en predespliegue	Corto plazo	Reducción de retrabajo y de incidencias evitables
ODS 16	Consolidar cuadro trimestral de trazabilidad y auditoría de ejecuciones	Medio plazo	Mayor transparencia y capacidad de rendición de cuentas

6.3 Impacto social, medioambiental e industrial

El impacto social y organizativo es principalmente interno: mejora coordinacion entre perfiles tecnicos, reduce ambiguedad en despliegues y facilita transferencia de conocimiento entre proyectos al trabajar con una estructura comun. En la practica, esto eleva la calidad operativa aunque no implique un impacto social directo de cara al cliente final en esta fase.

Este impacto interno tiene dos componentes. Por un lado, la estandarizacion reduce el esfuerzo cognitivo asociado a arrancar proyectos nuevos. Por otro, mejora la colaboracion entre equipos al compartir un mismo lenguaje tecnico de contratos, artefactos y flujos de trabajo. Estos elementos se alinean con la mejora de condiciones de trabajo y la profesionalizacion de perfiles especializados [46].

En impacto medioambiental, el efecto principal proviene del consumo de computo cloud. No se elimina ese consumo, pero puede gestionarse mejor con politicas de apagado/escalado, ejecucion bajo demanda y control de jobs innecesarios. La racionalizacion del consumo se orienta a alinear recursos con el patron real de uso sin degradar calidad de servicio [48].

El impacto industrial es el mas fuerte del capitulo: se consolida una forma estandar de productivizar modelos transferible entre casos dentro del mismo marco corporativo, sin rehacer toda la infraestructura. Esto aporta tres ventajas operativas:

- escalabilidad interna para incorporar nuevos casos con menor variabilidad tecnica;
- mayor auditabilidad al mantener contratos y lifecycle comunes;
- menor coste marginal de arranque cuando se reutiliza la plantilla.

En conjunto, el valor del trabajo no esta solo en un modelo concreto, sino en dejar una base de operacion reutilizable y gobernable para futuros proyectos de IA en entorno corporativo.

6.3.1 Impacto en la gobernanza y cultura de datos

Además del impacto técnico, el proyecto refuerza una cultura de gobierno basada en evidencias. La estandarización de contratos, el registro de artefactos y la disciplina de promoción crean hábitos de trabajo más trazables y coherentes entre equipos, reduciendo decisiones ad hoc [2, 47].

La Tabla 6.4 resume prácticas incorporadas y su efecto organizativo. El foco no es cuantificar, sino clarificar cómo se traduce MLOps en gobernanza operativa.

Tabla 6.4: *Prácticas de gobernanza y su impacto organizativo.*

Práctica	**Efecto**	**Relación con ODS**
Registro de versiones	Transparencia técnica	ODS 16
Promoción controlada	Reducción de riesgo operativo	ODS 16
Contratos de pipeline	Consistencia entre equipos	ODS 9
Documentación operativa	Transferencia de conocimiento	ODS 8

6.4 Criterios de sostenibilidad operativa a futuro

Como cierre del capitulo, conviene explicitar que significa sostener esta solucion en el tiempo sin degradar calidad tecnica ni control de costes. La sostenibilidad operativa no se limita a consumir menos recursos cloud. Tambien implica conservar trazabilidad, mantener consistencia entre entornos y evitar que la evolucion funcional rompa contratos criticos de operacion.

Un primer criterio es la **estabilidad de configuracion**. En sistemas MLOps, una parte relevante de incidencias no aparece por errores algoritmicos, sino por desviaciones de parametria, secretos o permisos entre entornos. Por ello, mantener plantillas declarativas y validaciones tempranas en despliegue sigue siendo una medida de sostenibilidad tecnica de alto impacto [3, 4].

Un segundo criterio es la **sostenibilidad del conocimiento**. Cuando la continuidad depende de conocimiento implicito en pocas personas, el coste real del sistema aumenta aunque el coste cloud se mantenga estable. Documentacion operativa, convenciones compartidas y contratos explicitos reducen ese riesgo y permiten que nuevos miembros del equipo puedan operar el ciclo de vida con menor friccion.

El tercer criterio es la **disciplina de medicion**. Para evitar discusiones basadas en percepciones, resulta util consolidar un cuadro minimo de indicadores por trimestre: tasa de ejecuciones exitosas, incidencias por tipo, tiempo de recuperacion y consumo por categoria de recurso. Esta informacion no busca burocratizar la operacion, sino mejorar la capacidad de priorizar inversiones tecnicas con evidencia comparable.

La Tabla 6.5 propone un conjunto de indicadores mínimos para seguimiento operativo. Se trata de métricas que pueden obtenerse sin introducir datos nuevos, apoyándose en evidencias existentes del pipeline y de la plataforma [2, 48].

Tabla 6.5: *Indicadores mínimos para sostenibilidad operativa.*

Indicador	**Fuente**	**Uso**
Tasa de ejecuciones exitosas	Logs de pipeline	Estabilidad operativa por entorno
Tiempo medio de recuperación	Histórico de incidencias	Priorizar mejoras de resiliencia
Consumo por tipo de recurso	Azure Cost Analysis	Control presupuestario y eficiencia
Retrabajo por release	Historial de despliegues	Medir fricción del proceso

Por ultimo, la sostenibilidad exige un equilibrio explicito entre velocidad de entrega y control. En fases de crecimiento, la presion por incorporar nuevos casos puede llevar a decisiones rapidas que incrementan deuda tecnica. El aprendizaje de este TFM sugiere el enfoque contrario: crecer sobre una base estable y gobernada produce un escalado mas robusto y, en el medio plazo, mas eficiente en coste total de operacion.

Con estos criterios de impacto y sostenibilidad establecidos, el capítulo siguiente desarrolla la memoria económica para cuantificar costes directos y criterios de imputación. Esta continuidad permite vincular el valor operativo con su viabilidad financiera.

7. Memoria económica

Este capítulo sintetiza el análisis económico del TFM a partir de costes directos observados durante el periodo septiembre–marzo. El objetivo es aportar trazabilidad de gasto real y evitar estimaciones especulativas que no puedan ser verificadas.

7.1 Costes directos observados

La base del análisis se apoya en informes de costes internos. En la Figura 7.1 se recoge el acumulado del grupo de recursos del entorno MLOps en Azure Cost Management. La Figura 7.2 muestra el detalle del recurso *ml workspace 2*, con un coste acumulado de 1.349,92 EUR (capturas anonimizadas por confidencialidad interna).

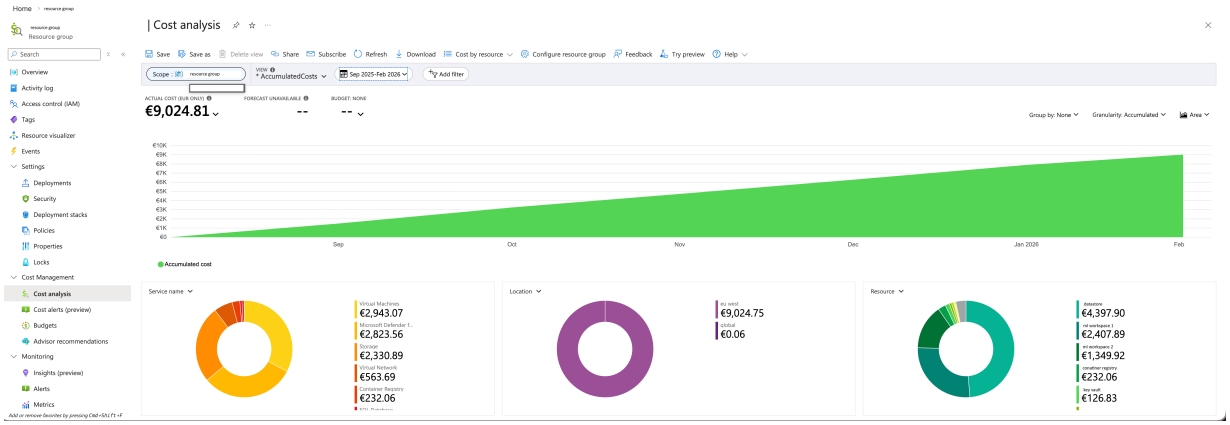

Figura 7.1: *Análisis de costes acumulados en el grupo de recursos del entorno MLOps.*

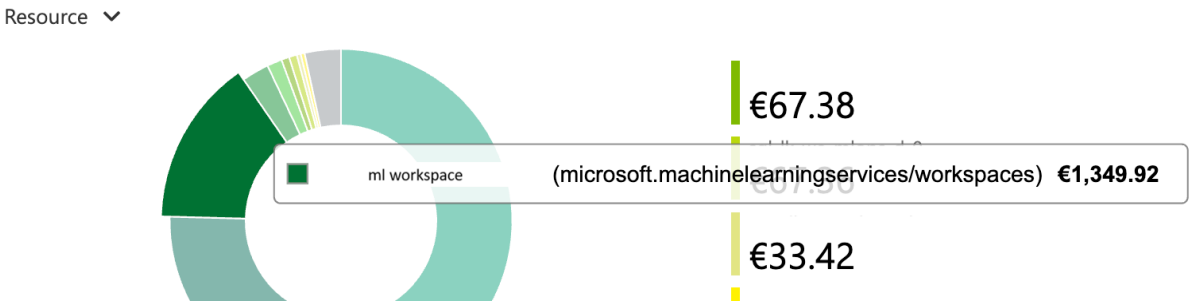

Figura 7.2: *Detalle de coste del workspace ml workspace 2 en Azure Cost Analysis (nombres corporativos censurados).*

Cabe señalar que, según los informes corporativos, el uso total del *datastore* de Laboral Kutxa asciende a 4.397,90 EUR. Dado que este recurso es compartido por múltiples departamentos y proyectos, para efectos de esta memoria se ha realizado una aproximación de 500,00 EUR como coste imputable al entrenamiento y operación de los modelos específicos del TFM. Esta cifra se interpreta como estimación proporcional del volumen total almacenado.

Además del coste cloud, se imputa el coste de dedicación del proyecto con salario bruto mensual de 1.381,20 EUR durante 6,5 meses, lo que equivale a 8.977,80 EUR. Esta valoración se incorpora para reflejar el esfuerzo directo del proyecto en términos de recursos humanos.

7.2 Resumen de costes directos

La Tabla 7.1 consolida los costes directos observados. El total directo imputado asciende a 10.827,72 EUR.

Tabla 7.1: *Coste directo observado del proyecto (periodo septiembre–marzo).*

Concepto	**Importe (EUR)**	**Fuente**
Coste cloud del workspace *ml workspace 2*	1.349,92	Azure Cost Analysis
Almacenamiento y Datastore (aproximación TFM)	500,00	Azure Storage Account
Coste de dedicación del estudiante (6,5 meses)	8.977,80	Coste salarial bruto
Total directo imputado	**10.827,72**	Suma de costes directos

7.3 Valoración de viabilidad y ROI

La viabilidad económica se valora, en esta memoria, desde una perspectiva operativa: el coste directo observado se asume como inversión para habilitar una operación gobernada y reproducible (pipelines, trazabilidad, registro de modelos y reporting), alineada con principios *FinOps* de control sobre consumo real [48].

Respecto al Retorno de la Inversión (ROI), no se presenta una cuantificación cerrada en términos monetarios porque no se dispone en esta fase de una serie histórica consolidada de beneficios económicos atribuibles de forma aislada al proyecto. Por tanto, el análisis debe interpretarse como coste directo trazable y base para evaluación posterior del retorno cuando exista evidencia financiera comparable.

8. Conclusiones y trabajo futuro

Este capítulo presenta una síntesis de los resultados, el grado de cumplimiento de objetivos y las líneas de evolución que se derivan del trabajo realizado. Se organiza en tres partes: conclusiones técnicas, valoración personal y propuestas de mejora futura.

8.1 Conclusiones

Las conclusiones se construyen a partir de los resultados sintetizados en el Capítulo 5, pero su foco es interpretar su significado para la operacion MLOps en banca. La evidencia sugiere que el resultado relevante no depende de un modelo aislado, sino de la capacidad de sostener un ciclo de vida auditable con contratos, trazabilidad y control de cambios [2, 4].

La aportacion principal es metodologica: convertir decisiones tacitas en contratos tecnicos verificables y separar la logica de negocio de la orquestacion y del despliegue. Esta separacion no es solo una preferencia de diseño, sino una condicion para mantener continuidad operativa cuando se incorporan nuevos modelos o cambian dependencias tecnicas [7–9].

En el plano tecnico, la migracion a Azure ML SDK v2 y la gobernanza de modelos e inferencias en Teradata consolidan la base operativa del estandar: por un lado, se alinea el stack con el SDK vigente para despliegues y recursos; por otro, se asegura trazabilidad corporativa con registro y seguimiento en el entorno de datos institucional.

En terminos de generalizacion, la plantilla y la gobernanza descritas son transferibles a otros modelos financieros con requisitos similares de trazabilidad y evidencia dentro de un marco corporativo homogéneo. Sin embargo, su eficacia depende del contexto organizativo: disponibilidad de entornos segregados, disciplina de control de cambios y soporte de plataforma corporativa.

Los limites del trabajo se concentran en el alcance de la validacion: se prueba la transferibilidad en un segundo caso, pero no se evalua a escala de cartera completa ni se mide el impacto longitudinal de la operacion en periodos extensos. Ademas, en Asset Allocation la validacion se centra en entrenamiento, inferencia, reporting y gobierno, sin cubrir backtesting end-to-end como pipeline independiente. Por ello, la evidencia es suficiente para sostener la viabilidad del estandar en el contexto analizado, pero no para concluir su rendimiento en escenarios de alta variabilidad o con multiples equipos en paralelo [2, 8].

Respecto al impacto operativo, la comparativa entre casos aporta evidencia de reduccion relativa de esfuerzo incremental, pero no constituye una validacion causal-experimental. La interpretacion debe entenderse como resultado observado en el marco temporal y organizativo del proyecto.

En una lectura mas reflexiva, el trabajo confirma que la madurez MLOps es tanto organizativa como tecnica: el rendimiento sostenido depende de procesos verificables, documentacion operativa y gobernanza de ciclo de vida, no solo de optimizar algoritmos. Esta conclusion conecta con la literatura y con las evidencias del proyecto, y establece una base robusta para la evolucion futura [2, 9].

Con la evidencia disponible, el nivel alcanzado por la plantilla y su transferencia se puede ubicar en una franja **Google Nivel 1–2 / Microsoft Nivel 3–4**: existe automatizacion real de entrenamiento/inferencia, despliegue por entornos y trazabilidad de lifecycle, pero la memoria no acredita aun un cierre longitudinal completo de operacion a escala multi-equipo (Tabla 2.5, Tabla 4.9, Tabla 4.11 y Tabla 5.2) [2, 8, 12].

Conclusión por objetivo específico

OE1. Se demuestra operativización end-to-end en Chronos con contratos, pipelines y reporting trazable (evidencia: Tabla 4.1, Figura 4.1, Figura 4.2). *Limitación*: la evidencia se concentra en un caso principal. *Implicación*: base operativa real para ejecución diaria auditable.

OE2. Se consolida una plantilla reusable con CI/CD y contratos estables (evidencia: Tabla 4.8, Tabla 4.10). *Limitación*: validación en un contexto cloud corporativo específico. *Implicación*: reducción de coste de arranque técnico en nuevos proyectos.

OE3. La transferibilidad se valida en Asset Allocation sin rediseñar infraestructura base (evidencia: Figura 4.10, Figura 4.11, Tabla 4.15). *Limitación*: no se cubre backtesting end-to-end como pipeline independiente. *Implicación*: el estándar es reutilizable con adaptación mínima en la capa de negocio.

OE4. Se observa reducción relativa de esfuerzo entre primer caso y transferencia (evidencia: Tabla 5.2). *Limitación*: resultado no causal-experimental. *Implicación*: existe señal operativa de mejora en productividad técnica.

OE5. Se migra la operativa a Azure ML SDK v2 (evidencia: utilidades y despliegues documentados en desarrollo/resultados). *Limitación*: sin benchmark de rendimiento frente a SDK anterior. *Implicación*: alineación tecnológica con stack vigente y menor riesgo de obsolescencia.

OE6. Se integra gobernanza de modelos e inferencias en Teradata (evidencia: Tabla 4.11, Figura 4.7). *Limitación*: falta serie histórica larga para análisis longitudinal. *Implicación*: trazabilidad corporativa verificable de decisiones de lifecycle.

8.2 Valoración personal y competencias adquiridas

Desde una perspectiva formativa, el proyecto ha permitido consolidar la idea de que la adopción efectiva de IA requiere una integración estrecha entre ciencia de datos e ingeniería de software. A continuación, se resume cómo el trabajo responde a las competencias del máster:

- **Técnicas**: Se han cumplido las competencias **M2N104**, **M2N110** y **M2N111**, evidenciando un flujo de datos profesional, una arquitectura escalable y una automatización efectiva del despliegue.
- **Transversales**: Se han alcanzado las competencias **CTFM01**, **CTFM02** y **CTFM03** mediante la redacción de este informe, la integración en el equipo de Tesorería y el análisis de los impactos económicos e industriales.

La ejecucion del proyecto tambien ha requerido un equilibrio continuo entre rigor tecnico y adaptacion organizativa. La coordinacion con equipos de negocio, la gestion de permisos y la resolucion de incidencias en entorno cloud han supuesto un aprendizaje practico en gestion de cambios y comunicacion tecnica, aspectos esenciales para consolidar autonomia profesional en entornos reales.

En términos personales, el trabajo ha reforzado la importancia de documentar decisiones y de convertir buenas prácticas en contratos técnicos estables. Esta disciplina reduce fricciones operativas recurrentes y mejora la capacidad de justificar técnicamente cada despliegue ante perfiles no técnicos.

En terminos academicos, el trabajo ha reforzado la importancia de justificar decisiones de arquitectura, explicitar supuestos y documentar evidencias verificables. Este enfoque ha sido clave para vincular objetivos, resultados y conclusiones de forma coherente, evitando que la memoria se limite a una descripcion operativa sin trazabilidad.

8.3 Líneas futuras

El desarrollo del proyecto abre líneas de evolución que combinan madurez técnica, automatización y gobierno operativo. Para evitar un listado aspiracional, la Tabla 8.1 organiza estas líneas como backlog priorizado con prerequisitos, complejidad, dependencia, riesgo y horizonte temporal.

Tabla 8.1: *Backlog de líneas futuras con viabilidad operativa.*

Línea futura	Prerequisito	Complejidad	Dependencia	Riesgo	Horizonte
Pruebas de integración completas	Base de tests unificada por plantilla	Media	Equipo MLOps	Medio	Corto plazo
Auto-promoción con criterios de negocio	Histórico de métricas estable	Alta	Datos + negocio	Alto	Medio plazo
Escalado de plantilla a más modelos	Gobierno de versiones de plantilla	Media	Equipos de Tesorería	Medio	Medio plazo
Gobernanza formal de plantillas	Comité técnico y criterios de aprobación	Media	Arquitectura / IT	Bajo-Medio	Corto / medio plazo
Monitorización longitudinal	Data mart operativo de métricas e inferencias	Alta	Plataforma de datos	Medio	Medio/largo plazo
Automatización de distribución de reportes	Canal seguro validado por cumplimiento	Media	Negocio + seguridad	Medio	Corto / medio plazo
Integración más estable con fuentes corporativas	Contratos de datos y SLA con proveedores internos	Alta	Datos corporativos	Alto	Largo plazo

Estas líneas futuras se alinean además con la evolución normativa del AIA, donde la trazabilidad, la documentación de ciclo de vida y el control de riesgos pasan de recomendación a requisito operativo progresivo [11, 39].

Estas lineas futuras estan alineadas con la necesidad de mantener un equilibrio entre velocidad de entrega y control de riesgos. El objetivo es consolidar una operacion que siga siendo reutilizable, auditable y sostenible conforme se incorporen nuevos modelos y fuentes de datos.

Como cierre, el siguiente salto de madurez pasa por institucionalizar el gobierno de la plantilla: definir versiones, compatibilidad entre proyectos y criterios de aceptación antes de promover cambios a producción. Este enfoque se alinea con recomendaciones de MLOps sobre control de versiones y automatización de evidencias [2, 8].

Para completar la trazabilidad documental, los anexos recopilan el pliego de condiciones, guías de uso y un glosario de términos. Esta sección facilita la consulta rápida de definiciones y documentación de soporte que complementan la memoria principal.

Bibliografía

[1] D. Kreuzberger, N. Kuehl, and S. Hirschl, "Machine learning operations (mlops): Overview, definition, and architecture," *IEEE Access*, 2023. Accedido: 6 mar. 2026.

[2] Microsoft, "Mlops: Machine learning operations." `https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/mlops-machine-learning`, 2025. Accedido: 1 mar. 2026.

[3] Microsoft, "What are azure machine learning pipelines?." `https://learn.microsoft.com/en-us/azure/machine-learning/concept-ml-pipelines`, 2025. Accedido: 1 mar. 2026.

[4] Microsoft, "Mlops best practices for azure machine learning." `https://learn.microsoft.com/en-us/azure/machine-learning/concept-model-management-and-deployment`, 2025. Accedido: 1 mar. 2026.

[5] LABORAL Kutxa, "Información corporativa de laboral kutxa." `https://www.laboralkutxa.com/`, s.f. Accedido: 11 mar. 2026.

[6] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann, "Software engineering for machine learning: A case study," in *Proceedings of the 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019. Accedido: 6 mar. 2026.

[7] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015. Accedido: 6 mar. 2026.

[8] E. Breck, S. Cai, E. Nielsen, M. Salib, and D. Sculley, "The ml test score: A rubric for ml production readiness and technical debt reduction," in *IEEE International Conference on Big Data*, 2017. Accedido: 6 mar. 2026.

[9] MLflow Authors, "Mlflow documentation." `https://mlflow.org/docs/latest/index.html`, 2025. Accedido: 1 mar. 2026.

[10] European Union, "Regulation (eu) 2016/679 (general data protection regulation)." `https://eur-lex.europa.eu/eli/reg/2016/679/oj`, 2016. Accedido: 1 mar. 2026.

[11] European Union, "Regulation (eu) 2024/1689 laying down harmonised rules on artificial intelligence (ai act)." https://eur-lex.europa.eu/eli/reg/2024/1689/oj, 2024. Accedido: 9 mar. 2026.

[12] Google Cloud, "Mlops: Continuous delivery and automation pipelines in machine learning." https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning, 2025. Accedido: 13 mar. 2026.

[13] Amazon Web Services, "Amazon sagemaker pipelines developer guide." https://docs.aws.amazon.com/sagemaker/latest/dg/pipelines.html, 2026. Accedido: 6 mar. 2026.

[14] Google Cloud, "Vertex ai pipelines overview." https://cloud.google.com/vertex-ai/docs/pipelines/introduction, 2026. Accedido: 6 mar. 2026.

[15] Kubeflow Authors, "Kubeflow documentation overview." https://www.kubeflow.org/docs/started/introduction/, 2026. Accedido: 6 mar. 2026.

[16] K. Schwaber and J. Sutherland, "The scrum guide: The definitive guide to scrum." https://scrumguides.org/scrum-guide.html, 2020. Accedido: 12 mar. 2026.

[17] D. J. Anderson, A. Carmichael, K. Leopold, M. Poppendieck, and T. Poppendieck, "The kanban guide." https://kanbanguides.org/guide/, 2020. Accedido: 12 mar. 2026.

[18] Microsoft, "Manage variable groups in azure pipelines." https://learn.microsoft.com/en-us/azure/devops/pipelines/library/variable-groups, 2026. Accedido: 11 mar. 2026.

[19] Microsoft, "What is an azure machine learning workspace?." https://learn.microsoft.com/en-us/azure/machine-learning/concept-workspace, 2026. Accedido: 11 mar. 2026.

[20] Microsoft, "Azure resource manager overview." https://learn.microsoft.com/en-us/azure/azure-resource-manager/management/overview, 2026. Accedido: 11 mar. 2026.

[21] Microsoft, "Connect to data with azure machine learning datastores." https://learn.microsoft.com/en-us/azure/machine-learning/how-to-datastore, 2026. Accedido: 11 mar. 2026.

[22] Microsoft, "What is azure key vault?." https://learn.microsoft.com/en-us/azure/key-vault/general/overview, 2026. Accedido: 11 mar. 2026.

[23] Microsoft, "Compute targets in azure machine learning." https://learn.microsoft.com/en-us/azure/machine-learning/concept-compute-target, 2026. Accedido: 11 mar. 2026.

[24] Microsoft, "Environments in azure machine learning." https://learn.microsoft.com/en-us/azure/machine-learning/concept-environments, 2026. Accedido: 11 mar. 2026.

[25] Microsoft, "Components in azure machine learning." https://learn.microsoft.com/en-us/azure/machine-learning/concept-component, 2026. Accedido: 11 mar. 2026.

[26] T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III, and K. Crawford, "Datasheets for datasets," *Communications of the ACM*, vol. 64, no. 12, pp. 86–92, 2021. Accedido: 6 mar. 2026.

[27] M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. Raji, and T. Gebru, "Model cards for model reporting," in *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, 2019. Accedido: 6 mar. 2026.

[28] N. Polyzotis, M. Zinkevich, S. Roy, E. Breck, and S. E. Whang, "Data validation for machine learning," *Proceedings of Machine Learning and Systems*, vol. 1, pp. 334–347, 2019. Accedido: 13 mar. 2026.

[29] NIST, "Ai risk management framework (ai rmf 1.0)." https://www.nist.gov/itl/ai-risk-management-framework, 2023. Accedido: 13 mar. 2026.

[30] J. C. Hull, *Options, Futures, and Other Derivatives*. Pearson, 10th ed., 2018.

[31] M. Choudhry, *The Credit Default Swap Basis*. Bloomberg Press, 2006.

[32] F. A. Longstaff, S. Mithal, and E. Neis, "Corporate yield spreads: Default risk or liquidity? new evidence from the credit-default swap market," *The Journal of Finance*, vol. 60, no. 5, pp. 2213–2253, 2005. Accedido: 13 mar. 2026.

[33] R. Blanco, S. Brennan, and I. W. Marsh, "An empirical analysis of the dynamic relation between investment-grade bonds and credit default swaps," *The Journal of Finance*, vol. 60, no. 5, pp. 2255–2281, 2005.

[34] E. J. Elton, M. J. Gruber, D. Agrawal, and C. Mann, "Explaining the rate spread on corporate bonds," *The Journal of Finance*, vol. 56, no. 1, pp. 247–277, 2001.

[35] J. Y. Campbell and G. B. Taksler, "Equity volatility and corporate bond yields," *The Journal of Finance*, vol. 58, no. 6, pp. 2321–2350, 2003. Accedido: 13 mar. 2026.

[36] R. C. Grinold and R. N. Kahn, *Active Portfolio Management*. McGraw-Hill, 2nd ed., 2000.

[37] Jefatura del Estado (Espana), "Ley organica 3/2018, de proteccion de datos personales y garantia de los derechos digitales." https://www.boe.es/buscar/act.php?id=BOE-A-2018-16673, 2018. Accedido: 9 mar. 2026.

[38] Board of Governors of the Federal Reserve System, "Sr 11-7: Supervisory guidance on model risk management." https://www.federalreserve.gov/supervisionreg/srletters/sr1107.htm, 2011. Accedido: 12 mar. 2026.

[39] European Commission, "Ai act: Timeline and implementation milestones." https://digital-strategy.ec.europa.eu/en/policies/regulatory-framework-ai, 2026. Accedido: 13 mar. 2026.

[40] G. Antonacci, "Risk premia harvesting through dual momentum." Whitepaper. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2042750, 2017. Accedido: 13 mar. 2026.

[41] W. J. Keller and J. W. Keuning, "Protective asset allocation (paa): A simple momentum-based alternative for term deposits." Technical report. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=2759734, 2016. Accedido: 13 mar. 2026.

[42] G. He and R. Litterman, "The intuition behind black-litterman model portfolios." Goldman Sachs Asset Management. https://www.scirp.org/reference/referencespapers?referenceid=3252213, 2002. Accedido: 13 mar. 2026.

[43] L. Chen, D. A. Lesmond, and J. Wei, "Corporate yield spreads and bond liquidity," *The Journal of Finance*, vol. 62, no. 1, pp. 119–149, 2007. Accedido: 13 mar. 2026.

[44] ISO, "Iso 45001:2018 occupational health and safety management systems – requirements with guidance for use." https://www.iso.org/standard/63787.html, 2018. Accedido: 6 mar. 2026.

[45] United Nations, "Goal 9: Industry, innovation and infrastructure." https://sdgs.un.org/goals/goal9, 2026. Accedido: 6 mar. 2026.

[46] United Nations, "Goal 8: Decent work and economic growth." https://sdgs.un.org/goals/goal8, 2026. Accedido: 6 mar. 2026.

[47] United Nations, "Goal 16: Peace, justice and strong institutions." https://sdgs.un.org/goals/goal16, 2026. Accedido: 6 mar. 2026.

[48] FinOps Foundation, "Finops framework: Principles and cloud cost governance." https://www.finops.org/framework/, 2025. Accedido: 4 mar. 2026.

ANEXOS

A. Anexo A. Pliego de condiciones

A.1 Objeto del pliego

Este pliego define las condiciones tecnicas, organizativas y de validacion del TFM. Su objetivo es fijar criterios verificables para disenar, implementar y validar una solucion MLOps sobre AML orientada a la operativizacion de modelos financieros en entorno empresarial.

El pliego se adapta al alcance real del proyecto y se actualiza de forma incremental durante su ejecucion, de manera que las decisiones tecnicas y de gobierno queden trazadas y sin ambiguedad.

Asimismo, delimita el perimetro de trabajo: establece condiciones de ejecucion tecnica, criterios de aceptacion y obligaciones de trazabilidad, pero no fija objetivos de negocio cuantitativos fuera de la evidencia tecnica disponible en el proyecto.

A.2 Descripcion del servicio a desarrollar

El servicio objeto de este proyecto consiste en establecer un marco operativo reusable para el ciclo de vida de modelos financieros, incluyendo entrenamiento, inferencia, registro de modelos, registro de inferencias y salida de reporting operativo.

La solucion se estructura sobre tres bloques funcionales:

- Operativización de Chronos Engine como caso principal de producción.
- Estandar de productivizacion mediante plantillas reutilizables para nuevos casos.
- Validacion de transferibilidad con el modelo Asset Allocation.

A.3 Recursos y medios requeridos

Los medios exigidos en este pliego se definen por funcion operativa. El objetivo no es enumerar tecnologia por catalogo, sino dejar claro que pieza habilita cada capacidad critica del servicio.

Tabla A.1: *Recursos minimos requeridos por bloque operativo.*

Bloque	Medios minimos	Finalidad en el servicio
Infraestructura y plataforma	Suscripcion corporativa Azure, recursos AML, compute para jobs y build/deploy, repositorios y pipeline CI/CD.	Ejecutar pipelines de forma reproducible y promocionar cambios por entorno sin despliegues manuales ad hoc.
Datos e integraciones	Datastore y/o Azure SQL para datos financieros, integracion con Teradata, mecanismo de distribucion de reportes.	Soportar ingestiones, salida operativa y registro de gobierno de modelos e inferencias.
Software base	Python, Azure SDK v2, MLflow, definiciones declarativas de componentes/pipelines.	Implementar logica de negocio, trazabilidad de ejecuciones y despliegue versionado de activos.
Seguridad operativa	Gestion de identidades y secretos (por ejemplo, Key Vault y credenciales de servicio), separacion DEV/PRE/PRO.	Evitar credenciales en codigo, aplicar minimo privilegio y reducir riesgo por diferencias de configuracion entre entornos.

Como condicion adicional, cada flujo debe declarar de forma explicita sus entradas y salidas (train/infer/monitor), incluyendo ruta de consumo y ruta de publicacion. Esta condicion es necesaria para poder reconstruir ejecuciones y responder a revisiones tecnicas.

A.4 Requisitos tecnicos y funcionales

Los requisitos tecnicos se formulan como condiciones de diseno y operacion que deben cumplirse durante toda la ejecucion del proyecto. En particular, se exige separar logica de negocio y logica cloud, modularizar componentes con parametros por entorno y mantener contratos explicitos de entrada/-salida para cada etapa [1, 4, 7].

Ademas, la solucion debe garantizar trazabilidad completa de ejecuciones, versiones y promociones, integrar el gobierno operacional en Teradata y sostener despliegue automatizado por CI/CD con control de cambios por entorno [2, 3, 9]. Esta base se complementa con dos condiciones de calidad: reproducibilidad tecnica (entornos y parametros versionados) y observabilidad minima (estado de ejecucion, logs y metadatos de corrida).

A.4.1 Requisitos no funcionales minimos

En este pliego se consideran no funcionales, como minimo, cuatro ejes de calidad. La solucion debe ser mantenible (codigo modular y legible), auditable (trazabilidad completa de cada salida relevante), segura en operacion (minimo privilegio y secretos fuera de repositorio) y portable dentro del ecosistema interno (despliegue por parametros sin rehacer definiciones por entorno).

A.5 Pruebas y criterios de aceptacion

La aceptacion se basa en evidencia tecnica verificable. No es suficiente con la ejecucion puntual de un pipeline: debe poder demostrarse continuidad operativa, trazabilidad y capacidad de recuperacion basica ante incidencias.

A.5.1 Pruebas minimas exigibles

Tabla A.2: *Pruebas minimas y evidencia de aceptacion.*

Ambito de prueba	Comprobacion minima	Evidencia esperada
Ejecucion funcional	Ejecucion E2E de train e infer en entorno objetivo.	Job finalizado con outputs esperados y metadatos de corrida accesibles.
Contratos tecnicos	Validacion de interfaces entre componentes sin roturas de artefacto.	Registro de entradas/salidas coherente y sin ajustes manuales de ultima hora.
Gobernanza	Registro de modelo e inferencias en la capa de gobierno definida.	Evidencia de version, fecha, contexto de ejecucion y persistencia de inferencia.
Entrega operativa	Generacion y distribucion de reportes operativos.	Artefacto final disponible en ruta acordada y canal de consumo definido.
Promocion por entornos	Verificacion de aprobacion y despliegue CI/CD por entorno.	Historial de pipeline con aprobaciones y despliegues trazables.
Recuperacion basica	Reintento o re-ejecucion controlada ante fallo.	Re-ejecucion sin perdida de trazabilidad ni inconsistencia de versionado.

A.5.2 Criterio general de aceptacion

Se considera aceptado cada bloque funcional cuando existe evidencia verificable de ejecucion reproducible, trazabilidad completa de artefactos/metadata y cumplimiento de los flujos definidos sin intervenciones manuales no previstas.

Adicionalmente, la aceptacion requiere que la evidencia quede documentada en repositorio y/o memoria tecnica con referencia a la ejecucion correspondiente (job, version y fecha), de forma que el resultado pueda revisarse por terceros.

A.6 Condiciones de ejecucion y organizacion

La ejecucion del proyecto se organiza con coordinacion continua entre tutela academica y validacion tecnica en empresa, y con una gestion incremental por fases (Chronos, estandar de plantillas y Asset Allocation). Esta organizacion busca asegurar continuidad, pero tambien control de cambios y traspaso de conocimiento.

Durante la ejecución, se exige mantener registro de decisiones técnicas relevantes, revisar periódicamente el avance por hitos y gestionar riesgos de entorno (permisos, disponibilidad de datos y dependencias de plataforma). También debe mantenerse separación de responsabilidades entre desarrollo de modelo, operación de plataforma y aprobación de paso entre entornos.

A.7 Condiciones legales, seguridad y cumplimiento

La ejecucion del servicio debe cumplir la normativa aplicable en proteccion de datos (RGPD y LOPDGDD) [10, 37] y considerar el marco regulatorio de IA en lo relativo al ciclo de vida del modelo [11]. En operacion, esto se concreta en minimo privilegio, trazabilidad de operaciones y custodia de secretos fuera del codigo fuente.

Asimismo, las evidencias tecnicas de ejecucion deben conservarse conforme a las politicas internas de seguridad y cumplimiento, de forma que puedan atender revisiones internas sin reconstrucciones manuales no controladas.

A.8 Entregables y condiciones de cierre

El proyecto se considera cerrado cuando se entregan y validan memoria tecnica, pipelines operativos documentados para los casos en alcance, estandar reusable para nuevos despliegues y evidencias de gobernanza de modelos e inferencias en Teradata.

Como condición final de cierre, debe quedar disponible una guía de operación y mantenimiento que permita continuidad tras el TFM, con foco en puesta en marcha, recuperación básica y extensión controlada de la solución.

B. Anexo B. Documentacion y guias de usuario

En este anexo se resumen las documentaciones creadas para facilitar la adopcion, uso y mantenimiento de cada bloque del TFM. La estrategia seguida ha sido generar documentacion especifica por proyecto, con distinto nivel de detalle segun su objetivo.

B.1 Documentacion por proyecto

La documentacion se organiza por bloque funcional para que cada equipo pueda localizar rapidamente que leer segun la fase en la que se encuentre: adopcion inicial, operacion diaria o extension de la solucion.

B.1.1 Chronos Engine

Para Chronos Engine se ha preparado un `README.md` con el resumen de su productivizacion. Este documento describe el alcance funcional, el flujo operativo principal y las piezas tecnicas clave para entender como se ejecuta el modelo en entorno productivo.

B.1.2 Asset Allocation

Para Asset Allocation se ha preparado un `README.md` con el resumen de su productivizacion. El contenido se centra en explicar como se aplica el mismo patron operativo del estandar en un segundo caso de uso, incluyendo su flujo de ejecucion y evidencias de transferencia.

B.1.3 Estandar de productivizacion mediante plantillas

Para el estándar se han elaborado tres documentos complementarios:

- `README.md`: ofrece una vista general del repositorio, su estructura por capas y el objetivo de cada bloque.
- `USER-GUIDE.md`: guia paso a paso para configurar el repositorio y detalla que ficheros editar para utilizar el estandar en un nuevo proyecto.
- `AZURE-DOCS.md`: documentacion tecnica de Azure DevOps y Azure Machine Learning para entender desde cero conceptos como componentes, pipelines, recursos y flujo de despliegue.

B.2 Criterio de uso de las guias

El uso recomendado de estas guías es progresivo: primero consultar los `README.md` para contexto general, después seguir `USER-GUIDE.md` para una puesta en marcha operativa y, finalmente, utilizar `AZURE-DOCS.md` como referencia técnica de apoyo durante configuración, despliegue y mantenimiento.

B.3 Mapa documental por proyecto

La Tabla B.1 sintetiza los documentos clave redactados en cada bloque del TFM y su utilidad para trazabilidad y transferencia. Esta tabla no pretende duplicar contenido, sino clarificar qué evidencia aporta cada documento a la memoria.

Tabla B.1: *Mapa documental de apoyo por proyecto.*

Proyecto	**Documento**	**Aporte académico**
Chronos Engine	`README.md`	Describe el flujo operativo, comandos de ejecución y pruebas, útil para reproducibilidad.
Chronos Engine	`docs.md`	Detalla criterios de AML/MLflow, contratos y troubleshooting, útil para trazabilidad técnica.
Chronos Engine	`CHANGELOG.md`	Registra decisiones técnicas y cambios relevantes, útil para reconstrucción de evolución.
Estándar de plantillas	`README.md`	Sintetiza arquitectura y flujo de CI/CD y AML.
Estándar de plantillas	`USER-GUIDE.md`	Explicita criterios de finalización y pasos de adopción para nuevos proyectos.
Estándar de plantillas	`AZURE-DOCS.md`	Referencia técnica de recursos AML, contratos y despliegue.
Asset Allocation	`README.md`	Documenta el flujo de entrenamiento e inferencia y la política de registro.

C. Anexo C. Glosario

Este glosario recoge los terminos tecnicos usados en la memoria para mantener consistencia terminologica. Las siglas se muestran enlazadas con la seccion de abreviaturas mediante el formato AML. Para facilitar lectura no tecnica, cada termino se define en lenguaje directo y, cuando aporta valor, se añade un ejemplo breve.

C.1 Terminos clave

Artefacto Salida versionable de una etapa de pipeline usada como evidencia técnica o entrada de otra etapa. *Ejemplo:* un `metrics.json` generado tras entrenamiento.

Backtesting Evaluación histórica del comportamiento de un modelo con datos pasados para comprobar su estabilidad. *Ejemplo:* simular cómo habría funcionado una estrategia en los últimos 24 meses.

Benchmark Referencia de comparación para evaluar rendimiento de una cartera o modelo. *Ejemplo:* comparar una cartera con un índice de renta fija.

Challenger/Champion Esquema de evaluación donde un modelo candidato (challenger) se compara con el vigente (champion) antes de promoverse. *Ejemplo:* solo se promueve si mejora la métrica principal por encima de un umbral.

Componente AML Unidad reutilizable de ejecución en Azure Machine Learning con contrato de entradas/salidas y lógica encapsulada. *Ejemplo:* un componente exclusivo para preprocesado.

Contrato de componente Especificación formal de parámetros, entradas, salidas y tipos de artefacto para componer pipelines sin ambigüedad. *Ejemplo:* exigir `uri_folder` como entrada obligatoria.

Datastore Recurso de almacenamiento referenciado desde AML para lectura y escritura de datos/artefactos en pipelines. *Ejemplo:* carpeta compartida para modelos y reportes.

Duracion Medida de sensibilidad del precio de un bono ante cambios en tipos de interes; a mayor duracion, mayor variacion ante movimientos de tipos.

Endpoint Punto de despliegue para inferencia operativa, normalmente gestionado por plataforma cloud para invocacion controlada.

Estandar de productivizacion
Conjunto reusable de estructura, contratos, pipelines y practicas de despliegue para operativizar modelos de forma consistente.

Fallback de modelo
Mecanismo de contingencia para recuperar una version estable cuando una promocion o ejecucion no cumple criterios operativos.

Gobernanza de modelos
Conjunto de reglas y registros para controlar versionado, promocion, trazabilidad y uso de modelos en produccion.

Inferencia Ejecucion del modelo entrenado sobre nuevos datos para generar predicciones y metadatos de explotacion.

Lifecycle MLOps
Flujo extremo a extremo de desarrollo, despliegue, monitorización y mantenimiento de modelos de IA en entorno productivo.

Operativizacion
Proceso de llevar un modelo desde entorno analitico a operacion estable en produccion con control tecnico, trazabilidad y mantenibilidad.

Orquestacion de pipelines
Coordinacion de etapas dependientes (carga, preprocess, train, infer, registro, reporte) en un flujo ejecutable y repetible.

Pipeline Flujo automatizado de etapas de procesamiento y/o ML que transforma entradas en artefactos de salida bajo una secuencia definida.

Punto basico (bp)
Unidad equivalente a 0,01 % usada para expresar variaciones de tipos y spreads de forma estandar.

Registro de inferencias
Persistencia de resultados de inferencia y metadatos asociados para auditoria, trazabilidad y consumo operativo.

Registro de modelos
Repositorio de versiones de modelo con sus metadatos tecnicos, usado para gobierno de promocion y despliegue.

Riesgo de liquidez
Riesgo de no poder ejecutar operaciones sin impacto relevante en precio por falta de contrapartida suficiente en el mercado.

Self-promotion
Etapa automatizada de decision de promocion de modelo basada en metricas y umbrales definidos.

Teradata Plataforma corporativa usada en este proyecto para persistencia de metadatos de gobierno y registro operativo de inferencias.

CI/CD Practica de integracion y despliegue continuos para automatizar validacion, promocion y actualizacion de soluciones software.

C.2 Nota de uso

Para evitar ambiguedades en la memoria, se mantiene la distincion entre nombre de proyecto/-modelo y nombre de repositorio. En particular, se emplean Chronos Engine y Asset Allocation como denominaciones funcionales, y se evita usar nombres de repositorio como si fueran nombre del producto.

TRABAJO DE FIN DE MÁSTER TRABAJO DE FIN DE MÁSTER TRABAJO DE FIN DE MÁSTER TRABAJO DE FIN DE MÁSTER TRABAJO DE FIN DE MÁSTER TRABAJO DE FIN DE MÁSTER